
```

import os
import sys
import datetime
import numpy as np
import uvicorn
import simpy
import glob
from fastapi import FastAPI, Depends, HTTPException, BackgroundTasks
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from typing import List
from tensorboard.backend.event_processing.event_accumulator import EventAccumulator
project_root = os.path.abspath(os.path.join(os.path.dirname(__file__), '../'))
if project_root not in sys.path:
    sys.path.append(project_root)
scenario_dir = os.path.join(project_root, 'scenarios', 'order_picking')
if scenario_dir not in sys.path:
    sys.path.append(scenario_dir)
from config import Config
from database import SessionLocal, OrderPool, OrderBOM, SimulationTask, PartMaster, DispatchResult
from sb3_contrib import MaskablePPO
from scenarios.order_picking.rl_environment import PickingEnv
from core_engine.rules.dispatch_rules import DispatchRules
from core_engine.models.resource_model import SimpyStation
app = FastAPI(title="Weichai APS AI 智能排产网关")
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
TASK_PROGRESS = {}
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
class OrderItem(BaseModel):
class OrderBatch(BaseModel):
    orders: List[OrderItem]
class SimulationRequest(BaseModel):
class DBLogger:
    def __init__(self, dispatch_records, task_id, base_time):
        self.dispatch_records = dispatch_records
        self.task_id = task_id
        self.base_time = base_time
        self.temp_records = {}
    def log_event(self, time, entity_id, event_type, station_id, details=None):
        if event_type == "spawn":
            self.temp_records[entity_id] = {
                "spawn_env_time": time,
                "target_station": station_id,
                "order_id": details.get("order_id", "") if details else ""
            }
        elif event_type == "start_process":
            if entity_id in self.temp_records:
                self.temp_records[entity_id]["start_env_time"] = time
        elif event_type == "end_process":
            if entity_id in self.temp_records:
                record = self.temp_records[entity_id]
                end_env_time = time
                spawn_dt = self.base_time + datetime.timedelta(seconds=float(record["spawn_env_time"])))

```

```

start_dt = self.base_time + datetime.timedelta(seconds=float(record["start_env_time"]))
end_dt = self.base_time + datetime.timedelta(seconds=float(end_env_time))
self.dispatch_records.append(
    DispatchResult(
        task_id=self.task_id,
        order_id=record["order_id"],
        box_id=entity_id,
        target_station=record["target_station"] + 1,
        predicted_spawn_time=spawn_dt,
        predicted_start_time=start_dt,
        predicted_end_time=end_dt
    )
)
del self.temp_records[entity_id]
def set_power_status(self, *args, **kwargs):
def launch_box(env, station, box_id, order_id, p_time, t_trans, entity_type, logger, delay):
    if delay > 0:
        yield env.timeout(delay)
    if logger:
        logger.log_event(env.now, box_id, "spawn", station.station_id, {"order_id": order_id})
    try:
        yield env.process(station.process_box(box_id, order_id, p_time, t_trans, entity_type))
    except TypeError:
        yield env.process(station.process_box(box_id, p_time, t_trans, entity_type))
def simpy_dispatch_engine(env, stations, rl_env, model, optimal_stations, dispatch_records, task_id, base_time):
    db_logger = DBLogger(dispatch_records, task_id, base_time)
    for s in stations:
        s.logger = db_logger
    obs, _ = rl_env.reset(seed=999)
    energy_saving_mask = np.array([True] * optimal_stations + [False] * (Config.NUM_STATION - optimal_stations))
    done = False
    dispatch_time_cursor = 0.0
    while not done:
        try: env_internal_mask = rl_env.unwrapped.action_masks()
        except AttributeError: env_internal_mask = np.ones(Config.NUM_STATIONS, dtype=bool)
        combined_masks = np.logical_and(energy_saving_mask, env_internal_mask)
        if not np.any(combined_masks): combined_masks = energy_saving_mask
        obs_state = rl_env.unwrapped.get_obs()
        action = int(DispatchRules.rule_ai_policy(model, obs=obs_state, valid_masks=combined_masks))
        current_order = rl_env.unwrapped.real_world_orders[rl_env.unwrapped.current_step]
        target_station = stations[action]
        d_main = Config.STATION_EXIT_FAR_DISTANCES[action] - (Config.EXIT_PORT_DE
LTA / 2.0)
        t_trans = (d_main / Config.BELT_SPEED) + (Config.BRANCH_IN_LENGTH / Config.B
ELT_SPEED)
        local_cursor = dispatch_time_cursor
        if hasattr(rl_env.unwrapped, 'station_active_boxes'):
            active_boxes = [b for b in rl_env.unwrapped.station_active_boxes[action] if b['finish_time'] > local_cursor]
            active_order_ids = set(b['order_id'] for b in active_boxes)
            is_new_order = current_order['order_id'] not in active_order_ids
            if len(active_boxes) >= getattr(Config, 'MAX_BOXES_PER_STATION', 8) or (is_new_order and len(active_order_ids) >= getattr(Config, 'MAX_ORDERS_PER_STATION', 2)):
                if active_boxes:
                    local_cursor = max(local_cursor, min(b['finish_time'] for b in active_boxes))
        for box in current_order['boxes']:
            local_cursor += Config.DISPATCH_INTERVAL
            delay_before_launch = max(0, local_cursor - env.now)
            env.process(launch_box(
                env, target_station, f'{current_order["order_id"]}-P{box["sku"]}', current_order["order_id"],
                box['p_time'], t_trans, box['sku'],
                db_logger, delay_before_launch
            ))

```

```

    ))
    dispatch_time_cursor = local_cursor
    obs, _, done, _, _ = rl_env.step(action)
    while any(s.machine.count > 0 or len(s.machine.queue) > 0 for s in stations):
        yield env.timeout(1.0)
def run_simulation_task(task_id: str, batch_no: str):
    db = SessionLocal()
    try:
        TASK_PROGRESS[task_id] = {"status": "running", "progress": "10%", "message": f"正在
提取波次 {batch_no} 的时空档案..."}
        db_orders = db.query(OrderPool).filter(OrderPool.batch_no == batch_no).all()
        if not db_orders: raise ValueError(f"波次 {batch_no} 下无订单数据")
        order_ids = [o.order_id for o in db_orders]
        all_boms = db.query(OrderBOM).filter(OrderBOM.order_id.in_(order_ids)).all()
        bom_dict = {}
        for bom in all_boms: bom_dict.setdefault(bom.order_id, []).append(bom)
        all_parts = db.query(PartMaster).all()
        part_time_dict = {p.part_type: float(p.standard_p_time) for p in all_parts}
        logical_orders = []
        for d_order in db_orders:
            boms = bom_dict.get(d_order.order_id, [])
            sku_map = {}
            for bom in boms:
                clean_id = bom.part_type.replace('零件', '') if '零件' in bom.part_type else bom.part_ty
            pe
                actual_p_time = part_time_dict.get(clean_id, 4.5) * bom.quantity
                if clean_id not in sku_map:
                    sku_map[clean_id] = {'qty': 0, 'p_time': 0.0}
                sku_map[clean_id]['qty'] += bom.quantity
                sku_map[clean_id]['p_time'] += actual_p_time
            boxes = [{ 'sku': k, 'qty': v['qty'], 'p_time': v['p_time'] } for k, v in sku_map.items()]
            total_p_time = sum(b['p_time'] for b in boxes)
            logical_orders.append({
                'order_id': d_order.order_id,
                'boxes': boxes,
            })
        logical_orders.sort(key=lambda x: x['total_p_time'], reverse=True)
        print(f"\n✅ 波次加载完成！按大单优先策略已重排 {len(logical_orders)} 个订单！")
        TASK_PROGRESS[task_id] = {"status": "running", "progress": "20%", "message": "正在
挂载环境与炼丹权重..."}
    rl_env = PickingEnv(dataset_type='test')
    rl_env.unwrapped.real_world_orders = logical_orders
    rl_env.unwrapped.total_orders = len(logical_orders)
    rl_env.unwrapped.episode_length = len(logical_orders)
    model_dir = os.path.join(project_root, "output/models")
    zip_files = glob.glob(os.path.join(model_dir, '*.zip'))
    if not zip_files:
        latest_model_path = max(zip_files, key=os.path.getctime)
        model = MaskablePPO.load(latest_model_path, env=rl_env)
    def fast_macro_simulate(strategy, limit):
        obs, _ = rl_env.reset(seed=999)
        done = False
        step_count = 0
        while not done:
            mask = np.array([True] * limit + [False] * (Config.NUM_STATIONS - limit))
            if strategy == "ai":
                try: env_mask = rl_env.unwrapped.action_masks()
                except AttributeError: env_mask = np.ones(Config.NUM_STATIONS, dtype=bool)
                combined_mask = np.logical_and(mask, env_mask)
                if not np.any(combined_mask): combined_mask = mask
                action, _ = model.predict(obs, action_masks=combined_mask, deterministic=True)
                action = int(action)
            elif strategy == "round_robin": action = step_count % limit
            elif strategy == "random": action = np.random.randint(0, limit)
            obs, _, done, _, _ = rl_env.step(int(action))
            step_count += 1

```

```

    return float(np.max(rl_env.unwrapped.station_available_time if hasattr(rl_env.unwrapped, 'station_available_time') else rl_env.unwrapped.station_workloads))
    def find_optimal(strategy):
        best_limit = Config.NUM_STATIONS; best_ms = 0
        for limit in range(Config.NUM_STATIONS, 0, -1):
            ms = fast_macro_simulate(strategy, limit)
            if ms <= Config.DEADLINE_SECONDS: best_limit = limit; best_ms = ms
            else:
                if limit == Config.NUM_STATIONS: best_ms = ms
        return best_limit, best_ms
    TASK_PROGRESS[task_id] = {"status": "running", "progress": "30%", "message": "【第一局】预演随机分发耗时..."}
    rand_st, rand_tm = find_optimal("random")
    TASK_PROGRESS[task_id] = {"status": "running", "progress": "50%", "message": "【第二局】预演传统轮询耗时..."}
    trad_st, trad_tm = find_optimal("round_robin")
    TASK_PROGRESS[task_id] = {"status": "running", "progress": "70%", "message": "【第三局】强化学习引擎寻优..."}
    ai_st, ai_tm = find_optimal("ai")
    TASK_PROGRESS[task_id] = {"status": "running", "progress": "85%", "message": "已锁定最优资源配比！准备刻录数据库剧本..."}
    sim_env = simpy.Environment()
    physical_stations = [SimpyStation(sim_env, i, getattr(Config, 'MAX_ORDERS_PER_STATION', 2)) for i in range(Config.NUM_STATIONS)]
    dispatch_records = []
    base_time = datetime.datetime.now().replace(hour=8, minute=0, second=0, microsecond=0)
    sim_env.process(simpy_dispatch_engine(sim_env, physical_stations, rl_env, model, ai_st, dispatch_records, task_id, base_time))
    sim_env.run()
    ai_real_makespan = float(sim_env.now)
    db.add_all(dispatch_records)
    db.add(SimulationTask(task_id=task_id+"-RAND", batch_no=batch_no, strategy_type="RANDOM", active_stations=rand_st, total_makespan_sec=rand_tm))
    db.add(SimulationTask(task_id=task_id+"-TRAD", batch_no=batch_no, strategy_type="TRADITIONAL", active_stations=trad_st, total_makespan_sec=trad_tm))
    db.add(SimulationTask(task_id=task_id+"-AI", batch_no=batch_no, strategy_type="AI_RL", active_stations=ai_st, total_makespan_sec=ai_real_makespan))
    db.commit()
    eff = (1 - (ai_st * ai_real_makespan) / (rand_st * rand_tm)) * 100 if rand_st * rand_tm > 0 else 0.0
    TASK_PROGRESS[task_id] = {
        "status": "completed", "progress": "100%", "deadline": Config.DEADLINE_SECONDS,
        "ai_result": {"active_stations": ai_st, "total_makespan": round(ai_real_makespan, 2)},
        "trad_result": {"active_stations": trad_st, "total_makespan": round(trad_tm, 2)},
        "rand_result": {"active_stations": rand_st, "total_makespan": round(rand_tm, 2)},
        "efficiency_up": f"{eff:.2f}%"
    }
except Exception as e:
    db.rollback()
    import traceback
    traceback.print_exc()
    TASK_PROGRESS[task_id] = {"status": "failed", "message": str(e)}
finally:
    db.close()
@app.post("/api/v1/orders/upload")
def upload_orders(batch_data: OrderBatch, db: SessionLocal = Depends(get_db)):
    try:
        unique_order_ids = list(set([item.order_id for item in batch_data.orders]))
        db.query(OrderBOM).filter(OrderBOM.order_id.in_(unique_order_ids)).delete(synchronize_session=False)
        db.flush()
        for o_id in unique_order_ids:
            exists = db.query(OrderPool).filter(OrderPool.order_id == o_id).first()
            if not exists:
                db.add(OrderPool(order_id=o_id, batch_no=batch_data.batch_no, priority_level=1))
        for item in batch_data.orders:

```

```

        db.add(OrderBOM(order_id=item.order_id, part_type=item.part_type, quantity=item.quantity))
    db.commit()
    return {"code": 200, "message": f"成功入库 {len(unique_order_ids)} 个真实拣选单的数据! "}
except Exception as e:
    db.rollback()
    raise HTTPException(status_code=500, detail=str(e))
@app.post("/api/v1/simulation/start")
def start_simulation(req: SimulationRequest, bg_tasks: BackgroundTasks):
    task_id = f"TASK-{datetime.datetime.now().strftime('%Y%m%d%H%M%S')}"
    bg_tasks.add_task(run_simulation_task, task_id, req.batch_no)
    return {"code": 200, "task_id": task_id}
@app.get("/api/v1/simulation/status/{task_id}")
def get_status(task_id: str):
    if task_id not in TASK_PROGRESS: raise HTTPException(status_code=404, detail="任务不存在")
    return {"code": 200, "data": TASK_PROGRESS[task_id]}
@app.get("/api/v1/simulation/playbook/{task_id}")
def get_simulation_playbook(task_id: str, db: SessionLocal = Depends(get_db)):
    try:
        ai_task_id = f"{task_id}-AI"
        task_info = db.query(SimulationTask).filter(SimulationTask.task_id == ai_task_id).first()
        if not task_info: raise HTTPException(status_code=404, detail="未找到任务宏观战报")
        records = db.query(DispatchResult).filter(DispatchResult.task_id == task_id).order_by(DispatchResult.predicted_spawn_time).all()
        if not records: raise HTTPException(status_code=404, detail="未找到派工明细")
        playbook = {
            "task_id": task_id, "strategy": "AI_RL", "active_stations": task_info.active_stations,
            "total_makespan_sec": task_info.total_makespan_sec, "total_boxes": len(records),
            "timeline": []
        }
        for r in records:
            playbook["timeline"].append({
                "order_id": r.order_id,
                "box_id": r.box_id,
                "target_station": r.target_station,
                "spawn_time": r.predicted_spawn_time.isoformat() if hasattr(r, 'predicted_spawn_time')
                and r.predicted_spawn_time else r.predicted_start_time.isoformat(),
                "start_time": r.predicted_start_time.isoformat(),
                "end_time": r.predicted_end_time.isoformat(),
            })
        return {"code": 200, "data": playbook}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
@app.get("/api/v1/model/training_metrics")
def get_training_metrics():
    try:
        base_dir = os.path.join(project_root, "scenarios", "order_picking")
        tb_dirs = [os.path.join(base_dir, d) for d in os.listdir(base_dir) if d.startswith("ppo_tensorboard_logs")]
        if not tb_dirs: return {"code": 404, "message": "未找到日志"}
        latest_tb_dir = max(tb_dirs, key=os.path.getmtime)
        subdirs = [os.path.join(latest_tb_dir, d) for d in os.listdir(latest_tb_dir) if os.path.isdir(os.path.join(latest_tb_dir, d))]
        log_dir = max(subdirs, key=os.path.getmtime) if subdirs else latest_tb_dir
        event_acc = EventAccumulator(log_dir)
        event_acc.Reload()
        target_tag = 'rollout/ep_rew_mean'
        tags = event_acc.Tags().get('scalars', [])
        if target_tag not in tags: return {"code": 404, "message": "无 Reward 数据"}
        events = event_acc.Scalars(target_tag)
        steps, rewards = [], []
        sample_rate = max(1, len(events))
        for i, event in enumerate(events):
            if i % sample_rate == 0 or i == len(events) - 1:
    
```

```

        step_str = f"{event.step / 10000:.1f} 万" if event.step >= 10000 else str(event.step)
        steps.append(step_str); rewards.append(round(event.value, 2))
        return {"code": 200, "data": {"steps": steps, "rewards": rewards}}
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"日志解析失败: {str(e)}")
if __name__ == "__main__":
    uvicorn.run("server:app", host="0.0.0.0", port=8088, reload=True)
import os
import sys
import datetime
import json
import numpy as np
import uvicorn
import simpy
import glob
from fastapi import FastAPI, Depends, HTTPException, BackgroundTasks
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from sqlalchemy import func
from typing import List
from tensorboard.backend.event_processing.event_accumulator import EventAccumulator
project_root = os.path.abspath(os.path.join(os.path.dirname(__file__), '../'))
if project_root not in sys.path:
    sys.path.append(project_root)
from scenarios.order_picking.config import Config
from backend.database import SessionLocal, OrderPool, OrderBOM, SimulationTask, PartMaster, DispatchResult
from sb3_contrib import MaskablePPO
from scenarios.order_picking.rl_environment import PickingEnv
from scenarios.order_picking.app_config import get_config_value
from scenarios.order_picking.data_paths import DEFAULT_DAILY_DATE, MODEL_DIR, resolve_model_path
from scenarios.order_picking.inventory_preprocess import list_snapshots, load_snapshot
from scenarios.order_picking.order_preprocessor import preprocess_orders
from backend.sku_avg_time import rebuild_sku_avg_time
from core_engine.rules.dispatch_rules import DispatchRules
from core_engine.models.resource_model import SimpyStation
app = FastAPI(title="Weichai APS AI 智能排产网关")
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
TASK_PROGRESS = {}
SCHEDULE_RESULTS = {}
DEFAULT_BATCH_NO = str(get_config_value("simulation", "default_batch_no", f"ORDER_WAVE_{DEFAULT_DAILY_DATE}"))
DEFAULT_INITIAL_SNAPSHOT_ID = str(
    get_config_value("simulation", "default_initial_snapshot_id", f"{DEFAULT_DAILY_DATE}-morning")
)
DEFAULT_EVENING_SNAPSHOT_ID = str(
    get_config_value("simulation", "default_evening_snapshot_id", f"{DEFAULT_DAILY_DATE}-evening")
)
DEFAULT_STRATEGY = str(get_config_value("simulation", "default_strategy", "ai"))
DEFAULT_ACTIVE_STATION_LIMIT = int(get_config_value("simulation", "default_active_station_limit", Config.NUM_STATIONS))
AUTO_REBUILD_SKU_TIME = bool(get_config_value("simulation", "auto_rebuild_sku_time", True))
def update_progress(task_id, progress, message):
    print(f"\n 🔄 [前端拉取进度 {progress}] {message}")
    TASK_PROGRESS[task_id] = {"status": "running", "progress": progress, "message": message}
e}

```

```

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
class PartMasterItem(BaseModel):
    part_type: str
    part_name: str
    process_time: float
class PartMasterBatch(BaseModel):
    parts: List[PartMasterItem]
class OrderItem(BaseModel):
    order_id: str
    part_type: str
    quantity: int
class OrderBatch(BaseModel):
    batch_no: str
    orders: List[OrderItem]
class SimulationRequest(BaseModel):
    batch_no: str = DEFAULT_BATCH_NO
    inventory_snapshot_id: str = DEFAULT_INITIAL_SNAPSHOT_ID
    evening_snapshot_id: str = DEFAULT_EVENING_SNAPSHOT_ID
    shortage_policy: str = "exception_queue"
class ScheduleDispatchRequest(BaseModel):
    batch_no: str = DEFAULT_BATCH_NO
    inventory_snapshot_id: str = DEFAULT_INITIAL_SNAPSHOT_ID
    strategy: str = DEFAULT_STRATEGY
    active_station_limit: int = DEFAULT_ACTIVE_STATION_LIMIT
class DBLogger:
    def __init__(self, dispatch_records, task_id, base_time):
        self.dispatch_records = dispatch_records
        self.task_id = task_id
        self.base_time = base_time
        self.temp_records = {}
    def log_event(self, time, entity_id, event_type, station_id, details=None):
        if event_type == "spawn":
            self.temp_records[entity_id] = {
                "spawn_env_time": time,
                "target_station": station_id,
                "order_id": details.get("order_id", "") if details else ""
            }
        elif event_type == "start_process":
            if entity_id in self.temp_records:
                self.temp_records[entity_id]["start_env_time"] = time
        elif event_type == "end_process":
            if entity_id in self.temp_records:
                record = self.temp_records[entity_id]
                end_env_time = time
                spawn_dt = self.base_time + datetime.timedelta(seconds=float(record["spawn_env_time"]))
                start_dt = self.base_time + datetime.timedelta(seconds=float(record["start_env_time"]))
                end_dt = self.base_time + datetime.timedelta(seconds=float(end_env_time))
                self.dispatch_records.append(
                    DispatchResult(
                        task_id=self.task_id,
                        order_id=record["order_id"],
                        box_id=entity_id,
                        target_station=record["target_station"] + 1,
                        predicted_spawn_time=spawn_dt,
                        predicted_start_time=start_dt,
                        predicted_end_time=end_dt
                    )
                )
            del self.temp_records[entity_id]
    def set_power_status(self, *args, **kwargs):

```

```

    pass
def launch_box(env, station, box_id, order_id, p_time, t_trans, entity_type, logger, delay):
    if delay > 0:
        yield env.timeout(delay)
    if logger:
        logger.log_event(env.now, box_id, "spawn", station.station_id, {"order_id": order_id})
    try:
        yield env.process(station.process_box(box_id, order_id, p_time, t_trans, entity_type))
    except TypeError:
        yield env.process(station.process_box(box_id, p_time, t_trans, entity_type))
def build_logical_orders(db, batch_no: str):
    db_orders = db.query(OrderPool).filter(OrderPool.batch_no == batch_no).all()
    if not db_orders:
        raise ValueError(f"批次 {batch_no} 下没有订单数据")
    order_ids = [o.order_id for o in db_orders]
    all_boms = db.query(OrderBOM).filter(OrderBOM.order_id.in_(order_ids)).all()
    bom_dict = {}
    for bom in all_boms:
        bom_dict.setdefault(bom.order_id, []).append(bom)
    all_parts = db.query(PartMaster).all()
    part_time_dict = {str(p.part_type).strip(): float(p.standard_p_time) for p in all_parts}
    logical_orders = []
    for d_order in db_orders:
        boms = bom_dict.get(d_order.order_id, [])
        sku_map = {}
        for bom in boms:
            clean_id = str(bom.part_type).strip().replace("零件", "")
            actual_p_time = part_time_dict.get(clean_id, 4.5) * bom.quantity
            if clean_id not in sku_map:
                sku_map[clean_id] = {"qty": 0, "p_time": 0.0}
            sku_map[clean_id]["qty"] += bom.quantity
            sku_map[clean_id]["p_time"] += actual_p_time
        boxes = [{"sku": k, "qty": v["qty"], "p_time": v["p_time"]} for k, v in sku_map.items()]
        logical_orders.append(
            {
                "order_id": d_order.order_id,
                "boxes": boxes,
                "total_p_time": sum(b["p_time"] for b in boxes),
            }
        )
    return logical_orders
def load_latest_ai_model(rl_env):
    model_path = resolve_model_path()
    if not model_path or not model_path.exists():
        return MaskablePPO.load(str(model_path), env=rl_env)
def prepare_orders_with_inventory(batch_no: str, inventory_snapshot_id: str, db):
    logical_orders = build_logical_orders(db, batch_no)
    inventory_snapshot = load_snapshot(inventory_snapshot_id) if inventory_snapshot_id else None
    one
    preprocessed = preprocess_orders(logical_orders, inventory_snapshot)
    processable_orders = preprocessed["processable_orders"]
    preprocess_stats = preprocessed["preprocess_stats"]
    shortage_orders = preprocessed["shortage_orders"]
    if not processable_orders:
        raise ValueError(
            f"库存预处理后没有可执行订单：输入 {preprocess_stats.get('input_order_count', 0)}
单， "
            f"缺料异常 {preprocess_stats.get('shortage_order_count', 0)} 单。请检查库存快照。"
        )
    return logical_orders, inventory_snapshot, processable_orders, shortage_orders, preprocess_stats
def generate_dispatch_mapping(processable_orders, strategy: str = "ai", active_station_limit: int = Config.NUM_STATIONS):
    station_limit = max(1, min(int(active_station_limit or Config.NUM_STATIONS), Config.NUM_STATIONS))
    strategy_key = (strategy or "ai").lower()

```



```

rl_env = PickingEnv(dataset_type="test")
rl_env.unwrapped.set_orders(processable_orders, episode_length=len(processable_orders))
obs, _ = rl_env.reset(seed=999)
model = load_latest_ai_model(rl_env) if strategy_key == "ai" else None
mapping = []
done = False
sequence = 1
while not done:
    current_order = rl_env.unwrapped.real_world_orders[rl_env.unwrapped.current_step]
    station_mask = np.array([True] * station_limit + [False] * (Config.NUM_STATIONS - station_limit))
    try:
        env_mask = rl_env.unwrapped.action_masks()
    except AttributeError:
        env_mask = np.ones(Config.NUM_STATIONS, dtype=bool)
    combined_mask = np.logical_and(station_mask, env_mask)
    if not np.any(combined_mask):
        combined_mask = station_mask
    if strategy_key == "ai":
        action = int(model.predict(obs, action_masks=combined_mask, deterministic=True)[0])
    elif strategy_key == "round_robin":
        valid_actions = np.flatnonzero(combined_mask)
        action = int(valid_actions[(sequence - 1) % len(valid_actions)])
    elif strategy_key == "random":
        valid_actions = np.flatnonzero(combined_mask)
        action = int(np.random.choice(valid_actions))
    else:
        raise ValueError(f"不支持的调度策略: {strategy}")
    mapping.append(
        {
            "sequence": sequence,
            "order_id": current_order["order_id"],
            "target_station": action + 1,
            "box_count": len(current_order.get("boxes", [])),
            "total_p_time": round(float(current_order.get("total_p_time", 0.0)), 3),
            "boxes": [
                {
                    "sku": box.get("sku", ""),
                    "qty": box.get("qty", 1),
                    "p_time": round(float(box.get("p_time", 0.0)), 3),
                }
                for box in current_order.get("boxes", [])
            ],
        }
    )
    obs, _, done, _, _ = rl_env.step(action)
    sequence += 1
return mapping, round(float(rl_env.unwrapped.global_time), 3), station_limit

def save_schedule_result(task_id: str, result: dict):
    output_dir = os.path.join(project_root, "output", "schedule_results")
    os.makedirs(output_dir, exist_ok=True)
    output_path = os.path.join(output_dir, f"{task_id}.json")
    latest_path = os.path.join(output_dir, "latest.json")
    with open(output_path, "w", encoding="utf-8") as f:
        json.dump(result, f, ensure_ascii=False, indent=2)
    with open(latest_path, "w", encoding="utf-8") as f:
        json.dump(result, f, ensure_ascii=False, indent=2)
    return output_path

def run_schedule_dispatch(task_id: str, batch_no: str, inventory_snapshot_id: str, strategy: str, active_station_limit: int):
    db = SessionLocal()
    try:
        SCHEDULE_RESULTS[task_id] = {
            "task_id": task_id,
            "batch_no": batch_no,
            "status": "running",

```

```

        "message": "正在执行纯智能调度...",
    }
    _, inventory_snapshot, processable_orders, shortage_orders, preprocess_stats = prepare_order_with_inventory(
        batch_no, inventory_snapshot_id, db
    )
    mapping, estimated_makespan, station_limit = generate_dispatch_mapping(
        processable_orders,
        strategy=strategy,
        active_station_limit=active_station_limit,
    )
    result = {
        "task_id": task_id,
        "batch_no": batch_no,
        "inventory_snapshot_id": inventory_snapshot_id,
        "strategy": strategy.upper() if strategy.lower() != "ai" else "AI_RL",
        "status": "completed",
        "output_file": "",
        "summary": {
            "input_order_count": preprocess_stats.get("input_order_count", 0),
            "scheduled_order_count": len(mapping),
            "shortage_order_count": len(shortage_orders),
            "reordered_count": preprocess_stats.get("reordered_count", 0),
            "scarce_sku_count": preprocess_stats.get("scarce_sku_count", 0),
            "inventory_sku_count": preprocess_stats.get("inventory_sku_count", 0),
            "station_count": station_limit,
            "estimated_env_makespan_sec": estimated_makespan,
            "inventory_summary": (inventory_snapshot or {}).get("summary", {}),
        },
        "mapping": mapping,
        "shortage_orders": [
            {
                "order_id": order.get("order_id", ""),
                "exception_reason": order.get("exception_reason", ""),
                "shortage_skus": order.get("shortage_skus", []),
                "box_count": len(order.get("boxes", [])),
                "total_p_time": round(float(order.get("total_p_time", 0.0)), 3),
            }
            for order in shortage_orders
        ],
    }
    output_path = save_schedule_result(task_id, result)
    result["output_file"] = os.path.relpath(output_path, project_root).replace("\\", "/")
    save_schedule_result(task_id, result)
    SCHEDULE_RESULTS[task_id] = result
except Exception as e:
    import traceback
    traceback.print_exc()
    SCHEDULE_RESULTS[task_id] = {
        "task_id": task_id,
        "batch_no": batch_no,
        "status": "failed",
        "message": str(e),
    }
finally:
    db.close()
def simpy_dispatch_engine(
    env,
    stations,
    rl_env,
    model,
    optimal_stations,
    dispatch_records,
    task_id,
    base_time,
    strategy="ai",

```

```

save_records=True,
random_seed=20260729,
use_action_mask=True,
):
    db_logger = DBLogger(dispatch_records, task_id, base_time) if save_records else None
    for s in stations:
        s.logger = db_logger
        obs, _ = rl_env.reset(seed=999)
        energy_saving_mask = np.array([True] * optimal_stations + [False] * (Config.NUM_STATIO
NS - optimal_stations))
        done = False
        step = 0
        dispatch_time_cursor = 0.0
        rng = np.random.default_rng(random_seed)
        while not done:
            strategy_key = str(strategy or "ai").lower()
            if strategy_key == "ai":
                try:
                    env_internal_mask = rl_env.unwrapped.action_masks()
                except AttributeError:
                    env_internal_mask = np.ones(Config.NUM_STATIONS, dtype=bool)
                    combined_masks = np.logical_and(energy_saving_mask, env_internal_mask) if use_acti
on_mask else energy_saving_mask
                    if not np.any(combined_masks):
                        combined_masks = energy_saving_mask
                    obs_state = rl_env.unwrapped.get_obs()
                    action = int(DispatchRules.rule_ai_policy(model, obs=obs_state, valid_masks=combined
_masks))
                elif strategy_key == "round_robin":
                    action = int(step % optimal_stations)
                elif strategy_key == "random":
                    action = int(rng.integers(0, optimal_stations))
                else:
                    raise ValueError(f"Unsupported simulation strategy: {strategy}")
            current_order = rl_env.unwrapped.real_world_orders[rl_env.unwrapped.current_step]
            target_station = stations[action]
            try:
                d_main = Config.get_station_main_distance(action)
                t_branch = Config.get_branch_info(action)["transit_time_s"]
                t_trans = (d_main / Config.BELT_SPEED) + t_branch
            except AttributeError:
                d_main = Config.STATION_EXIT_FAR_DISTANCES[action] - (Config.EXIT_PORT
DELTA / 2.0)
                t_trans = (d_main / Config.BELT_SPEED) + (Config.BRANCH_IN_LENGTH / Config.
BELT_SPEED)
            local_cursor = dispatch_time_cursor
            if hasattr(rl_env.unwrapped, 'station_active_boxes'):
                while True:
                    active_boxes = [b for b in rl_env.unwrapped.station_active_boxes[action] if b['finish_t
ime'] > local_cursor]
                    active_order_ids = set(b['order_id'] for b in active_boxes)
                    is_new_order = current_order['order_id'] not in active_order_ids
                    order_limit = getattr(Config, 'MAX_ORDERS_PER_STATION', 2)
                    box_limit = getattr(Config, 'MAX_BOXES_PER_STATION', 8)
                    if not (len(active_boxes) >= box_limit or (is_new_order and len(active_order_ids) >=
order_limit)):
                        break
                    if active_boxes:
                        local_cursor = max(local_cursor, min(b['finish_time'] for b in active_boxes))
                    else:
                        local_cursor += 1.0
            for box in current_order['boxes']:
                local_cursor += Config.DISPATCH_INTERVAL
                delay_before_launch = max(0, local_cursor - env.now)
                env.process(launch_box(
                    env, target_station, f"{current_order['order_id']}-P{box['sku']}", current_order['order

```

```

id'],
    box['p_time'], t_trans, box['sku'],
    db_logger, delay_before_launch
))
dispatch_time_cursor = local_cursor
obs, _, done, _, _ = rl_env.step(action)
step += 1
while any(s.machine.count > 0 or len(s.machine.queue) > 0 for s in stations):
    yield env.timeout(1.0)
def run_simulation_task(
    task_id: str,
    batch_no: str = DEFAULT_BATCH_NO,
    inventory_snapshot_id: str = DEFAULT_INITIAL_SNAPSHOT_ID,
    evening_snapshot_id: str = DEFAULT_EVENING_SNAPSHOT_ID,
):
    db = SessionLocal()
    try:
        update_progress(task_id, "10%", f"正在提取波次 {batch_no} 的时空档案...")
        if AUTO_REBUILD_SKU_TIME:
            update_progress(task_id, "12%", "正在刷新 SKU 平均处理时间...")
            sku_time_result = rebuild_sku_avg_time()
            print(
                "SKU average picking time refreshed: "
                f"inserted={sku_time_result['inserted']}, "
                f"source_files={len(sku_time_result['source_files'])}"
            )
        update_progress(task_id, "15%", f"正在提取波次 {batch_no} 的订单数据...")
        db_orders = db.query(OrderPool).filter(OrderPool.batch_no == batch_no).all()
        if not db_orders: raise ValueError(f"波次 {batch_no} 下无订单数据")
        order_ids = [o.order_id for o in db_orders]
        all_boms = db.query(OrderBOM).filter(OrderBOM.order_id.in_(order_ids)).all()
        bom_dict = {}
        for bom in all_boms: bom_dict.setdefault(bom.order_id, []).append(bom)
        all_parts = db.query(PartMaster).all()
        part_time_dict = {str(p.part_type).strip(): float(p.standard_p_time) for p in all_parts}
        logical_orders = []
        for d_order in db_orders:
            boms = bom_dict.get(d_order.order_id, [])
            sku_map = {}
            for bom in boms:
                clean_id = str(bom.part_type).strip().replace('零件', '')
                actual_p_time = part_time_dict.get(clean_id, 4.5) * bom.quantity
                if clean_id not in sku_map: sku_map[clean_id] = {'qty': 0, 'p_time': 0.0}
                sku_map[clean_id]['qty'] += bom.quantity
                sku_map[clean_id]['p_time'] += actual_p_time
            boxes = [{'sku': k, 'qty': v['qty'], 'p_time': v['p_time']} for k, v in sku_map.items()]
            logical_orders.append({'order_id': d_order.order_id, 'boxes': boxes, 'total_p_time': sum(b
['p_time'] for b in boxes)})
        logical_orders.sort(key=lambda x: x['total_p_time'], reverse=True)
        print(f"\n✅ 数据提取完毕！总波次订单数: {len(logical_orders)} (已关闭大单优先，
还原原版时间)")
        initial_inventory_snapshot = load_snapshot(inventory_snapshot_id) if inventory_snapshot_
id else None
        preprocessed = preprocess_orders(logical_orders, initial_inventory_snapshot)
        processable_orders = preprocessed["processable_orders"]
        preprocess_stats = preprocessed["preprocess_stats"]
        shortage_orders = preprocessed["shortage_orders"]
        processable_order_ids = {str(order["order_id"]) for order in processable_orders}
        baseline_orders = [
            order for order in logical_orders
            if str(order["order_id"]) in processable_order_ids
        ]
        if not processable_orders:
            raise ValueError(
                f"库存预处理后没有可执行订单：输入 {preprocess_stats.get('input_order_count',
0)} 单，"

```

```

        f'缺料异常 {preprocess_stats.get('shortage_order_count', 0)} 单。请检查库存快照。
    """
    )
    rl_env = PickingEnv(dataset_type='test')
    rl_env.unwrapped.set_orders(processable_orders, episode_length=len(processable_orders))
    rl_env.reset(seed=999)
    print(
        f'库存预处理完成: 输入 {preprocess_stats['input_order_count']} 单, "
        f'可执行 {preprocess_stats['processable_order_count']} 单, "
        f'缺料异常 {preprocess_stats['shortage_order_count']} 单。"
    )
    model_path = resolve_model_path()
    if not model_path or not model_path.exists():
        model = MaskablePPO.load(str(model_path), env=rl_env)
    def run_simpy_strategy(strategy, limit, save_records=False, result_task_id=None, seed_offset=0):
        strategy_key = str(strategy or "ai").lower()
        strategy_orders = processable_orders if strategy_key == "ai" else baseline_orders
        strategy_env = PickingEnv(dataset_type='test')
        strategy_env.unwrapped.set_orders(strategy_orders, episode_length=len(strategy_orders))
        sim_env = simpy.Environment()
        records = []
        base_time = datetime.datetime.now().replace(hour=8, minute=0, second=0, microsecond=0)
        physical_stations = [
            SimpyStation(sim_env, i, getattr(Config, 'MAX_ORDERS_PER_STATION', 2))
            for i in range(Config.NUM_STATIONS)
        ]
        sim_env.process(simpy_dispatch_engine(
            sim_env,
            physical_stations,
            strategy_env,
            model,
            limit,
            records,
            result_task_id or task_id,
            base_time,
            strategy=strategy,
            save_records=save_records,
            random_seed=20260729 + seed_offset,
            use_action_mask=(strategy_key == "ai"),
        ))
        sim_env.run()
        return {
            "strategy": strategy,
            "active_stations": limit,
            "total_makespan": float(sim_env.now),
            "dispatch_records": records,
        }
    def fast_macro_simulate(strategy, limit):
        rl_env.reset(seed=999)
        if len(rl_env.unwrapped.real_world_orders) == 0:
            raise ValueError(
                f'库存预处理后没有可执行订单: 输入 {preprocess_stats.get('input_order_count', 0)} 单, "
                f'缺料异常 {preprocess_stats.get('shortage_order_count', 0)} 单。请检查库存快照。"
            )
        done = False
        step = 0
        while not done:
            mask = np.array([True] * limit + [False] * (Config.NUM_STATIONS - limit))
            try:
                env_mask = rl_env.unwrapped.action_masks()
            except:
                env_mask = np.ones(Config.NUM_STATIONS, dtype=bool)
            combined = np.logical_and(mask, env_mask)

```

```

if not np.any(combined): combined = mask
if strategy == "ai": action = int(model.predict(rl_env.unwrapped._get_obs(), action_ma
sks=combined, deterministic=True)[0])
elif strategy == "round_robin": action = step % limit
else: action = np.random.randint(0, limit)
_, _, done, _, _ = rl_env.step(action)
step += 1
return float(rl_env.unwrapped.global_time)
results = {}
strategy_list = ["random", "round_robin", "ai"]
progress_map = {"random": "30%", "round_robin": "50%", "ai": "70%"}
for strategy in strategy_list:
    results[strategy] = []
    update_progress(task_id, progress_map[strategy], f'正在测算 {strategy.upper()} 策略极
限探底...')
    for limit in range(Config.NUM_STATIONS, 0, -1):
        ms = run_simpy_strategy(strategy, limit, save_records=False, seed_offset=limit)["total
        _makespan"]
        results[strategy].append((limit, ms))
        if ms > Config.DEADLINE_SECONDS:
            print(f' - active stations: {limit:02d} | SimPy makespan: {ms:.1f}s | timeout')
            break
        status_txt = "✅ 满足" if ms <= Config.DEADLINE_SECONDS else "❌ 超时"
        print(f' — 开机数: {limit:02d} | 完工耗时: {ms:.1f}s | {status_txt}')
    update_progress(task_id, "85%", "已锁定最优资源配比! 准备刻录物理引擎与 3D 剧
本...")
def get_best(strat):
    valid = [r for r in results[strat] if r[1] <= Config.DEADLINE_SECONDS]
    cannot meet the deadline, show its 16-station result instead of the
    return min(valid, key=lambda x: x[0]) if valid else results[strat][0]
rand_st, rand_tm = get_best("random")
trad_st, trad_tm = get_best("round_robin")
ai_st, ai_tm = get_best("ai")
sim_env = simpy.Environment()
dispatch_records = []
base_time = datetime.datetime.now().replace(hour=8, minute=0, second=0, microsecond=0)
physical_stations = [SimpyStation(sim_env, i, getattr(Config, 'MAX_ORDERS_PER_STA
TION', 2)) for i in range(Config.NUM_STATIONS)]
sim_env.process(simpy_dispatch_engine(sim_env, physical_stations, rl_env, model, ai_st, d
ispatch_records, task_id, base_time))
sim_env.run()
ai_real_makespan = float(sim_env.now)
db.add_all(dispatch_records)
db.add(SimulationTask(task_id=task_id+"-RAND", batch_no=batch_no, strategy_type="R
ANDOM", active_stations=rand_st, total_makespan_sec=rand_tm))
db.add(SimulationTask(task_id=task_id+"-TRAD", batch_no=batch_no, strategy_type="T
RADITIONAL", active_stations=trad_st, total_makespan_sec=trad_tm))
db.add(SimulationTask(task_id=task_id+"-AI", batch_no=batch_no, strategy_type="AI_RL
", active_stations=ai_st, total_makespan_sec=ai_real_makespan))
db.commit()
def station_hour_efficiency(baseline_st, baseline_tm, ai_station_count, ai_makespan):
    baseline_station_time = float(baseline_st) * float(baseline_tm)
    ai_station_time = float(ai_station_count) * float(ai_makespan)
    if baseline_station_time <= 0:
        return 0.0
    return (1 - ai_station_time / baseline_station_time) * 100
eff_vs_trad = station_hour_efficiency(trad_st, trad_tm, ai_st, ai_real_makespan)
eff_vs_rand = station_hour_efficiency(rand_st, rand_tm, ai_st, ai_real_makespan)
inventory_summary = (initial_inventory_snapshot or {}).get("summary", {})
update_progress(task_id, "100%", "✅ 战报生成完毕! 大屏可提取渲染!")
TASK_PROGRESS[task_id].update({
    "status": "completed", "deadline": Config.DEADLINE_SECONDS,
    "ai_result": {"active_stations": ai_st, "total_makespan": round(ai_real_makespan, 2)},
    "trad_result": {"active_stations": trad_st, "total_makespan": round(trad_tm, 2)},
    "rand_result": {"active_stations": rand_st, "total_makespan": round(rand_tm, 2)},
    "efficiency_up": f'{eff_vs_trad:.2f}%',

```

```

    "efficiency_vs_round_robin": f"{eff_vs_trad:.2f}%",
    "efficiency_vs_random": f"{eff_vs_rand:.2f}%",
    "efficiency_formula": "1 - (ai_stations * ai_makespan) / (baseline_stations * baseline_ma
kespan)",
    "inventory_result": {
        "initial_snapshot_id": inventory_snapshot_id,
        "evening_snapshot_id": evening_snapshot_id,
        "summary": inventory_summary,
        "preprocess_stats": preprocess_stats,
        "exception_order_count": len(shortage_orders),
        "evening_validation": None,
        "shortage_policy": "exception_queue",
    }
})
except Exception as e:
    db.rollback()
    import traceback
    traceback.print_exc()
    update_progress(task_id, "failed", f"报错中断: {str(e)}")
finally:
    db.close()
@app.post("/api/v1/orders/upload")
def upload_orders(batch_data: OrderBatch, db: SessionLocal = Depends(get_db)):
    try:
        unique_order_ids = list(set([item.order_id for item in batch_data.orders]))
        db.query(OrderBOM).filter(OrderBOM.order_id.in_(unique_order_ids)).delete(synchroniz
e_session=False)
        db.flush()
        for o_id in unique_order_ids:
            exists = db.query(OrderPool).filter(OrderPool.order_id == o_id).first()
            if not exists:
                db.add(OrderPool(order_id=o_id, batch_no=batch_data.batch_no, priority_level=1))
            for item in batch_data.orders:
                db.add(OrderBOM(order_id=item.order_id, part_type=item.part_type, quantity=item.qua
ntity))
        db.commit()
        return {"code": 200, "message": f"成功入库 {len(unique_order_ids)} 个真实拣选单的数据!"}
    except Exception as e:
        db.rollback()
        raise HTTPException(status_code=500, detail=str(e))
@app.get("/api/v1/orders/batches")
def list_order_batches(db: SessionLocal = Depends(get_db)):
    rows = (
        db.query(OrderPool.batch_no, func.count(OrderPool.order_id))
        .filter(OrderPool.batch_no.isnot(None))
        .group_by(OrderPool.batch_no)
        .order_by(OrderPool.batch_no.desc())
        .all()
    )
    batches = [
        {"batch_no": str(batch_no), "order_count": int(order_count)}
        for batch_no, order_count in rows
        if batch_no
    ]
    if DEFAULT_BATCH_NO and not any(item["batch_no"] == DEFAULT_BATCH_NO for item
in batches):
        batches.insert(0, {"batch_no": DEFAULT_BATCH_NO, "order_count": 0})
    return {"code": 200, "data": batches}
@app.post("/api/v1/sku-time/rebuild")
def rebuild_sku_time():
    try:
        result = rebuild_sku_avg_time()
        return {
            "code": 200,
            "message": "SKU average picking time refreshed",
        }
    except Exception as e:
        db.rollback()
        raise HTTPException(status_code=500, detail=str(e))

```

```

        "data": result,
    }
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))
@app.post("/api/v1/simulation/start")
def start_simulation(req: SimulationRequest, bg_tasks: BackgroundTasks):
    task_id = f"TASK-{datetime.datetime.now().strftime('%Y%m%d%H%M%S')}"
    bg_tasks.add_task(run_simulation_task, task_id, req.batch_no, req.inventory_snapshot_id, req.
evening_snapshot_id)
    return {"code": 200, "task_id": task_id}
@app.post("/api/v1/schedule/dispatch")
def start_schedule_dispatch(req: ScheduleDispatchRequest, bg_tasks: BackgroundTasks):
    task_id = f"SCHEDULE-{datetime.datetime.now().strftime('%Y%m%d%H%M%S')}"
    bg_tasks.add_task(
        run_schedule_dispatch_task,
        task_id,
        req.batch_no,
        req.inventory_snapshot_id,
        req.strategy,
        req.active_station_limit,
    )
    SCHEDULE_RESULTS[task_id] = {
        "task_id": task_id,
        "batch_no": req.batch_no,
        "status": "queued",
        "message": "纯智能调度任务已提交",
    }
    return {"code": 200, "task_id": task_id, "message": "纯智能调度任务已启动"}
@app.get("/api/v1/schedule/result/{task_id}")
def get_schedule_result(task_id: str):
    if task_id in SCHEDULE_RESULTS:
        return {"code": 200, "data": SCHEDULE_RESULTS[task_id]}
    output_path = os.path.join(project_root, "output", "schedule_results", f"{task_id}.json")
    if os.path.exists(output_path):
        with open(output_path, "r", encoding="utf-8") as f:
            return {"code": 200, "data": json.load(f)}
    raise HTTPException(status_code=404, detail="调度任务不存在")
@app.get("/api/v1/app/config")
def get_app_config():
    return {
        "code": 200,
        "data": {
            "active_date": DEFAULT_DAILY_DATE,
            "default_batch_no": DEFAULT_BATCH_NO,
            "default_initial_snapshot_id": DEFAULT_INITIAL_SNAPSHOT_ID,
            "default_evening_snapshot_id": DEFAULT_EVENING_SNAPSHOT_ID,
            "default_strategy": DEFAULT_STRATEGY,
            "default_active_station_limit": DEFAULT_ACTIVE_STATION_LIMIT,
        },
    }
@app.get("/api/v1/inventory/snapshots")
def get_inventory_snapshots():
    return {"code": 200, "data": list_snapshots()}
@app.get("/api/v1/inventory/summary/{snapshot_id}")
def get_inventory_summary(snapshot_id: str):
    try:
        snapshot = load_snapshot(snapshot_id)
        return {
            "code": 200,
            "data": {
                "snapshot_id": snapshot.get("snapshot_id"),
                "source_file": snapshot.get("source_file"),
                "summary": snapshot.get("summary", {}),
            },
        }
    except Exception as e:

```



```

        raise HTTPException(status_code=404, detail=str(e))
@app.get("/api/v1/simulation/status/{task_id}")
def get_status(task_id: str):
    if task_id not in TASK_PROGRESS: raise HTTPException(status_code=404, detail="任务不存在")
    return {"code": 200, "data": TASK_PROGRESS[task_id]}
@app.get("/api/v1/simulation/playbook/{task_id}")
def get_simulation_playbook(task_id: str, db: SessionLocal = Depends(get_db)):
    try:
        ai_task_id = f"{task_id}-AI"
        task_info = db.query(SimulationTask).filter(SimulationTask.task_id == ai_task_id).first()
        if not task_info: raise HTTPException(status_code=404, detail="未找到任务宏观战报")
        records = db.query(DispatchResult).filter(DispatchResult.task_id == task_id).order_by(DispatchResult.predicted_spawn_time).all()
        if not records: raise HTTPException(status_code=404, detail="未找到派工明细")
        playbook = {
            "task_id": task_id, "strategy": "AI_RL", "active_stations": task_info.active_stations,
            "total_makespan_sec": task_info.total_makespan_sec, "total_boxes": len(records),
            "timeline": []
        }
        for r in records:
            item = {
                "order_id": r.order_id,
                "box_id": r.box_id,
                "target_station": r.target_station,
                "spawn_time": r.predicted_spawn_time.isoformat() if hasattr(r, 'predicted_spawn_time')
                and r.predicted_spawn_time else r.predicted_start_time.isoformat(),
                "start_time": r.predicted_start_time.isoformat(),
                "end_time": r.predicted_end_time.isoformat(),
            }
            playbook["timeline"].append(item)
        return {"code": 200, "data": playbook}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
@app.get("/api/v1/model/training_metrics")
def get_training_metrics():
    try:
        base_dir = os.path.join(project_root, "scenarios", "order_picking")
        tb_dirs = [os.path.join(base_dir, d) for d in os.listdir(base_dir) if d.startswith("ppo_tensorboard_logs")]
        if not tb_dirs: return {"code": 404, "message": "未找到日志"}
        latest_tb_dir = max(tb_dirs, key=os.path.getmtime)
        subdirs = [os.path.join(latest_tb_dir, d) for d in os.listdir(latest_tb_dir) if os.path.isdir(os.path.join(latest_tb_dir, d))]
        log_dir = max(subdirs, key=os.path.getmtime) if subdirs else latest_tb_dir
        event_acc = EventAccumulator(log_dir)
        event_acc.Reload()
        target_tag = 'rollout/ep_rew_mean'
        tags = event_acc.Tags().get('scalars', [])
        if target_tag not in tags: return {"code": 404, "message": "无 Reward 数据"}
        events = event_acc.Scalars(target_tag)
        steps, rewards = [], []
        sample_rate = max(1, len(events))
        for i, event in enumerate(events):
            if i % sample_rate == 0 or i == len(events) - 1:
                step_str = f"{event.step / 10000:.1f} 万" if event.step >= 10000 else str(event.step)
                steps.append(step_str); rewards.append(round(event.value, 2))
        return {"code": 200, "data": {"steps": steps, "rewards": rewards}}
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"日志解析失败: {str(e)}")
if __name__ == "__main__":
    uvicorn.run("server:app", host="0.0.0.0", port=8088, reload=True)
import os
import sys
import datetime
from sqlalchemy import create_engine, Column, Integer, String, Float, Boolean, DateTime, BigI

```

```

integer
from sqlalchemy.orm import declarative_base, sessionmaker
project_root = os.path.abspath(os.path.join(os.path.dirname(__file__), '../'))
if project_root not in sys.path:
    sys.path.append(project_root)
scenario_dir = os.path.join(project_root, 'scenarios', 'order_picking')
if scenario_dir not in sys.path:
    sys.path.append(scenario_dir)
from config import Config
DATABASE_URL = "mysql+pymysql://root:mengdi@127.0.0.1:3306/weichai_aps"
engine = create_engine(DATABASE_URL, echo=False)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

class PartMaster(Base):
    tablename = "t_part_master"
    id = Column(Integer, primary_key=True, autoincrement=True)
    part_type = Column(String(50), nullable=False, comment="零件类型")
    standard_p_time = Column(Float, nullable=False, comment="标准加工耗时(秒)")
    variance_p_time = Column(Float, default=0.0, comment="耗时波动方差(模拟工人疲劳)")

class StationMaster(Base):
    """2. 车间物理站台主表：记录 1-16 号机床的全量真实物理状态 (对齐 Config)"""
    tablename = "t_station_master"
    station_id = Column(Integer, primary_key=True, autoincrement=False, comment="站台编号 (1-16)")
    main_distance_m = Column(Float, nullable=False, comment="主线物理距离(米)")
    branch_length_m = Column(Float, nullable=False, comment="支线物理长度(米)")
    is_upper = Column(Boolean, nullable=False, comment="是否为上层支线")
    status = Column(String(20), default="ACTIVE", comment="状态: ACTIVE / BROKEN")

class OrderPool(Base):
    tablename = "t_order_pool"
    order_id = Column(String(100), primary_key=True, comment="订单流水号")
    batch_no = Column(String(100), nullable=False, comment="生产批次号")
    priority_level = Column(Integer, default=1, comment="优先级")
    deadline_time = Column(DateTime, default=datetime.datetime.now, comment="甲方要求的交期")
    is_simulated = Column(Boolean, default=False, comment="是否已被推演过")

class OrderBOM(Base):
    tablename = "t_order_bom"
    id = Column(BigInteger, primary_key=True, autoincrement=True)
    order_id = Column(String(100), nullable=False, index=True, comment="关联订单号")
    part_type = Column(String(50), nullable=False, comment="零件类型")
    quantity = Column(Integer, nullable=False, comment="需求数量")

class SimulationTask(Base):
    tablename = "t_simulation_task"
    task_id = Column(String(100), primary_key=True, comment="推演任务批次号")
    batch_no = Column(String(100), nullable=False, comment="推演的订单批次")
    strategy_type = Column(String(50), nullable=False, comment="策略: AI_RL / TRADITION  
AL / RANDOM")
    active_stations = Column(Integer, comment="推演最终确定的启用站台数")
    total_makespan_sec = Column(Float, comment="计算出的总完工时间(秒)")
    created_at = Column(DateTime, default=datetime.datetime.now)

class DispatchResult(Base):
    tablename = "t_dispatch_result"
    id = Column(BigInteger, primary_key=True, autoincrement=True)
    task_id = Column(String(100), nullable=False, index=True, comment="关联推演任务")
    order_id = Column(String(100), nullable=False)
    box_id = Column(String(150), nullable=False, index=True, comment="实体箱 ID")
    target_station = Column(Integer, nullable=False, comment="派往几号站台")
    predicted_start_time = Column(DateTime, nullable=False, comment="预测开工时间")
    predicted_end_time = Column(DateTime, nullable=False, comment="预测完工时间")

def init_db():
    print("="*60)
    print("🔗 正在连接 MySQL 并初始化工业数据库结构...")
    Base.metadata.create_all(bind=engine)
    print("✅ 所有数据表结构创建/映射完毕！")

def seed_test_data():

```

```

db = SessionLocal()
try:
    if db.query(StationMaster).count() == 0:
        print("🚧 检测到站台表为空，正在根据物理引擎 Config 自动灌入 16 台机床配置...")
    stations = []
    for i in range(Config.NUM_STATIONS):
        info = Config.get_branch_info(i)
        stations.append(StationMaster(
            station_id=i + 1,
            main_distance_m=Config.get_station_main_distance(i),
            branch_length_m=info["length_m"],
            is_upper=info["is_upper"]
        ))
    db.add_all(stations)
    db.commit()
    print("✅ 16 台机床物理底座数据灌入成功！这回全满编了，编号严格从 1 到 16!")
except Exception as e:
    print("💡 数据库中已有站台主数据，跳过初始化。")
    db.rollback()
    print(f"❌ 数据库操作失败: {e}")
finally:
    db.close()
if __name__ == "__main__":
    init_db()
    seed_test_data()
import os
import sys
import datetime
from sqlalchemy import create_engine, Column, Integer, String, Float, Boolean, DateTime, BigInteger
from sqlalchemy.orm import declarative_base, sessionmaker
project_root = os.path.abspath(os.path.join(os.path.dirname(__file__), '../'))
if project_root not in sys.path:
    sys.path.append(project_root)
from scenarios.order_picking.config import Config
DATABASE_URL = "mysql+pymysql://root:mengdi@127.0.0.1:3306/weichai_aps"
engine = create_engine(DATABASE_URL, echo=False)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()
class PartMaster(Base):
    __tablename__ = "t_part_master"
    id = Column(Integer, primary_key=True, autoincrement=True)
    part_type = Column(String(50), nullable=False, comment="零件类型")
    standard_p_time = Column(Float, nullable=False, comment="标准加工耗时(秒)")
    variance_p_time = Column(Float, default=0.0, comment="耗时波动方差")
class StationMaster(Base):
    __tablename__ = "t_station_master"
    station_id = Column(Integer, primary_key=True, autoincrement=False, comment="站台编号 (1-16)")
    main_distance_m = Column(Float, nullable=False, comment="主线物理距离(米) - 真实测绘")
    branch_length_m = Column(Float, nullable=False, comment="支线物理长度(米)")
    status = Column(String(20), default="ACTIVE", comment="状态: ACTIVE / BROKEN")
class OrderPool(Base):
    __tablename__ = "t_order_pool"
    order_id = Column(String(100), primary_key=True, comment="订单/拣选流水号")
    batch_no = Column(String(100), nullable=False, comment="生产批次号/波次名")
    priority_level = Column(Integer, default=1, comment="优先级")
    deadline_time = Column(DateTime, default=datetime.datetime.now, comment="交期死线")
    is_simulated = Column(Boolean, default=False, comment="是否已推演")
class OrderBOM(Base):
    __tablename__ = "t_order_bom"
    id = Column(BigInteger, primary_key=True, autoincrement=True)

```

```

    order_id = Column(String(100), nullable=False, index=True, comment="关联拣选单号")
    part_type = Column(String(50), nullable=False, comment="零件类型(SKU)")
    quantity = Column(Integer, nullable=False, comment="需求数量")
class SimulationTask(Base):
    __tablename__ = "t_simulation_task"
    task_id = Column(String(100), primary_key=True, comment="推演任务批次号")
    batch_no = Column(String(100), nullable=False, comment="推演的订单批次")
    strategy_type = Column(String(50), nullable=False, comment="策略")
    active_stations = Column(Integer, comment="启用站台数")
    total_makespan_sec = Column(Float, comment="总完工时间(秒)")
    created_at = Column(DateTime, default=datetime.datetime.now)
class DispatchResult(Base):
    __tablename__ = "t_dispatch_result"
    id = Column(BigInteger, primary_key=True, autoincrement=True)
    task_id = Column(String(100), nullable=False, index=True, comment="关联推演任务")
    order_id = Column(String(100), nullable=False)
    box_id = Column(String(150), nullable=False, index=True, comment="实体箱 ID")
    target_station = Column(Integer, nullable=False, comment="派往几号站台")
    predicted_spawn_time = Column(DateTime, nullable=False, comment="预测发车时间")
    predicted_start_time = Column(DateTime, nullable=False, comment="预测开工时间")
    predicted_end_time = Column(DateTime, nullable=False, comment="预测完工时间")
def init_db():
    print("="*60)
    print("🔗 正在连接 MySQL 并初始化工业数据库结构...")
    Base.metadata.create_all(bind=engine)
    print("✅ 所有数据表结构创建/映射完毕！")
def seed_test_data():
    db = SessionLocal()
    try:
        if db.query(StationMaster).count() == 0:
            stations = []
            for i in range(Config.NUM_STATIONS):
                info = Config.get_branch_info(i)
                stations.append(StationMaster(
                    station_id=i + 1,
                    main_distance_m=Config.STATION_EXIT_FAR_DISTANCES[i],
                    branch_length_m=info["branch_length"]
                ))
            db.add_all(stations)
            db.commit()
            print("✅ 16 台机床真实物理参数(精确测绘版)写入数据库成功！")
        except Exception as e:
            db.rollback()
            print(f"❌ 数据库操作失败: {e}")
        finally:
            db.close()
if __name__ == "__main__":
    init_db()
    seed_test_data()
import os
import sys
from typing import Dict, List, Optional
import pandas as pd
from openpyxl import load_workbook
current_dir = os.path.dirname(os.path.abspath(__file__))
project_root = os.path.abspath(os.path.join(current_dir, ".."))
if project_root not in sys.path:
    sys.path.insert(0, project_root)
from backend.database import PartMaster, SessionLocal
from scenarios.order_picking.data_paths import (
    DEFAULT_DAILY_DATE,
    SKU_TIME_DIR,
    find_daily_picking_excel,
    historical_picking_excel,
)
def _first_existing_column(df: pd.DataFrame, names: List[str], fallback_index: int):

```

```

    for name in names:
        if name in df.columns:
            return name
    if fallback_index < len(df.columns):
        return df.columns[fallback_index]
    raise ValueError(f"Missing required column. Tried names={names}, fallback_index={fallback_index}")
def find_excel_files() -> List[str]:
    files = []
    canonical = [historical_picking_excel(), find_daily_picking_excel(DEFAULT_DAILY_DATA)]
    for path in canonical:
        if path and path.exists():
            files.append(str(path))
    if not files and SKU_TIME_DIR.is_dir():
        for name in os.listdir(SKU_TIME_DIR):
            lower_name = name.lower()
            if name.startswith("~$") or not lower_name.endswith((".xlsx", ".xls")):
                continue
            files.append(str(SKU_TIME_DIR / name))
    unique_files = []
    seen = set()
    for path in files:
        abs_path = os.path.abspath(path)
        if abs_path not in seen and os.path.exists(abs_path):
            unique_files.append(abs_path)
            seen.add(abs_path)
    return unique_files
def _normalize_sku(value) -> str:
    return str(value or "").strip()
def load_clean_rows(path: str) -> pd.DataFrame:
    wb = load_workbook(path, read_only=True, data_only=True)
    ws = wb.worksheets[0]
    rows = ws.iter_rows(values_only=True)
    header = next(rows, None)
    if not header:
        wb.close()
        return pd.DataFrame()
    index_by_name = {str(name).strip(): idx for idx, name in enumerate(header) if name is not None}
    def idx(names: List[str], fallback: int) -> int:
        for name in names:
            if name in index_by_name:
                return index_by_name[name]
        return fallback
    status_idx = idx(["状态"], 8)
    sku_idx = idx(["SKU"], 9)
    name_idx = idx(["物料名称"], 10)
    picked_qty_idx = idx(["已拣选数量"], 12)
    confirm_idx = idx(["确认代码"], 16)
    records = []
    max_idx = max(status_idx, end_idx, status_idx, sku_idx, name_idx, picked_qty_idx, confirm_idx)
    for row in rows:
        if len(row) <= max_idx:
            continue
        sku = _normalize_sku(row[sku_idx])
        if not sku or sku.lower() == "nan":
            continue
        status = str(row[status_idx] or "").strip()
        confirm = str(row[confirm_idx] or "").strip()
        if status not in {"完成", "确定", ""}:
            continue
        if confirm not in {"确定", "完成", ""}:
            continue
        try:

```

```

        qty = float(row[picked_qty_idx] or 0.0)
    except Exception:
        continue
    if qty <= 0:
        continue
    start_time = pd.to_datetime(row[start_idx], errors="coerce")
    end_time = pd.to_datetime(row[end_idx], errors="coerce")
    if pd.isna(start_time) or pd.isna(end_time):
        continue
    duration = (end_time - start_time).total_seconds()
    if duration <= 0:
        continue
    records.append(
        {
            "source_file": os.path.basename(path),
            "SKU": sku,
            "物料名称": str(row[name_idx] or ""),
            "已拣选数量": qty,
            "耗时秒": duration,
            "单件耗时": duration / qty,
        }
    )
wb.close()
return pd.DataFrame.from_records(records)
def filter_long_time_outliers(df: pd.DataFrame) -> pd.DataFrame:
    if df.empty:
        return df
    q1 = float(df["单件耗时"].quantile(0.25))
    q3 = float(df["单件耗时"].quantile(0.75))
    iqr = q3 - q1
    iqr_upper = q3 + 3.0 * iqr if iqr > 0 else q3
    hard_upper = 300.0
    upper = max(iqr_upper, hard_upper)
    clean = df[df["单件耗时"] <= upper].copy()
    removed = len(df) - len(clean)
    clean.attrs["long_outlier_upper"] = upper
    clean.attrs["long_outlier_removed"] = removed
    clean.attrs["long_outlier_iqr_upper"] = iqr_upper
    clean.attrs["long_outlier_hard_upper"] = hard_upper
    return clean
def build_sku_average_times(files: Optional[List[str]] = None) -> pd.DataFrame:
    source_files = files or _find_excel_files()
    if not source_files:
        raise FileNotFoundError("No picking Excel files found under canonical raw_data directories")
    frames = []
    print("=" * 80)
    print("SKU average picking time rebuild")
    print("=" * 80)
    for path in source_files:
        clean = _load_clean_rows(path)
        frames.append(clean)
        fast_rows = int((clean["单件耗时"] <= 0.5).sum())
        print(f"Source: {path}")
        print(f" valid rows: {len(clean)}")
        print(f" unique SKU : {clean['SKU'].nunique()}")
        print(f" fast rows : {fast_rows} (unit time <= 0.5s, kept)")
    df_before_long_filter = pd.concat(frames, ignore_index=True)
    df_clean = filter_long_time_outliers(df_before_long_filter)
    upper = float(df_clean.attrs.get("long_outlier_upper", 0.0))
    removed = int(df_clean.attrs.get("long_outlier_removed", 0))
    iqr_upper = float(df_clean.attrs.get("long_outlier_iqr_upper", 0.0))
    hard_upper = float(df_clean.attrs.get("long_outlier_hard_upper", 0.0))
    print("=" * 80)
    print(f"Total valid rows before long-outlier filter: {len(df_before_long_filter)}")
    print(f"Long outlier upper bound: {upper:.3f}s per unit (max of IQR upper {iqr_upper:.3f}s a

```

```

nd hard upper {hard_upper:.3f}s)")
print(f"Removed long-outlier rows: {removed}")
print(f"Total valid rows after long-outlier filter : {len(df_clean)}")
print(f"Total unique SKU : {df_clean['SKU'].nunique()}")
print("Only long-time outliers are filtered; fast full-package picking records are kept.")
sku_stats = (
    df_clean.groupby("SKU")
    .agg(
        物料名称=("物料名称", "first"),
        出现行数=("耗时秒", "count"),
        总已拣选数量=("已拣选数量", "sum"),
        总耗时秒=("耗时秒", "sum"),
    )
    .reset_index()
)
sku_stats["单件平均耗时秒"] = (sku_stats["总耗时秒"] / sku_stats["总已拣选数量"]).round
(3)
sku_stats = sku_stats.sort_values("单件平均耗时秒", ascending=False).reset_index(drop=True)
e)
return sku_stats
def sync_to_part_master(sku_stats: pd.DataFrame, dry_run: bool = False) -> Dict[str, int]:
    if dry_run:
        return {"deleted": 0, "inserted": int(len(sku_stats))}
    db = SessionLocal()
    if hasattr(db, "__next__"):
        db = next(db)
    try:
        deleted_rows = db.query(PartMaster).delete()
        inserted = 0
        for _, row in sku_stats.iterrows():
            db.add(
                PartMaster(
                    part_type=str(row["SKU"]).strip(),
                    standard_p_time=float(row["单件平均耗时秒"]),
                )
            )
            inserted += 1
        db.commit()
        return {"deleted": int(deleted_rows), "inserted": inserted}
    except Exception:
        db.rollback()
        raise
    finally:
        db.close()
def rebuild_sku_avg_time(files: Optional[List[str]] = None, dry_run: bool = False) -> Dict[str, object]:
    source_files = [os.path.abspath(path) for path in files] if files else _find_excel_files()
    sku_stats = build_sku_average_times(source_files)
    result = sync_to_part_master(sku_stats, dry_run=dry_run)
    return {
        "source_files": source_files,
        "sku_count": int(len(sku_stats)),
        "deleted": int(result["deleted"]),
        "inserted": int(result["inserted"]),
        "dry_run": bool(dry_run),
    }
def main():
    import argparse
    parser = argparse.ArgumentParser(
        description="Rebuild t_part_master from historical picking Excel files, keeping full-package fast picks."
    )
    parser.add_argument(
        "--excel",
        action="append",
        default=None,
    )

```

```

        help="Picking Excel path. Can be provided multiple times. Default: historical picking plus
        active daily picking.",
    )
    parser.add_argument("--dry-run", action="store_true", help="Compute only; do not update dat
    abase.")
    args = parser.parse_args()
    files = [os.path.abspath(path) for path in args.excel] if args.excel else None
    sku_stats = build_sku_average_times(files)
    print("-" * 80)
    print("Top 10 slowest SKU:")
    d00_count = int(sku_stats["SKU"].astype(str).str.endswith(":D00").sum())
    a01_count = int(sku_stats["SKU"].astype(str).str.endswith(":A01").sum())
    print("-" * 80)
    print(f"SKU suffix count: D00={d00_count}, A01={a01_count}")
    print("Full SKU codes are kept separate, for example 610800050009:D00 != 610800050009:
    A01.")
    result = sync_to_part_master(sku_stats, dry_run=args.dry_run)
    print("-" * 80)
    if args.dry_run:
        print(f"Dry run complete. Would insert {result['inserted']} SKU rows into t_part_master.")
    else:
        print(f"Database updated. Deleted {result['deleted']} old rows, inserted {result['inserted']} S
        KU rows.")
    print("-" * 80)
    if __name__ == "__main__":
        main()
    from functools import lru_cache
    from pathlib import Path
    from typing import Any
    try:
        import toml
    except ModuleNotFoundError:
        toml = None
    PROJECT_ROOT = Path(__file__).resolve().parents[2]
    CONFIG_PATH = PROJECT_ROOT / "config" / "app_config.toml"
    DEFAULT_CONFIG = {
        "app": {
            "backend_port": 8088,
            "api_base_url": "http://127.0.0.1:8088/api/v1",
        },
        "datasets": {
            "active_date": "2025-07-01",
            "historical_picking_excel": "raw_data/historical/picking/DMS 拣选 20260201-0429.XLSX",
        },
        "daily_data_root": "raw_data/daily",
    },
    "inventory": {
        "cache_dir": "data/inventory",
        "morning_file_hint": "早库存单元",
        "evening_file_hint": "晚库存单元",
    },
    "simulation": {
        "default_batch_no": "ORDER WAVE 2025-07-01",
        "default_initial_snapshot_id": "2025-07-01-morning",
        "default_evening_snapshot_id": "2025-07-01-evening",
        "default_strategy": "ai",
        "default_active_station_limit": 16,
    },
    "system": {
        "num_stations": 16,
        "deadline_seconds": 30600.0,
        "max_orders_per_station": 2,
        "max_boxes_per_station": 8,
    },
    "model": {
        "active_model": "ppo_masking_model_v6.zip",
    },

```

```

        "model_dir": "output/models",
        "checkpoint_dir": "scenarios/order_picking/checkpoints_v6",
        "tensorboard_dir": "scenarios/order_picking/ppo_tensorboard_logs_v6",
    },
    "training": {
        "total_timesteps": 3_000_000,
        "initial_learning_rate": 3e-4,
        "final_learning_rate": 3e-5,
        "n_steps": 4000,
        "batch_size": 1000,
        "ent_coef": 0.005,
    },
}
def _deep_merge(base: dict, override: dict) -> dict:
    merged = dict(base)
    for key, value in override.items():
        if isinstance(value, dict) and isinstance(merged.get(key), dict):
            merged[key] = _deep_merge(merged[key], value)
        else:
            merged[key] = value
    return merged
@lru_cache(maxsize=1)
def load_app_config() -> dict:
    if not CONFIG_PATH.exists():
        return DEFAULT_CONFIG
    if tomlib is None:
        return DEFAULT_CONFIG
    with CONFIG_PATH.open("rb") as fp:
        loaded = tomlib.load(fp)
    return _deep_merge(DEFAULT_CONFIG, loaded)
def get_config_value(section: str, key: str, default: Any = None) -> Any:
    return load_app_config().get(section, {}).get(key, default)
def project_path(value: str | Path) -> Path:
    path = Path(value)
    if path.is_absolute():
        return path
    return PROJECT_ROOT / path
import os
from pathlib import Path
from typing import Iterable, Optional
from scenarios.order_picking.app_config import get_config_value, project_path
PROJECT_ROOT = Path(__file__).resolve().parents[2]
RAW_DATA_DIR = PROJECT_ROOT / "raw_data"
HISTORICAL_PICKING_DIR = project_path("raw_data/historical/picking")
DAILY_DATA_DIR = project_path(get_config_value("datasets", "daily_data_root", "raw_data/daily"))
INVENTORY_CACHE_DIR = project_path(get_config_value("inventory", "cache_dir", "data/inventory"))
SKU_TIME_DIR = RAW_DATA_DIR / "sku_time"
OUTPUT_DIR = project_path("output")
MODEL_DIR = project_path(get_config_value("model", "model_dir", "output/models"))
ACTIVE_MODEL_NAME = str(get_config_value("model", "active_model", "ppo_masking_model_v6.zip"))
DEFAULT_HISTORICAL_PICKING_NAME = "DMS 拣选 20260201-0429.XLSX"
DEFAULT_DAILY_DATE = os.environ.get(
    "WEICHAI_ACTIVE_DATE",
    str(get_config_value("datasets", "active_date", "2025-07-01")),
)
LEGACY_DAILY_DIRS = {
    "2025-07-01": RAW_DATA_DIR / "7.1",
}
def first_existing(paths: Iterable[Path]) -> Optional[Path]:
    for path in paths:
        if path and path.exists():
            return path
    return None

```

```

def historical_picking_excel() -> Path:
    configured = project_path(
        get_config_value(
            "datasets",
            "historical_picking_excel",
            "raw_data/historical/picking/DMS 拣选 20260201-0429.XLSX",
        )
    )
    candidates = [
        configured,
        RAW_DATA_DIR / DEFAULT_HISTORICAL_PICKING_NAME,
        SKU_TIME_DIR / DEFAULT_HISTORICAL_PICKING_NAME,
    ]
    found = first_existing(candidates)
    return found or candidates[0]
def daily_dir(date: str = DEFAULT_DAILY_DATE) -> Path:
    return DAILY_DATA_DIR / date
def daily_picking_dir(date: str = DEFAULT_DAILY_DATE) -> Path:
    return daily_dir(date) / "picking"
def daily_inventory_dir(date: str = DEFAULT_DAILY_DATE) -> Path:
    return daily_dir(date) / "inventory"
def legacy_daily_dir(date: str = DEFAULT_DAILY_DATE) -> Optional[Path]:
    return LEGACY_DAILY_DIRS.get(date)
def _excel_files(base_dir: Path):
    if not base_dir.exists():
        return []
    files = []
    for suffix in ("*.XLSX", "*.xlsx", "*.xls"):
        files.extend(base_dir.glob(suffix))
    return sorted(path for path in files if not path.name.startswith("~$"))
def find_daily_picking_excel(date: str = DEFAULT_DAILY_DATE) -> Optional[Path]:
    search_dirs = [daily_picking_dir(date)]
    legacy_dir = legacy_daily_dir(date)
    if legacy_dir:
        search_dirs.append(legacy_dir)
    for base_dir in search_dirs:
        for path in _excel_files(base_dir):
            if "拣选" in path.name:
                return path
    for base_dir in search_dirs:
        files = _excel_files(base_dir)
        if files:
            return files[0]
    return None
def find_inventory_excel(name_hint: str, date: str = DEFAULT_DAILY_DATE) -> Optional[Path]:
    search_dirs = [daily_inventory_dir(date)]
    legacy_dir = legacy_daily_dir(date)
    if legacy_dir:
        search_dirs.append(legacy_dir)
    for base_dir in search_dirs:
        for path in _excel_files(base_dir):
            if name_hint in path.name:
                return path
    return None
def iter_sku_time_excels():
    if SKU_TIME_DIR.exists():
        yield from _excel_files(SKU_TIME_DIR)
    hist = historical_picking_excel()
    if hist.exists():
        yield hist
    daily = find_daily_picking_excel(DEFAULT_DAILY_DATE)
    if daily and daily.exists():
        yield daily
def historical_picking_excel() -> Path:
    default_name = "DMS\u62e3\u900920260201-0429.XLSX"

```

```

configured = project_path(
    get_config_value(
        "datasets",
        "historical_picking_excel",
        f"raw_data/historical/picking/{default_name}",
    )
)
candidates = [
    configured,
    RAW_DATA_DIR / default_name,
    SKU_TIME_DIR / default_name,
]
found = first_existing(candidates)
return found or candidates[0]
def find_daily_picking_excel(date: str = DEFAULT_DAILY_DATE) -> Optional[Path]:
    search_dirs = [daily_picking_dir(date)]
    legacy_dir = legacy_daily_dir(date)
    if legacy_dir:
        search_dirs.append(legacy_dir)
    for base_dir in search_dirs:
        for path in excel_files(base_dir):
            if "\u62e3\u9009" in path.name:
                return path
    for base_dir in search_dirs:
        files = excel_files(base_dir)
        if files:
            return files[0]
    return None
def configured_model_path() -> Path:
    model_name = ACTIVE_MODEL_NAME if ACTIVE_MODEL_NAME.endswith(".zip") else f"{ACTIVE_MODEL_NAME}.zip"
    return MODEL_DIR / model_name
def latest_model_path() -> Optional[Path]:
    if not MODEL_DIR.exists():
        return None
    candidates = sorted(MODEL_DIR.glob("*.zip"), key=lambda path: path.stat().st_ctime, reverse=True)
    return candidates[0] if candidates else None
def resolve_model_path(model_path: Optional[str] = None, allow_latest: bool = True) -> Optional[Path]:
    if model_path:
        explicit = Path(model_path)
        return explicit if explicit.is_absolute() else project_path(model_path)
    configured = configured_model_path()
    if configured.exists():
        return configured
    return latest_model_path() if allow_latest else configured
class Config:
    【业务场景配置表：潍柴数字孪生分拣线(回归绝对物理法则版)】
    NUM_STATIONS = 16
    NUM_PART_TYPES = 50
    BUFFER_CAPACITY = 8
    DISPATCH_INTERVAL = 1.5
    BELT_SPEED = 1.0
    STATION_DISTANCE = 5.0
    MAIN_LINE_OFFSET = 6.0
    UPPER_BRANCH_LENGTH = 10.4
    LOWER_BRANCH_LENGTH = 9.3
    DEADLINE_SECONDS = 55000.0
    STATION_OPEN_PENALTY = 10.0
    @classmethod
    def get_station_main_distance(cls, station_idx: int) -> float:
        return cls.MAIN_LINE_OFFSET + station_idx * cls.STATION_DISTANCE
    @classmethod
    def get_branch_info(cls, station_idx: int) -> dict:
        is_upper = (station_idx % 2 != 0)

```

```

        actual_length = cls.UPPER_BRANCH_LENGTH if is_upper else cls.LOWER_BRANCH_LENGTH
        transit_time = actual_length / cls.BELT_SPEED
        return {
            "is_upper": is_upper,
            "length_m": actual_length,
            "transit_time_s": round(transit_time, 3),
        }
class Config:
    NUM_STATIONS = 16
    NUM_PART_TYPES = 50
    MAX_ORDERS_PER_STATION = 2
    MAX_BOXES_PER_STATION = 8
    2. 传送带空间与运动学约束 (物理测绘级)
    DISPATCH_INTERVAL = 1.5
    BELT_SPEED = 0.7
    各站台到【更远端出库口】的精确物理测绘距离 (单位: 米)
    STATION_EXIT_FAR_DISTANCES = [
        41.214, 44.703, 47.155, 50.811, 52.945, 56.601, 59.040, 62.696,
        70.495, 74.153, 76.335, 79.991, 83.191, 86.875, 88.517, 92.199
    ]
    远端出库口与近端出库口的固定物理位移差 (单位: 米)
    EXIT_PORT_DELTA = 3.661
    PROB_FAR_EXIT = 0.5
    PROB_NEAR_EXIT = 0.5
    BRANCH_IN_LENGTH = 9.552
    BRANCH_OUT_LENGTH = 10.395
    DEADLINE_SECONDS = 30600.0
    STATION_OPEN_PENALTY = 10.0
    @classmethod
    def get_expected_exit_time(cls, station_idx: int) -> float:
        far_dist = cls.STATION_EXIT_FAR_DISTANCES[station_idx]
        near_dist = far_dist - cls.EXIT_PORT_DELTA
        expected_dist = (cls.PROB_FAR_EXIT * far_dist) + (cls.PROB_NEAR_EXIT * near_dist)
        总体物理位移 = 支线段 + 主线期望段
        total_expected_dist = cls.BRANCH_OUT_LENGTH + expected_dist
        return total_expected_dist / cls.BELT_SPEED
    @classmethod
    def get_branch_info(cls, station_idx: int) -> dict:
        return {
            "branch_length": cls.BRANCH_IN_LENGTH,
            "transit_time_s": cls.BRANCH_IN_LENGTH / cls.BELT_SPEED
        }
import os
import sys
import gymnasium as gym
import numpy as np
import pandas as pd
from gymnasium import spaces
current_dir = os.path.dirname(os.path.abspath(__file__))
project_root = os.path.abspath(os.path.join(current_dir, "../.."))
if project_root not in sys.path:
    sys.path.insert(0, project_root)
try:
    from backend.database import PartMaster, SessionLocal
except Exception:
    PartMaster = None
    SessionLocal = None
from scenarios.order_picking.config import Config
from scenarios.order_picking.data_paths import historical_picking_excel
EXCEL_PATH = str(historical_picking_excel())
TARGET_MAKESPAN_FACTOR = 1.2
MAKESPAN_SUCCESS_REWARD = 100.0
MAKESPAN_OVERRUN_PENALTY = 0.01
MAKESPAN_MAX_PENALTY = 300.0
def find_default_excel():

```

```

    if os.path.exists(EXCEL_PATH):
        return EXCEL_PATH
    return EXCEL_PATH
EXCEL_PATH = find_default_excel()
def load_part_times_from_database():
    if PartMaster is None or SessionLocal is None:
        return {}
    db = SessionLocal()
    if hasattr(db, "__next__"):
        db = next(db)
    try:
        all_parts = db.query(PartMaster).all()
        return {str(p.part_type).strip(): float(p.standard_p_time) for p in all_parts}
    except Exception:
        return {}
    finally:
        try:
            db.close()
        except Exception:
            pass
def build_part_times_from_excel(excel_path):
    if not os.path.exists(excel_path):
        return {}
    try:
        df = pd.read_excel(excel_path, sheet_name=0)
    except Exception:
        return {}
    rows = []
    for _, row in df.iterrows():
        try:
            row_vals = row.values
            if len(row_vals) < 13:
                continue
            start_time = pd.to_datetime(row_vals[2], errors="coerce")
            end_time = pd.to_datetime(row_vals[3], errors="coerce")
            status = str(row_vals[8]).strip()
            sku = str(row_vals[9]).strip()
            picked_qty = float(row_vals[12])
            if pd.isna(start_time) or pd.isna(end_time):
                continue
            duration = (end_time - start_time).total_seconds()
            if status not in ["确定", "完成"] or not sku or picked_qty <= 0 or duration <= 0:
                continue
            rows.append((sku, duration, picked_qty, duration / picked_qty))
        except Exception:
            continue
    if not rows:
        return {}
    clean = pd.DataFrame(rows, columns=["sku", "duration", "qty", "unit_time"])
    q1 = clean["unit_time"].quantile(0.25)
    q3 = clean["unit_time"].quantile(0.75)
    upper = max(float(q3 + 3.0 * (q3 - q1)), 300.0)
    clean = clean[clean["unit_time"] <= upper]
    grouped = clean.groupby("sku").agg(duration=("duration", "sum"), qty=("qty", "sum"))
    return (grouped["duration"] / grouped["qty"]).to_dict()
def load_and_aggregate_real_orders():
    print("[RL 数据管道] 正在装载标准工艺定额...")
    part_time_dict = load_part_times_from_database()
    if part_time_dict:
        print(f"[RL 数据管道] 已从数据库读取 {len(part_time_dict)} 条 SKU 工艺定额。")
    else:
        part_time_dict = build_part_times_from_excel(EXCEL_PATH)
        print(f"[RL 数据管道] 数据库不可用，已从 Excel 计算 {len(part_time_dict)} 条 SKU 工
        艺定额。")
    print("[RL 数据管道] 正在按照【同订单同 SKU 合并为 1 箱】法则解析大盘...")
    try:

```

```

    df = pd.read_excel(EXCEL_PATH, sheet_name=0)
except Exception as exc:
    print(f"[RL 数据管道] 读取订单 Excel 失败: {EXCEL_PATH} ({exc})")
    return []
order_aggregation = {}
for _, row in df.iterrows():
    try:
        row_vals = row.values
        if len(row_vals) < 13:
            continue
        time_start = pd.to_datetime(row_vals[2])
        order_id = str(row_vals[6]).strip()
        status = str(row_vals[8]).strip()
        sku = str(row_vals[9]).strip()
        qty = float(row_vals[11])
    except Exception:
        continue
    if status not in ["确定", "完成"] or qty <= 0:
        continue
    box_process_time = part_time_dict.get(sku, 4.5) * int(qty)
    if order_id not in order_aggregation:
        order_aggregation[order_id] = {
            "order_id": order_id,
            "start_time": time_start,
            "sku_map": {},
            "total_p_time": 0.0,
        }
    if sku not in order_aggregation[order_id]["sku_map"]:
        order_aggregation[order_id]["sku_map"][sku] = 0.0
    order_aggregation[order_id]["sku_map"][sku] += box_process_time
    order_aggregation[order_id]["total_p_time"] += box_process_time
aggregated_orders = []
for data in order_aggregation.values():
    boxes = [{"sku": sku, "p_time": p_time} for sku, p_time in data["sku_map"].items()]
    data["boxes"] = boxes
    del data["sku_map"]
    aggregated_orders.append(data)
aggregated_orders.sort(key=lambda x: x["start_time"])
print(f"[RL 数据管道] 大盘聚合完成, 微观订单总数: {len(aggregated_orders)}")
return aggregated_orders
GLOBAL_ORDERS_CACHE = None
class PickingEnv(gym.Env):
    def __init__(self, dataset_type="train", initial_orders=None, **_ignored_kwargs):
        super().__init__()
        self.action_space = spaces.Discrete(Config.NUM_STATIONS)
        self.obs_dim = Config.NUM_STATIONS * 4 + 2
        self.observation_space = spaces.Box(low=-np.inf, high=np.inf, shape=(self.obs_dim,), dtype=np.float32)
        if initial_orders is not None:
            self.real_world_orders = list(initial_orders)
        else:
            global GLOBAL_ORDERS_CACHE
            if GLOBAL_ORDERS_CACHE is None:
                GLOBAL_ORDERS_CACHE = load_and_aggregate_real_orders()
            all_orders = GLOBAL_ORDERS_CACHE
            train_bound = int(len(all_orders) * 0.67)
            if dataset_type == "train":
                self.real_world_orders = all_orders[:train_bound]
            elif dataset_type == "test":
                self.real_world_orders = all_orders[train_bound:]
            else:
                self.real_world_orders = all_orders
        self.total_orders = len(self.real_world_orders)
        self.episode_length = 1000
        self.expected_exit_times = np.array(
            [Config.get_expected_exit_time(i) for i in range(Config.NUM_STATIONS)], dtype=np.f

```

```

float32
    )
    def set_orders(self, orders, episode_length=None):
        self.real_world_orders = list(orders)
        self.total_orders = len(self.real_world_orders)
        self.episode_length = episode_length or len(self.real_world_orders)
    def reset(self, seed=None, options=None):
        super().reset(seed=seed)
        max_start = max(0, self.total_orders - self.episode_length)
        self.start_idx = int(self.np_random.integers(0, max_start + 1)) if max_start > 0 else 0
        self.end_idx = min(self.start_idx + self.episode_length, self.total_orders)
        self.current_step = self.start_idx
        self.global_time = 0.0
        self.station_available_time = np.zeros(Config.NUM_STATIONS, dtype=np.float32)
        self.station_active_boxes = [[] for _ in range(Config.NUM_STATIONS)]
        return self._get_obs(), {}
    def _get_obs(self):
        for i in range(Config.NUM_STATIONS):
            self.station_active_boxes[i] = [
                b for b in self.station_active_boxes[i] if b["finish_time"] > self.global_time
            ]
        workloads = np.array(
            [max(0.0, self.station_available_time[i] - self.global_time) for i in range(Config.NUM_S
TATIONS)],
            dtype=np.float32,
        )
        boxes_count = np.array([len(self.station_active_boxes[i]) for i in range(Config.NUM_STA
TIONS)], dtype=np.float32)
        orders_count = np.array(
            [len(set(b["order_id"] for b in self.station_active_boxes[i])) for i in range(Config.NUM_
STATIONS)],
            dtype=np.float32,
        )
        station_state = np.column_stack((workloads, boxes_count, orders_count, self.expected_exit
_times)).flatten()
        if self.current_step >= self.end_idx:
            curr_info = np.array([0.0, 0.0], dtype=np.float32)
        else:
            curr_order = self.real_world_orders[self.current_step]
            curr_info = np.array([curr_order["total_p_time"], len(curr_order["boxes"])], dtype=np.fl
oat32)
        return np.concatenate((station_state, curr_info))
    def action_masks(self):
        if self.current_step >= self.end_idx:
            return np.ones(Config.NUM_STATIONS, dtype=bool)
        curr_order = self.real_world_orders[self.current_step]
        order_id = curr_order["order_id"]
        mask = np.ones(Config.NUM_STATIONS, dtype=bool)
        for i in range(Config.NUM_STATIONS):
            self.station_active_boxes[i] = [
                b for b in self.station_active_boxes[i] if b["finish_time"] > self.global_time
            ]
            active_boxes = len(self.station_active_boxes[i])
            active_order_ids = set(b["order_id"] for b in self.station_active_boxes[i])
            is_new_order = order_id not in active_order_ids
            if active_boxes >= Config.MAX_BOXES_PER_STATION:
                mask[i] = False
            elif is_new_order and len(active_order_ids) >= Config.MAX_ORDERS_PER_STATIO
N:
                mask[i] = False
        if not np.any(mask):
            return np.ones(Config.NUM_STATIONS, dtype=bool)
        return mask
    def step(self, action):
        if self.current_step >= self.end_idx:
            return self._get_obs(), 0.0, True, False, {}

```

```

old_time = self.global_time
curr_order = self.real_world_orders[self.current_step]
order_id = curr_order["order_id"]
for box in curr_order["boxes"]:
    p_time = box["p_time"]
    while True:
        self.station_active_boxes[action] = [
            b for b in self.station_active_boxes[action] if b["finish_time"] > self.global_time
        ]
        active_boxes = len(self.station_active_boxes[action])
        active_order_ids = set(b["order_id"] for b in self.station_active_boxes[action])
        is_new_order = order_id not in active_order_ids
        order_limit_hit = is_new_order and len(active_order_ids) >= Config.MAX_ORDERS
        _PER_STATION
        box_limit_hit = active_boxes >= Config.MAX_BOXES_PER_STATION
        if not order_limit_hit and not box_limit_hit:
            break
        if self.station_active_boxes[action]:
            next_finish = min(b["finish_time"] for b in self.station_active_boxes[action])
            self.global_time = max(self.global_time, next_finish)
        else:
            self.global_time += 1.0
        self.global_time += Config.DISPATCH_INTERVAL
        t_trans_in = ((action * 5.0) / Config.BELT_SPEED) + Config.get_branch_info(action)["t
ransit_time_s"]
        arrival_time = self.global_time + t_trans_in
        start_process = max(self.station_available_time[action], arrival_time)
        finish_process = start_process + p_time
        self.station_available_time[action] = finish_process
        self.station_active_boxes[action].append({"finish_time": finish_process, "order_id": orde
r_id})
        self.current_step += 1
        done = self.current_step >= self.end_idx
        if done:
            self.global_time = max(self.global_time, np.max(self.station_available_time))
            time_passed = self.global_time - old_time
            expected_time = len(curr_order["boxes"]) * Config.DISPATCH_INTERVAL
            jam_delay = max(0.0, time_passed - expected_time)
            reward = -(jam_delay * 0.5)
            reward -= self.expected_exit_times[action] * 0.001
            info = {}
            if done:
                episode_orders = self.real_world_orders[self.start_idx : self.end_idx]
                total_process_time = sum(
                    float(box.get("p_time", 0.0))
                    for order in episode_orders
                    for box in order.get("boxes", [])
                )
                target_makespan = (total_process_time / Config.NUM_STATIONS) * TARGET_MAKE
SPAN_FACTOR
                makespan = float(self.global_time)
                if target_makespan > 0:
                    if makespan <= target_makespan:
                        terminal_reward = MAKESPAN_SUCCESS_REWARD
                    else:
                        overrun = makespan - target_makespan
                        terminal_reward = -min(MAKESPAN_MAX_PENALTY, overrun * MAKESPAN_
OVERRUN_PENALTY)
                reward += terminal_reward
                info.update(
                    {
                        "makespan": makespan,
                        "target_makespan": float(target_makespan),
                        "terminal_makespan_reward": float(terminal_reward),
                    }
                )

```

```

        return self._get_obs(), float(reward), done, False, info
import os
import sys
import time
from types import SimpleNamespace
project_root = os.path.abspath(os.path.join(os.path.dirname(__file__), "../.."))
if project_root not in sys.path:
    sys.path.append(project_root)
from sb3_contrib import MaskablePPO
from sb3_contrib.common.wrappers import ActionMasker
from stable_baselines3.common.callbacks import CheckpointCallback
from scenarios.order_picking.app_config import get_config_value, project_path
from scenarios.order_picking.rl_environment import PickingEnv
TOTAL_TIMESTEPS = int(get_config_value("training", "total_timesteps", 3_000_000))
INITIAL_LEARNING_RATE = float(get_config_value("training", "initial_learning_rate", 3e-4))
FINAL_LEARNING_RATE = float(get_config_value("training", "final_learning_rate", 3e-5))
PPO_N_STEPS = int(get_config_value("training", "n_steps", 4000))
PPO_BATCH_SIZE = int(get_config_value("training", "batch_size", 1000))
PPO_ENT_COEF = float(get_config_value("training", "ent_coef", 0.005))
MODEL_NAME = str(get_config_value("model", "active_model", "ppo_masking_model_v6.zip")).replace(".zip", "")
MODEL_VERSION = MODEL_NAME.replace("ppo_masking_model_", "")
CHECKPOINT_DIR = str(project_path(get_config_value("model", "checkpoint_dir", "scenarios/order_picking/checkpoints_v6")))
TENSORBOARD_DIR = str(project_path(get_config_value("model", "tensorboard_dir", "scenarios/order_picking/ppo_tensorboard_logs_v6")))
TRAINING_SEED = int(os.environ.get("PICKING_TRAIN_SEED", str(int(time.time()))))
RESUME_TRAINING = os.environ.get("PICKING_RESUME", "0") == "1"
AUTO_SELECT_BEST_MODEL = bool(get_config_value("training", "auto_select_best_model", True))
BEST_MODEL_CHECKPOINT_STRIDE = int(get_config_value("training", "best_model_checkpoint_stride", 5))
BEST_MODEL_EVAL_ORDER_COUNT = int(get_config_value("training", "best_model_eval_order_count", 2000))
BEST_MODEL_EVAL_SEED = int(get_config_value("training", "best_model_eval_seed", 20260630))
BEST_MODEL_MIN_STATIONS = int(get_config_value("training", "best_model_min_stations", 2))
BEST_MODEL_MAX_STATIONS = int(get_config_value("training", "best_model_max_stations", 16))
BEST_MODEL_EVAL_ORDERS_FILE = str(
    project_path(get_config_value("training", "best_model_eval_orders_file", "output/model_selection/fixed_eval_orders.json"))
)
def mask_fn(env):
    return env.action_masks()
def linear_schedule(initial_value: float, final_value: float):
    def schedule(progress_remaining: float) -> float:
        return final_value + (initial_value - final_value) * progress_remaining
    return schedule
def main():
    print("=" * 80)
    print(f"Starting {MODEL_VERSION.upper()} station-assignment training")
    print("=" * 80)
    print(f"Model name: {MODEL_NAME}")
    print(f"Total timesteps: {TOTAL_TIMESTEPS:,}")
    print(f"Learning rate schedule: {INITIAL_LEARNING_RATE:g} -> {FINAL_LEARNING_RATE:g}")
    print(f"Training seed: {TRAINING_SEED}")
    print(f"Resume existing model: {RESUME_TRAINING}")
    raw_env = PickingEnv()
    env = ActionMasker(raw_env, mask_fn)
    os.makedirs(CHECKPOINT_DIR, exist_ok=True)
    checkpoint_callback = CheckpointCallback(
        save_freq=20000,
        save_path=CHECKPOINT_DIR,

```

```

    name_prefix=f'ppo_{MODEL_VERSION}_order_level',
)
model_dir = str(project_path(get_config_value("model", "model_dir", "output/models")))
os.makedirs(model_dir, exist_ok=True)
model_path = os.path.join(model_dir, MODEL_NAME + ".zip")
lr_schedule = linear_schedule(INITIAL_LEARNING_RATE, FINAL_LEARNING_RATE)
if RESUME_TRAINING and os.path.exists(model_path):
    print(f'Loading existing {MODEL_VERSION.upper()} model: {model_path}')
    model = MaskablePPO.load(
        model_path,
        env=env,
        tensorboard_log=TENSORBOARD_DIR,
        custom_objects={
            "learning_rate": lr_schedule,
            "lr_schedule": lr_schedule,
            "n_steps": PPO_N_STEPS,
            "batch_size": PPO_BATCH_SIZE,
            "ent_coef": PPO_ENT_COEF,
        },
    )
else:
    if os.path.exists(model_path):
        print(f'Existing {MODEL_VERSION.upper()} model will be replaced after training: {model_path}')
    print(f'Initializing new {MODEL_VERSION.upper()} model')
    model = MaskablePPO(
        "MlpPolicy",
        env,
        verbose=1,
        learning_rate=lr_schedule,
        n_steps=PPO_N_STEPS,
        batch_size=PPO_BATCH_SIZE,
        ent_coef=PPO_ENT_COEF,
        seed=TRAINING_SEED,
        tensorboard_log=TENSORBOARD_DIR,
    )
training_completed = False
try:
    print(f'Training {MODEL_VERSION.upper()} model...')
    model.learn(
        total_timesteps=TOTAL_TIMESTEPS,
        callback=checkpoint_callback,
        reset_num_timesteps=not RESUME_TRAINING,
    )
    training_completed = True
    print("Training finished")
except KeyboardInterrupt:
    print("Training interrupted, saving current model")
finally:
    model.save(model_path.replace(".zip", ""))
    print(f'Saved {MODEL_VERSION.upper()} model: {model_path}')
if training_completed and AUTO_SELECT_BEST_MODEL:
    print("=" * 80)
    print("Selecting best checkpoint after training")
    print("=" * 80)
    from scenarios.order_picking.select_best_model import select_best_model
    select_best_model(
        SimpleNamespace(
            eval_order_count=BEST_MODEL_EVAL_ORDER_COUNT,
            seed=BEST_MODEL_EVAL_SEED,
            checkpoint_stride=BEST_MODEL_CHECKPOINT_STRIDE,
            min_stations=BEST_MODEL_MIN_STATIONS,
            max_stations=BEST_MODEL_MAX_STATIONS,
            checkpoint_dir=CHECKPOINT_DIR,
            target_model=model_path,
            eval_orders_file=BEST_MODEL_EVAL_ORDERS_FILE,

```

```

        refresh_eval_orders=False,
    )
)
elif training_completed:
    print("Best checkpoint selection is disabled by config.")
if __name__ == "__main__":
    main()
from typing import Dict, List, Optional, Tuple
def _normalize_sku(value) -> str:
    return str(value or "").strip()
def _order_time_key(order: dict):
    return order.get("order_time") or order.get("sequence_no") or order.get("original_sequence")
or 0
def _build_inventory_view(inventory_snapshot: Optional[dict]) -> Tuple[Dict[str, float], Dict[str,
int]]:
    qty_by_sku: Dict[str, float] = {}
    unit_count_by_sku: Dict[str, int] = {}
    if not inventory_snapshot:
        return qty_by_sku, unit_count_by_sku
    for raw_sku, units in inventory_snapshot.get("sku_units", {}).items():
        sku = _normalize_sku(raw_sku)
        if not sku:
            continue
        live_units = []
        total_qty = 0.0
        for unit in units or []:
            qty = float(unit.get("remaining_qty", unit.get("qty", 0.0)) or 0.0)
            if qty <= 0:
                continue
            live_units.append(unit)
            total_qty += qty
        qty_by_sku[sku] = qty_by_sku.get(sku, 0.0) + total_qty
        unit_count_by_sku[sku] = unit_count_by_sku.get(sku, 0) + len(live_units)
    return qty_by_sku, unit_count_by_sku
def _build_inventory_units(inventory_snapshot: Optional[dict]) -> Dict[str, List[dict]]:
    units_by_sku: Dict[str, List[dict]] = {}
    if not inventory_snapshot:
        return units_by_sku
    for raw_sku, units in inventory_snapshot.get("sku_units", {}).items():
        sku = _normalize_sku(raw_sku)
        if not sku:
            continue
        units_by_sku[sku] = []
        for unit in units or []:
            qty = float(unit.get("remaining_qty", unit.get("qty", 0.0)) or 0.0)
            if qty <= 0:
                continue
            units_by_sku[sku].append(
                {
                    "remaining_qty": qty,
                    "available_at": float(unit.get("available_at", 0.0) or 0.0),
                    "unit_id": str(unit.get("unit_id", unit.get("load_unit_id", ""))).strip(),
                }
            )
        units_by_sku[sku].sort(key=lambda item: (item["available_at"], item["remaining_qty"], item["unit_id"]))
    return units_by_sku
def _order_demand(order: dict) -> Dict[str, float]:
    demand: Dict[str, float] = {}
    for box in order.get("boxes", []):
        sku = _normalize_sku(box.get("sku"))
        if not sku:
            continue
        demand[sku] = demand.get(sku, 0.0) + float(box.get("qty", 1.0) or 1.0)
    return demand
def _build_order_demand_view(orders: List[dict]) -> Tuple[Dict[str, float], Dict[str, int]]:

```

```

total_demand_by_sku: Dict[str, float] = {}
order_count_by_sku: Dict[str, int] = {}
for order in orders:
    demand = _order_demand(order)
    for sku, qty in demand.items():
        total_demand_by_sku[sku] = total_demand_by_sku.get(sku, 0.0) + qty
        order_count_by_sku[sku] = order_count_by_sku.get(sku, 0) + 1
    return total_demand_by_sku, order_count_by_sku
def _scarcity_score(order: dict, qty_by_sku: Dict[str, float], unit_count_by_sku: Dict[str, int]) -> float:
    scores = []
    for sku, qty in _order_demand(order).items():
        available_qty = qty_by_sku.get(sku, 0.0)
        available_units = unit_count_by_sku.get(sku, 0)
        if available_qty <= 0 or available_units <= 0:
            scores.append(9999.0)
            continue
        scores.append((qty / available_qty) + (1.0 / available_units))
    return max(scores) if scores else 0.0
def _estimate_inventory_wait(orders: List[dict], inventory_snapshot: Optional[dict]) -> float:
    units_by_sku = _build_inventory_units(inventory_snapshot)
    if not units_by_sku:
        return 0.0
    dispatch_cursor = 0.0
    total_wait = 0.0
    for order in orders:
        for box in order.get("boxes", []):
            sku = _normalize_sku(box.get("sku"))
            qty = float(box.get("qty", 1.0) or 1.0)
            p_time = float(box.get("p_time", 0.0) or 0.0)
            units = units_by_sku.get(sku, [])
            enough_units = [unit for unit in units if unit["remaining_qty"] >= qty]
            if not enough_units:
                continue
            ready_units = [unit for unit in enough_units if unit["available_at"] <= dispatch_cursor]
            if ready_units:
                selected = min(
                    ready_units,
                    key=lambda unit: (unit["remaining_qty"] - qty, unit["available_at"], unit["unit_id"]),
                )
            else:
                selected = min(
                    enough_units,
                    key=lambda unit: (unit["available_at"], unit["remaining_qty"] - qty, unit["unit_id"]),
                )
            wait = selected["available_at"] - dispatch_cursor
            total_wait += wait
            dispatch_cursor = selected["available_at"]
            selected["remaining_qty"] -= qty
            selected["available_at"] = dispatch_cursor + p_time
            dispatch_cursor += 1.0
    return total_wait
def _order_inventory_score(order: dict, units_by_sku: Dict[str, List[dict]], cursor: float) -> Tuple
[float, float, int, int, int]:
    waits = []
    shortage_count = 0
    tight_unit_count = 0
    for box in order.get("boxes", []):
        sku = _normalize_sku(box.get("sku"))
        qty = float(box.get("qty", 1.0) or 1.0)
        enough_units = [unit for unit in units_by_sku.get(sku, []) if unit["remaining_qty"] >= qty]
        if not enough_units:
            shortage_count += 1
            waits.append(0.0)
            continue
        if len(enough_units) <= 2:

```

```

        tight_unit_count += 1
        ready_units = [unit for unit in enough_units if unit["available_at"] <= cursor]
        if ready_units:
            waits.append(0.0)
        else:
            selected = min(enough_units, key=lambda unit: (unit["available_at"], unit["remaining_qty"] - qty, unit["unit_id"]))
            waits.append(max(0.0, selected["available_at"] - cursor))
        return (
            sum(waits),
            max(waits) if waits else 0.0,
            shortage_count,
            tight_unit_count,
            int(order.get("original_sequence", 0) or 0),
        )
def consume_order_for_inventory_sequence(order: dict, units_by_sku: Dict[str, List[dict]], cursor: float) -> float:
    finish = cursor
    for box in order.get("boxes", []):
        sku = normalize_sku(box.get("sku"))
        qty = float(box.get("qty", 1.0) or 1.0)
        p_time = float(box.get("p_time", 0.0) or 0.0)
        enough_units = [unit for unit in units_by_sku.get(sku, []) if unit["remaining_qty"] >= qty]
        if not enough_units:
            continue
        ready_units = [unit for unit in enough_units if unit["available_at"] <= cursor]
        if ready_units:
            selected = min(ready_units, key=lambda unit: (unit["remaining_qty"] - qty, unit["available_at"], unit["unit_id"]))
            start = cursor
        else:
            selected = min(enough_units, key=lambda unit: (unit["available_at"], unit["remaining_qty"] - qty, unit["unit_id"]))
            start = selected["available_at"]
            selected["remaining_qty"] -= qty
            selected["available_at"] = start + p_time
            finish = max(finish, start + p_time)
        return max(cursor + 1.0, cursor * 0.98 + finish * 0.02)
def inventory_aware_order_sequence(
    orders: List[dict],
    inventory_snapshot: Optional[dict],
    window_size: int = 10,
) -> List[dict]:
    units_by_sku = build_inventory_units(inventory_snapshot)
    if not units_by_sku:
        return list(orders)
    remaining = sorted(orders, key=lambda order: order.get("original_sequence", 0))
    output = []
    cursor = 0.0
    while remaining:
        window = remaining[: max(1, min(window_size, len(remaining)))]
        pick_idx, order = min(
            enumerate(window),
            key=lambda item: _order_inventory_score(item[1], units_by_sku, cursor),
        )
        output.append(order)
        remaining.pop(pick_idx)
        cursor = _consume_order_for_inventory_sequence(order, units_by_sku, cursor)
    return output
def scarce_skus_for_order(
    order: dict,
    qty_by_sku: Dict[str, float],
    unit_count_by_sku: Dict[str, int],
    total_demand_by_sku: Optional[Dict[str, float]] = None,
    order_count_by_sku: Optional[Dict[str, int]] = None,
    scarce_unit_threshold: int = 3,

```

```

    scarce_demand_ratio: float = 0.25,
    daily_demand_ratio: float = 0.70,
) -> set:
    total_demand_by_sku = total_demand_by_sku or {}
    order_count_by_sku = order_count_by_sku or {}
    scarce = set()
    for sku, qty in _order_demand(order).items():
        available_qty = qty_by_sku.get(sku, 0.0)
        available_units = unit_count_by_sku.get(sku, 0)
        daily_demand = total_demand_by_sku.get(sku, qty)
        daily_order_count = order_count_by_sku.get(sku, 1)
        if available_qty <= 0 or available_units <= 0:
            scarce.add(sku)
            continue
        order_need_is_large = (qty / available_qty) >= scarce_demand_ratio
        daily_need_is_tight = daily_order_count > 1 and (daily_demand / available_qty) >= daily_demand_ratio
        unit_contention = daily_order_count > available_units and available_units <= scarce_unit_threshold
        daily_qty_shortage = daily_demand > available_qty
        if order_need_is_large or daily_need_is_tight or unit_contention or daily_qty_shortage:
            scarce.add(sku)
    return scarce
def preprocess_orders(
    orders: List[dict],
    inventory_snapshot: Optional[dict] = None,
    spread_scarce_skus: bool = True,
    exclude_shortage: bool = True,
) -> dict:
    qty_by_sku, unit_count_by_sku = _build_inventory_view(inventory_snapshot)
    has_inventory = bool(qty_by_sku)
    sorted_orders = sorted(orders, key=_order_time_key)
    total_demand_by_sku, order_count_by_sku = _build_order_demand_view(sorted_orders)
    processable = []
    shortage = []
    observed_scarce_skus = set()
    spread_accepted = False
    wait_estimate_before = 0.0
    wait_estimate_after = 0.0
    for idx, order in enumerate(sorted_orders):
        copied = dict(order)
        copied.setdefault("original_sequence", idx)
        demand = _order_demand(copied)
        shortage_skus = []
        if has_inventory:
            shortage_skus = [
                sku for sku, qty in demand.items()
                if qty_by_sku.get(sku, 0.0) < qty or unit_count_by_sku.get(sku, 0) <= 0
            ]
        if shortage_skus:
            shortage_order = {
                **copied,
                "exception_reason": "initial_inventory_shortage",
                "shortage_skus": shortage_skus,
                "scarce_skus": sorted(shortage_skus),
                "scarcity_score": 9999.0,
            }
            observed_scarce_skus.update(shortage_order)
            shortage.append(shortage_order)
            if exclude_shortage:
                continue
            processable.append(shortage_order)
            continue
        copied["scarcity_score"] = _scarcity_score(copied, qty_by_sku, unit_count_by_sku) if has_inventory else 0.0
        scarce_skus = (

```

```

        _scarce_skus_for_order(
            copied,
            qty_by_sku,
            unit_count_by_sku,
            total_demand_by_sku=total_demand_by_sku,
            order_count_by_sku=order_count_by_sku,
        )
        if has_inventory
        else set()
    )
    copied["scarce_skus"] = sorted(scarce_skus)
    observed_scarce_skus.update(scarce_skus)
    processable.append(copied)
    before_spread_conflicts = count_adjacent_scarce_conflicts(processable)
    before_primary_conflicts = count_adjacent_primary_conflicts(processable)
    if spread_scarce_skus and has_inventory:
        original_processable = list(processable)
        candidate = inventory_aware_order_sequence(processable, inventory_snapshot, window_size=10)
        wait_estimate_before = _estimate_inventory_wait(original_processable, inventory_snapshot)
    )
    wait_estimate_after = _estimate_inventory_wait(candidate, inventory_snapshot)
    processable = candidate
    spread_accepted = processable != original_processable
    after_spread_conflicts = count_adjacent_scarce_conflicts(processable)
    after_primary_conflicts = count_adjacent_primary_conflicts(processable)
    scarce_skus = {
        sku for sku, count in unit_count_by_sku.items()
        if count > 0 and count <= 2
    }
    return {
        "processable_orders": processable,
        "shortage_orders": shortage,
        "preprocess_stats": {
            "input_order_count": len(orders),
            "processable_order_count": len(processable),
            "shortage_order_count": len(shortage),
            "reordered_count": sum(1 for i, o in enumerate(processable) if o.get("original_sequence")
!= i),
            "moved_order_count": sum(
                1
                for previous, current in zip(sorted(processable, key=lambda o: o.get("original_sequence", 0)), processable)
                if previous.get("order_id") != current.get("order_id")
            ),
            "scarce_sku_count": len(observed_scarce_skus),
            "inventory_low_unit_sku_count": len(scarce_skus),
            "inventory_sku_count": len(qty_by_sku),
            "inventory_wait_estimate_before": round(float(wait_estimate_before), 3),
            "inventory_wait_estimate_after": round(float(wait_estimate_after), 3),
            "spread_accepted": spread_accepted,
            "adjacent_scarce_conflicts_before": before_spread_conflicts,
            "adjacent_scarce_conflicts_after": after_spread_conflicts,
            "adjacent_primary_conflicts_before": before_primary_conflicts,
            "adjacent_primary_conflicts_after": after_primary_conflicts,
        },
    }
}
def spread_orders_by_scarce_sku(orders: List[dict], window_size: int = 8, recent_size: int = 3) -> List[dict]:
    remaining = sorted(orders, key=lambda o: o.get("original_sequence", 0))
    output = []
    recent_scarce_skus = []
    recent_primary_skus = []
    while remaining:
        previous_scarce = recent_scarce_skus[-1] if recent_scarce_skus else set()
        recent = set().union(*recent_scarce_skus[-recent_size:]) if recent_scarce_skus else set()

```

```

last_primary = recent_primary_skus[-1] if recent_primary_skus else ""
first_order = remaining[0]
first_scarce = set(first_order.get("scarce_skus", []))
first_primary = primary_sku(first_order)
first_conflicts = bool(first_scarce & previous_scarce) or bool(first_primary and first_primary)
y == last_primary)
pick_idx = 0
if first_conflicts:
    window = remaining[: max(1, window_size)]
    def candidate_score(item):
        idx, order = item
        scarce_skus = set(order.get("scarce_skus", []))
        primary = primary_sku(order)
        is_shortage = 1 if order.get("exception_reason") else 0
        same_primary = 1 if primary and primary == last_primary else 0
        immediate_overlap = len(scarce_skus & previous_scarce)
        recent_overlap = len(scarce_skus & recent)
        return (is_shortage, same_primary, immediate_overlap, recent_overlap, idx)
    best_idx, best_order = min(enumerate(window), key=candidate_score)
    best_scarce = set(best_order.get("scarce_skus", []))
    best_primary = primary_sku(best_order)
    best_conflicts = bool(best_scarce & previous_scarce) or bool(best_primary and best_primary)
    if not best_conflicts and not best_order.get("exception_reason"):
        pick_idx = best_idx
    order = remaining.pop(pick_idx)
    output.append(order)
    recent_scarce_skus.append(set(order.get("scarce_skus", [])))
    recent_primary_skus.append(primary_sku(order))
return output
def primary_sku(order: dict) -> str:
    boxes = order.get("boxes", [])
    if not boxes:
        return ""
    return normalize_sku(max(boxes, key=lambda b: float(b.get("qty", 1.0) or 1.0)).get("sku"))
def count_adjacent_scarce_conflicts(orders: List[dict]) -> int:
    conflicts = 0
    prev = set()
    for order in orders:
        current = set(order.get("scarce_skus", []))
        if prev and current and (prev & current):
            conflicts += 1
        prev = current
    return conflicts
def count_adjacent_primary_conflicts(orders: List[dict]) -> int:
    conflicts = 0
    prev = ""
    for order in orders:
        current = primary_sku(order)
        if prev and current and prev == current:
            conflicts += 1
        prev = current
    return conflicts
import argparse
import json
import os
import sys
from functools import lru_cache
from pathlib import Path
from typing import Dict, List, Optional
import pandas as pd
PROJECT_ROOT = Path(__file__).resolve().parents[2]
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))
from scenarios.order_picking.data_paths import (
    DEFAULT_DAILY_DATE,

```

```

    INVENTORY_CACHE_DIR,
    daily_inventory_dir,
    find_inventory_excel,
)
from scenarios.order_picking.app_config import get_config_value
DEFAULT_INVENTORY_DATE = DEFAULT_DAILY_DATE
DEFAULT_INVENTORY_DIR = daily_inventory_dir(DEFAULT_INVENTORY_DATE)
DEFAULT_CACHE_DIR = INVENTORY_CACHE_DIR
COLUMN_CANDIDATES = {
    "sku": ["物料编号/SKU", "鋼+枋緯棧佛/SKU", "SKU"],
    "sku_name": ["物料名称/SKU", "鋼+枋錫嶺 O/SKU", "物料名称"],
    "qty": ["数量", "鑲伴嘶"],
    "free_qty": ["空闲数量", "綰洪琴鑲伴嘶"],
    "base_unit": ["ID/基础装载单元", "ID/鋳虹 瑁皆澆鐮囙"],
    "base_status": ["状态/基础装载单元", "鐘舵€?鋳虹 瑁皆澆鐮囙"],
    "load_unit": ["ID/装载单元", "ID/瑁皆澆鐮囙"],
    "location": ["详细位置/装载单元", "璇≈欸浣嶇疆/瑁皆澆鐮囙"],
    "lock_status": ["锁定状态/装载单元", "閤估晶鐘舵€?瑁皆澆鐮囙"],
    "availability": ["可用性状态", "鑒 敷鎬 x 妍鎬?"],
}
def normalize_sku(value) -> str:
    if value is None:
        return ""
    return str(value).strip()
def _safe_float(value, default: float = 0.0) -> float:
    try:
        if pd.isna(value):
            return default
        return float(value)
    except Exception:
        return default
def _snapshot_path(snapshot_id: str, cache_dir: Path = DEFAULT_CACHE_DIR) -> Path:
    return cache_dir / f"{snapshot_id}.json"
def _date_from_snapshot_id(snapshot_id: str) -> str:
    parts = str(snapshot_id or "").split("-")
    if len(parts) >= 3:
        return "-".join(parts[:3])
    return DEFAULT_INVENTORY_DATE
def _snapshot_specs(date: str = DEFAULT_INVENTORY_DATE):
    morning_hint = str(get_config_value("inventory", "morning_file_hint", "早库存单元"))
    evening_hint = str(get_config_value("inventory", "evening_file_hint", "晚库存单元"))
    return (
        (f"{date}-morning", morning_hint),
        (f"{date}-evening", evening_hint),
    )
def find_file(name_hint: str, base_dir: Optional[Path] = None, date: str = DEFAULT_INVENTORY_DATE) -> Optional[Path]:
    if base_dir is None:
        source = find_inventory_excel(name_hint, date=date)
        if source is not None:
            return source
        base_dir = DEFAULT_INVENTORY_DIR
    if not base_dir.exists():
        return None
    for suffix in ("*.XLSX", "*.xlsx", "*.xls"):
        for path in base_dir.glob(suffix):
            if name_hint in path.name:
                return path
    return None
def _resolve_columns(df: pd.DataFrame) -> Dict[str, Optional[str]]:
    columns = {str(col).strip(): col for col in df.columns}
    resolved: Dict[str, Optional[str]] = {}
    for key, candidates in COLUMN_CANDIDATES.items():
        resolved[key] = next((columns[name] for name in candidates if name in columns), None)
    return resolved
def _get(row, columns: Dict[str, Optional[str]], key: str, default=""):

```

```

col = columns.get(key)
if col is None:
    return default
return row.get(col, default)
def read_inventory_excel(path: os.PathLike, snapshot_id: Optional[str] = None) -> dict:
    path = Path(path)
    df = pd.read_excel(path, sheet_name=0)
    columns = _resolve_columns(df)
    if not columns.get("sku") or not columns.get("base_unit"):
        raise ValueError(f"Inventory file is missing SKU/base-unit columns: {path}")
    units_by_sku: Dict[str, Dict[str, dict]] = {}
    for _, row in df.iterrows():
        sku = normalize_sku(_get(row, columns, "sku"))
        unit_id = str(_get(row, columns, "base_unit", "")).strip()
        if not sku or not unit_id or unit_id.lower() == "nan":
            continue
        qty = _safe_float(_get(row, columns, "free_qty", None), default=None)
        if qty is None:
            qty = _safe_float(_get(row, columns, "qty", 0.0), default=0.0)
        if qty <= 0:
            continue
        locked = str(_get(row, columns, "lock_status", "")).strip()
        availability = str(_get(row, columns, "availability", "")).strip()
        if "锁" in locked or "闕" in locked or "冻结" in availability or "鏽草粕" in availability:
            continue
        sku_units = units_by_sku.setdefault(sku, {})
        unit = sku_units.setdefault(
            unit_id,
            {
                "unit_id": unit_id,
                "sku": sku,
                "sku_name": str(_get(row, columns, "sku_name", "")).strip(),
                "qty": 0.0,
                "remaining_qty": 0.0,
                "available_at": 0.0,
                "status": str(_get(row, columns, "base_status", "存储")).strip() or "存储",
                "load_unit_id": str(_get(row, columns, "load_unit", "")).strip(),
                "location": str(_get(row, columns, "location", "")).strip(),
            },
        )
        unit["qty"] += qty
        unit["remaining_qty"] += qty
    sku_units = {
        sku: sorted(units.values(), key=lambda u: (u["available_at"], u["unit_id"]))
        for sku, units in units_by_sku.items()
    }
    total_units = sum(len(units) for units in sku_units.values())
    total_qty = sum(unit["remaining_qty"] for units in sku_units.values() for unit in units)
    return {
        "snapshot_id": snapshot_id or path.stem,
        "source_file": str(path),
        "sku_units": sku_units,
        "summary": {
            "sku_count": len(sku_units),
            "base_unit_count": total_units,
            "total_qty": total_qty,
        },
    }
}
def save_snapshot(snapshot: dict, cache_dir: Path = DEFAULT_CACHE_DIR) -> Path:
    cache_dir.mkdir(parents=True, exist_ok=True)
    path = _snapshot_path(snapshot["snapshot_id"], cache_dir)
    with path.open("w", encoding="utf-8") as f:
        json.dump(snapshot, f, ensure_ascii=False, indent=2)
    return path
def load_snapshot(snapshot_id: str, cache_dir: Path = DEFAULT_CACHE_DIR) -> dict:
    path = _snapshot_path(snapshot_id, cache_dir)

```

```

    if not path.exists():
        generated = ensure_default_snapshots(cache_dir=cache_dir, date=_date_from_snapshot_id
(snapshot_id))
        if snapshot_id not in generated and not path.exists():
            raise FileNotFoundError(f"Inventory snapshot not found: {snapshot_id}")
        with path.open("r", encoding="utf-8") as f:
            return json.load(f)
def list_snapshots(cache_dir: Path = DEFAULT_CACHE_DIR) -> List[dict]:
    ensure_default_snapshots(cache_dir=cache_dir)
    snapshots = []
    for path in sorted(cache_dir.glob("*.json")):
        try:
            with path.open("r", encoding="utf-8") as f:
                data = json.load(f)
                snapshots.append(
                    {
                        "snapshot_id": data.get("snapshot_id", path.stem),
                        "source_file": data.get("source_file", ""),
                        "summary": data.get("summary", {}),
                    }
                )
        except Exception:
            continue
    return snapshots
def generate_inventory_snapshots(
    date: str = DEFAULT_INVENTORY_DATE,
    cache_dir: Path = DEFAULT_CACHE_DIR,
    force: bool = False,
) -> List[dict]:
    generated = []
    for snapshot_id, hint in snapshot_specs(date):
        path = _snapshot_path(snapshot_id, cache_dir)
        if path.exists() and not force:
            with path.open("r", encoding="utf-8") as f:
                generated.append(json.load(f))
            continue
        source = _find_file(hint, date=date)
        if source is None:
            continue
        snapshot = read_inventory_excel(source, snapshot_id=snapshot_id)
        saved_path = save_snapshot(snapshot, cache_dir=cache_dir)
        snapshot["cache_file"] = str(saved_path)
        generated.append(snapshot)
    return generated
@lru_cache(maxsize=32)
def ensure_default_snapshots(cache_dir: Path = DEFAULT_CACHE_DIR, date: str = DEFAULT_INVENTORY_DATE) -> tuple:
    snapshots = generate_inventory_snapshots(date=date, cache_dir=cache_dir, force=False)
    return tuple(snapshot.get("snapshot_id") for snapshot in snapshots)
def main():
    parser = argparse.ArgumentParser(description="Generate inventory snapshot JSON files from
daily inventory Excel.")
    parser.add_argument("--date", default=DEFAULT_INVENTORY_DATE, help="Daily data d
ate, for example 2025-07-01.")
    parser.add_argument("--cache-dir", default=str(DEFAULT_CACHE_DIR), help="Inventory s
napshot cache directory.")
    parser.add_argument("--force", action="store_true", help="Rebuild snapshots even if cache JS
ON already exists.")
    parser.add_argument("--list", action="store_true", help="List cached inventory snapshots after
generation.")
    args = parser.parse_args()
    cache_dir = Path(args.cache_dir)
    snapshots = generate_inventory_snapshots(date=args.date, cache_dir=cache_dir, force=args.fo
rce)
    print("=" * 80)
    print("Inventory snapshot preprocessing")

```

```

print("=" * 80)
print(f'Date      : {args.date}')
print(f'Cache dir : {cache_dir}')
if not snapshots:
    print("No snapshots generated or found. Please check daily inventory Excel files.")
for snapshot in snapshots:
    summary = snapshot.get("summary", {})
    print(
        f'{snapshot.get("snapshot_id")}: '
        f'sku={summary.get("sku_count", 0)}, '
        f'base_units={summary.get("base_unit_count", 0)}, '
        f'source={snapshot.get("source_file", "")}'
    )
if args.list:
    print("-" * 80)
    for item in list_snapshots(cache_dir=cache_dir):
        print(item.get("snapshot_id"))
print("=" * 80)
if __name__ == "__main__":
    main()
import argparse
import glob
import json
import os
import sys
import time
from collections import defaultdict
from datetime import date
import numpy as np
import simpy
from sb3_contrib import MaskablePPO
current_dir = os.path.dirname(os.path.abspath(__file__))
project_root = os.path.abspath(os.path.join(current_dir, "../.."))
if project_root not in sys.path:
    sys.path.insert(0, project_root)
from scenarios.order_picking.config import Config
from core_engine.models.resource_model import SimpyStation
from scenarios.order_picking.data_paths import OUTPUT_DIR, MODEL_DIR, resolve_model_path
from scenarios.order_picking.rl_environment import PickingEnv
def parse_args():
    parser = argparse.ArgumentParser(
        description=(
            "Run a SimPy physical verification for the current order-picking RL logic. "
            "The script uses the same 33% test split and station deadline scan as compare.py."
        )
    )
    parser.add_argument("--date", help="Verify one date, for example 2026-04-11.")
    parser.add_argument("--date-from", dest="date_from", help="Inclusive start date.")
    parser.add_argument("--date-to", dest="date_to", help="Inclusive end date.")
    parser.add_argument("--min-stations", type=int, default=1, help="Minimum station count for deadline scan.")
    parser.add_argument("--max-stations", type=int, default=Config.NUM_STATIONS, help="Maximum station count for deadline scan.")
    parser.add_argument(
        "--optimize-deadline",
        action="store_true",
        help="Choose the minimum AI station count satisfying Config.DEADLINE_SECONDS.",
    )
    parser.add_argument("--station-limit", type=int, help="Use a fixed station count instead of scaling.")
    parser.add_argument("--model", help="Optional model .zip path. Defaults to the configured active model.")
    parser.add_argument("--seed", type=int, default=999)
    parser.add_argument(
        "--max-orders",

```

```

        type=int,
        help="Optional cap for quick screenshots. The default verifies all selected orders.",
    )
    parser.add_argument(
        "--output",
        default=os.path.join(str(OUTPUT_DIR), "simply_verify_results.json"),
        help="JSON output path.",
    )
    return parser.parse_args()
def order_date(order):
    start_time = order.get("start_time")
    if hasattr(start_time, "date"):
        return start_time.date().isoformat()
    return str(start_time)[:10]
def select_dates(orders_by_date, args):
    dates = sorted(orders_by_date)
    if args.date:
        return [args.date] if args.date in orders_by_date else []
    if args.date_from:
        dates = [d for d in dates if d >= args.date_from]
    if args.date_to:
        dates = [d for d in dates if d <= args.date_to]
    return dates
def set_orders(env, orders):
    env.unwrapped.episode_mode = "sequential"
    env.unwrapped.max_episode_orders = None
    if hasattr(env.unwrapped, "set_orders"):
        env.unwrapped.set_orders(orders, episode_length=len(orders))
    else:
        env.unwrapped.base_orders = list(orders)
        env.unwrapped.real_world_orders = list(orders)
        env.unwrapped.total_orders = len(orders)
        env.unwrapped.episode_length = len(orders)
def load_model(model_path, env):
    resolved = resolve_model_path(model_path)
    if not resolved or not resolved.exists():
        raise FileNotFoundError(f"No configured or fallback .zip model found under {MODEL_DIR}.")
    model = MaskablePPO.load(str(resolved), env=env)
    return model, str(resolved)
def run_rl_episode(orders, model, station_limit, seed):
    rl_env = PickingEnv(dataset_type="test", episode_mode="sequential", enable_inventory=False)
    set_orders(rl_env, orders)
    obs, _ = rl_env.reset(seed=seed)
    done = False
    actions = []
    enabled_mask = np.array(
        [True] * station_limit + [False] * (Config.NUM_STATIONS - station_limit),
        dtype=bool,
    )
    while not done:
        try:
            env_mask = rl_env.unwrapped.action_masks()
        except AttributeError:
            env_mask = np.ones(Config.NUM_STATIONS, dtype=bool)
        mask = np.logical_and(enabled_mask, env_mask)
        if not np.any(mask):
            mask = enabled_mask
        action, _ = model.predict(obs, action_masks=mask, deterministic=True)
        action = int(action)
        actions.append(action)
        obs, _, done, _, _ = rl_env.step(action)
    return {
        "makespan": float(rl_env.unwrapped.global_time),
        "actions": actions,
    }

```

```

        "orders": list(rl_env.unwrapped.real_world_orders),
    }
def choose_station_limit(orders, model, args):
    if args.station_limit:
        limit = max(1, min(args.station_limit, Config.NUM_STATIONS))
        rl_result = run_rl_episode(orders, model, limit, args.seed + limit)
        return limit, rl_result, [{"station_limit": limit, "ai_makespan": round(rl_result["makespan"],
3)}]
    min_stations = max(1, min(args.min_stations, Config.NUM_STATIONS))
    max_stations = max(min_stations, min(args.max_stations, Config.NUM_STATIONS))
    scan = []
    selected_limit = max_stations
    selected_result = None
    for limit in range(min_stations, max_stations + 1):
        rl_result = run_rl_episode(orders, model, limit, args.seed + limit)
        scan.append({"station_limit": limit, "ai_makespan": round(rl_result["makespan"], 3)})
        print(f" scan stations={limit:02d}, rl_makespan={rl_result['makespan']:.1f}s")
        if args.optimize_deadline and rl_result["makespan"] <= Config.DEADLINE_SECONDS:
            selected_limit = limit
            selected_result = rl_result
            break
    if selected_result is None:
        best = min(scan, key=lambda item: item["ai_makespan"])
        selected_limit = int(best["station_limit"])
        selected_result = run_rl_episode(orders, model, selected_limit, args.seed + selected_limit)
    return selected_limit, selected_result, scan
def station_active_box_count(station):
    return int(sum(station.active_orders.values()))
def station_inbound_travel_time(station_idx):
    if hasattr(Config, "get_station_main_distance"):
        main_distance = Config.get_station_main_distance(station_idx)
    elif hasattr(Config, "STATION_EXIT_FAR_DISTANCES"):
        main_distance = Config.STATION_EXIT_FAR_DISTANCES[station_idx] - (
            getattr(Config, "EXIT_PORT_DELTA", 0.0) / 2.0
        )
    else:
        main_distance = station_idx * 5.0
    branch_info = Config.get_branch_info(station_idx)
    branch_time = float(branch_info.get("transit_time_s", 0.0))
    return (main_distance / Config.BELT_SPEED) + branch_time
def make_stats():
    return {
        "peak_active_orders": np.zeros(Config.NUM_STATIONS, dtype=int),
        "peak_active_boxes": np.zeros(Config.NUM_STATIONS, dtype=int),
        "busy_times": np.zeros(Config.NUM_STATIONS, dtype=float),
        "processed_boxes": np.zeros(Config.NUM_STATIONS, dtype=int),
        "total_boxes": 0,
    }
def monitor_process(env, stations, stats):
    while True:
        for station in stations:
            sid = station.station_id
            stats["peak_active_orders"][sid] = max(stats["peak_active_orders"][sid], station.active_o
rder_count)
            stats["peak_active_boxes"][sid] = max(stats["peak_active_boxes"][sid], station_active_b
ox_count(station))
        yield env.timeout(0.1)
def dispatch_physical_orders(env, stations, orders, actions, stats):
    for order, action in zip(orders, actions):
        target_station = stations[action]
        order_id = str(order["order_id"])
        t_trans = station_inbound_travel_time(action)
        for box in order["boxes"]:
            while True:
                active_boxes = station_active_box_count(target_station)
                active_order_ids = set(target_station.active_orders)

```

```

        is_new_order = order_id not in active_order_ids
        order_limit_hit = is_new_order and len(active_order_ids) >= Config.MAX_ORDERS
    _PER_STATION
        box_limit_hit = active_boxes >= Config.MAX_BOXES_PER_STATION
        if not order_limit_hit and not box_limit_hit:
            break
        yield env.timeout(0.5)
        sku = str(box["sku"])
        p_time = float(box["p_time"])
        box_id = f"{order_id}-P{sku}-{stats['total_boxes'] + 1}"
        stats["total_boxes"] += 1
        stats["busy_times"][action] += p_time
        env.process(target_station.process_box(box_id, order_id, p_time, t_trans, sku))
        yield env.timeout(Config.DISPATCH_INTERVAL)
    while any(station.active_order_count > 0 for station in stations):
        yield env.timeout(1.0)
def run_simpy_verification(orders, actions):
    sim_env = simpy.Environment()
    stations = [
        SimpyStation(sim_env, i, getattr(Config, "MAX_ORDERS_PER_STATION", 2))
        for i in range(Config.NUM_STATIONS)
    ]
    stats = make_stats()
    sim_env.process(monitor_process(sim_env, stations, stats))
    process = sim_env.process(dispatch_physical_orders(sim_env, stations, orders, actions, stats))
    start = time.time()
    sim_env.run(until=process)
    elapsed = time.time() - start
    for station in stations:
        stats["processed_boxes"][station.station_id] = station.processed_boxes
    return sim_env.now, elapsed, stats
def stats_to_records(stats, makespan, station_limit):
    records = []
    for station_id in range(Config.NUM_STATIONS):
        busy = float(stats["busy_times"][station_id])
        records.append(
            {
                "station_id": station_id + 1,
                "enabled": station_id < station_limit,
                "peak_active_orders": int(stats["peak_active_orders"][station_id]),
                "peak_active_boxes": int(stats["peak_active_boxes"][station_id]),
                "processed_boxes": int(stats["processed_boxes"][station_id]),
                "busy_seconds": round(busy, 3),
                "utilization_pct": round((busy / makespan * 100.0), 3) if makespan > 0 else 0.0,
            }
        )
    return records
def save_results(payload, output_path):
    os.makedirs(os.path.dirname(output_path), exist_ok=True)
    with open(output_path, "w", encoding="utf-8") as fp:
        json.dump(payload, fp, ensure_ascii=False, indent=2)
def main():
    args = parse_args()
    print("=" * 80)
    print("SimPy physical verification for current order-picking RL logic")
    print("=" * 80)
    base_env = PickingEnv(dataset_type="test", episode_mode="sequential", enable_inventory=F
alse)
    model, model_path = load_model(args.model, base_env)
    print(f"Loaded model: {model_path}")
    orders_by_date = defaultdict(list)
    for order in base_env.unwrapped.base_orders:
        orders_by_date[order_date(order)].append(order)
    selected_dates = select_dates(orders_by_date, args)
    if not selected_dates:
        raise RuntimeError("No orders found for requested date range in the 33% test split.")

```

```

results = []
for idx, day in enumerate(selected_dates, start=1):
    orders = list(orders_by_date[day])
    if args.max_orders:
        orders = orders[: args.max_orders]
    print(f"[{idx}/{len(selected_dates)}] {day}: {len(orders)} order(s)")
    station_limit, rl_result, station_scan = choose_station_limit(orders, model, args)
    print(f"selected station_limit={station_limit}, rl_makespan={rl_result['makespan']:.1f}s")
    simpy_makespan, runtime_seconds, stats = run_simpy_verification(
        rl_result["orders"], rl_result["actions"]
    )
    station_records = stats_to_records(stats, simpy_makespan, station_limit)
    order_limit_ok = all(
        row["peak_active_orders"] <= Config.MAX_ORDERS_PER_STATION for row in station_records
    )
    box_limit_ok = all(row["peak_active_boxes"] <= Config.MAX_BOXES_PER_STATION for row in station_records)
    processed_boxes = int(sum(row["processed_boxes"] for row in station_records))
    result = {
        "date": day,
        "orders": len(orders),
        "boxes": processed_boxes,
        "station_limit": station_limit,
        "deadline_seconds": Config.DEADLINE_SECONDS,
        "rl_makespan_seconds": round(float(rl_result["makespan"]), 3),
        "simpy_makespan_seconds": round(float(simpy_makespan), 3),
        "runtime_seconds": round(float(runtime_seconds), 3),
        "deadline_met_by_rl": rl_result["makespan"] <= Config.DEADLINE_SECONDS,
        "deadline_met_by_simpy": simpy_makespan <= Config.DEADLINE_SECONDS,
        "order_limit_ok": order_limit_ok,
        "box_limit_ok": box_limit_ok,
        "station_scan": station_scan,
        "stations": station_records,
    }
    results.append(result)
    print(
        f"simpy_makespan={simpy_makespan:.1f}s, boxes={processed_boxes}, "
        f"order_limit_ok={order_limit_ok}, box_limit_ok={box_limit_ok}"
    )
payload = {
    "generated_on": date.today().isoformat(),
    "dataset_type": "test",
    "split_rule": "rl_environment.py uses 67% train and 33% test split.",
    "model_path": model_path,
    "results": results,
}
save_results(payload, args.output)
print("=" * 80)
print(f"Result JSON: {args.output}")
print("=" * 80)
if __name__ == "__main__":
    main()
import argparse
import glob
import json
import os
import sys
from collections import defaultdict
from datetime import date
import matplotlib.pyplot as plt
import numpy as np
project_root = os.path.abspath(os.path.join(os.path.dirname(__file__), "../.."))
if project_root not in sys.path:
    sys.path.append(project_root)
from sb3_contrib import MaskablePPO

```

```

from scenarios.order_picking.config import Config
from scenarios.order_picking.data_paths import OUTPUT_DIR, MODEL_DIR, resolve_model_path
from scenarios.order_picking.rl_environment import PickingEnv
plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei", "PingFang SC", "sans-serif"]
plt.rcParams["axes.unicode_minus"] = False
def parse_args():
    parser = argparse.ArgumentParser(
        description="Compare AI, random and round-robin order picking strategies on the test split.
    "
    )
    parser.add_argument("--date", help="Only run one day, for example 2026-04-11.")
    parser.add_argument("--date-from", dest="date_from", help="Inclusive start date, for example 2026-04-01.")
    parser.add_argument("--date-to", dest="date_to", help="Inclusive end date, for example 2026-04-29.")
    parser.add_argument("--min-stations", type=int, default=1, help="Minimum enabled stations.")
    parser.add_argument("--max-stations", type=int, default=Config.NUM_STATIONS, help="Maximum enabled stations.")
    parser.add_argument(
        "--optimize-deadline",
        action="store_true",
        help="For each day, choose the minimum AI station count that satisfies Config.DEADLINE_SECONDS.",
    )
    parser.add_argument("--seed", type=int, default=888, help="Simulation seed.")
    return parser.parse_args()
def order_date(order):
    start_time = order.get("start_time")
    if hasattr(start_time, "date"):
        return start_time.date().isoformat()
    return str(start_time)[:10]
def filter_dates(orders_by_date, args):
    dates = sorted(orders_by_date)
    if args.date:
        return [args.date] if args.date in orders_by_date else []
    if args.date_from:
        dates = [d for d in dates if d >= args.date_from]
    if args.date_to:
        dates = [d for d in dates if d <= args.date_to]
    return dates
def set_daily_orders(env, daily_orders):
    env.unwrapped.episode_mode = "sequential"
    env.unwrapped.max_episode_orders = None
    if hasattr(env.unwrapped, "set_orders"):
        env.unwrapped.set_orders(daily_orders, episode_length=len(daily_orders))
    else:
        env.unwrapped.base_orders = list(daily_orders)
        env.unwrapped.real_world_orders = list(daily_orders)
        env.unwrapped.total_orders = len(daily_orders)
        env.unwrapped.episode_length = len(daily_orders)
def load_latest_model(env):
    model_path = resolve_model_path()
    if not model_path or not model_path.exists():
        print(f"WARN: No configured or fallback .zip model found under {MODEL_DIR}; AI strategy will be skipped.")
        return None, None
    try:
        model = MaskablePPO.load(str(model_path), env=env)
        print(f"Loaded model: {model_path.name}")
        return model, str(model_path)
    except Exception as exc:
        print(f"WARN: Model load failed: {exc}; AI strategy will be skipped.")
        return None, str(model_path)
def run_simulation(env, strategy, max_stations, model=None, seed=888):
    rng = np.random.default_rng(seed)

```

```

obs, _ = env.reset(seed=seed)
if env.unwrapped.total_orders == 0:
    raise RuntimeError("No orders available after reset. Please check the selected dataset/date range.")
done = False
step_count = 0
while not done:
    if strategy == "round_robin":
        action = step_count % max_stations
    elif strategy == "random":
        action = int(rng.integers(0, max_stations))
    elif strategy == "ai":
        if model is None:
            raise ValueError("AI strategy requested but model is not loaded.")
        enabled_mask = np.array(
            [True] * max_stations + [False] * (Config.NUM_STATIONS - max_stations),
            dtype=bool,
        )
        try:
            env_mask = env.unwrapped.action_masks()
        except AttributeError:
            env_mask = np.ones(Config.NUM_STATIONS, dtype=bool)
        combined_mask = np.logical_and(enabled_mask, env_mask)
        if not np.any(combined_mask):
            combined_mask = enabled_mask
        action, _ = model.predict(obs, action_masks=combined_mask, deterministic=True)
        action = int(action)
    else:
        raise ValueError(f"Unsupported strategy: {strategy}")
    obs, _, done, _, _ = env.step(action)
    step_count += 1
return float(env.unwrapped.global_time)

def summarize_records(records, selected_dates, orders_by_date, model_path):
    by_limit = defaultdict(list)
    daily_best = defaultdict(dict)
    for record in records:
        by_limit[record["station_limit"]].append(record)
        daily_best[record["date"]][record["station_limit"]] = record
    strategies = ["round_robin", "random"]
    if any(record.get("ai_makespan") is not None for record in records):
        strategies.append("ai")
    station_summary = []
    for station_limit in sorted(by_limit):
        row = {"station_limit": station_limit}
        rows = by_limit[station_limit]
        for strategy in strategies:
            key = f"{strategy}_makespan"
            values = [r[key] for r in rows if r.get(key) is not None]
            row[f"{strategy}_avg_seconds"] = round(float(np.mean(values)), 3) if values else None
            row[f"{strategy}_max_seconds"] = round(float(np.max(values)), 3) if values else None
            row[f"{strategy}_deadline_pass_days"] = sum(v <= Config.DEADLINE_SECONDS for
v in values)
        station_summary.append(row)
    total_orders = sum(len(orders_by_date[d]) for d in selected_dates)
    summary = {
        "generated_on": date.today().isoformat(),
        "dataset_type": "test",
        "split_rule": "The rl_environment.py dataset split remains 67% train and 33% test.",
        "date_from": selected_dates[0] if selected_dates else None,
        "date_to": selected_dates[-1] if selected_dates else None,
        "dates_tested": len(selected_dates),
        "total_orders": total_orders,
        "deadline_seconds": Config.DEADLINE_SECONDS,
        "station_summary": station_summary,
        "avg_selected_stations": round(float(np.mean([r["station_limit"] for r in records])), 3) if records else None,
    }

```

```

        "deadline_met_days": sum(
            1 for r in records if r.get("ai_makespan") is not None and r["ai_makespan"] <= Config.D
EADLINE_SECONDS
        ),
        "model_path": model_path,
    }
    if "ai" in strategies:
        for station_row in station_summary:
            ai_avg = station_row.get("ai_avg_seconds")
            rr_avg = station_row.get("round_robin_avg_seconds")
            rand_avg = station_row.get("random_avg_seconds")
            station_row["ai_vs_round_robin_avg_improvement_pct"] = (
                round((rr_avg - ai_avg) / rr_avg * 100, 3) if ai_avg and rr_avg else None
            )
            station_row["ai_vs_random_avg_improvement_pct"] = (
                round((rand_avg - ai_avg) / rand_avg * 100, 3) if ai_avg and rand_avg else None
            )
        return summary
    def save_results(records, summary):
        output_dir = str(OUTPUT_DIR)
        os.makedirs(output_dir, exist_ok=True)
        detail_path = os.path.join(output_dir, "comparison_daily_results.json")
        summary_path = os.path.join(output_dir, "comparison_daily_summary.json")
        with open(detail_path, "w", encoding="utf-8") as fp:
            json.dump(records, fp, ensure_ascii=False, indent=2)
        with open(summary_path, "w", encoding="utf-8") as fp:
            json.dump(summary, fp, ensure_ascii=False, indent=2)
        return detail_path, summary_path
    def plot_summary(summary):
        station_summary = summary["station_summary"]
        station_limits = [row["station_limit"] for row in station_summary]
        plt.figure(figsize=(10, 6))
        plt.plot(
            station_limits,
            [row["random_avg_seconds"] for row in station_summary],
            marker="x",
            linestyle=":",
            color="gray",
            label="Random avg",
            linewidth=2,
        )
        plt.plot(
            station_limits,
            [row["round_robin_avg_seconds"] for row in station_summary],
            marker="o",
            linestyle="--",
            color="blue",
            label="Round-robin avg",
            linewidth=2,
        )
        if any(row.get("ai_avg_seconds") is not None for row in station_summary):
            plt.plot(
                station_limits,
                [row["ai_avg_seconds"] for row in station_summary],
                marker="D",
                linestyle="-",
                color="red",
                label="RL-Agent avg",
                linewidth=3,
            )
        plt.axhline(
            y=Config.DEADLINE_SECONDS,
            color="darkred",
            linestyle="-.",
            linewidth=2,
            label=f"Deadline ({Config.DEADLINE_SECONDS:.0f}s)",

```

```

)
plt.title(
    f"测试集日期范围 {summary['date_from']} 至 {summary['date_to']} 平均完工时间对比",
    fontsize=14,
    pad=14,
)
plt.xlabel("允许启用的最大站台数量")
plt.ylabel("平均 Makespan (秒)")
plt.xticks(station_limits)
plt.grid(True, alpha=0.3)
plt.legend(fontsize=10)
output_img = os.path.join(str(OUTPUT_DIR), "performance_comparison.png")
plt.savefig(output_img, dpi=300, bbox_inches="tight")
plt.close()
return output_img
def main():
    args = parse_args()
    min_stations = max(1, min(args.min_stations, Config.NUM_STATIONS))
    max_stations = max(min_stations, min(args.max_stations, Config.NUM_STATIONS))
    station_limits = list(range(min_stations, max_stations + 1))
    print("=" * 80)
    print("Order picking strategy comparison on the 33% test split")
    print("=" * 80)
    env = PickingEnv(dataset_type="test", episode_mode="sequential", enable_inventory=False)
    test_orders = list(getattr(env.unwrapped, "base_orders", env.unwrapped.real_world_orders))
    if not test_orders:
        raise RuntimeError("The test split is empty. Please check raw data and rl_environment.py.")
    orders_by_date = defaultdict(list)
    for order in test_orders:
        orders_by_date[order.date(order)].append(order)
    selected_dates = filter_dates(orders_by_date, args)
    if not selected_dates:
        raise RuntimeError("No orders found for the requested date range in the test split.")
    model, model_path = load_latest_model(env)
    if args.optimize_deadline and model is None:
        raise RuntimeError("--optimize-deadline requires an AI model.")
    records = []
    print(
        f"Running {len(selected_dates)} date(s), {sum(len(orders_by_date[d]) for d in selected_dates)} orders, "
        f"stations {min_stations}-{max_stations}."
    )
    for date_idx, day in enumerate(selected_dates, start=1):
        daily_orders = orders_by_date[day]
        set_daily_orders(env, daily_orders)
        print(f"[{date_idx}/{len(selected_dates)}] {day}: {len(daily_orders)} orders")
        if args.optimize_deadline:
            ai_scans = []
            selected_limit = None
            selected_ai = None
            for limit in station_limits:
                ai_ms = run_simulation(env, "ai", limit, model=model, seed=args.seed + limit)
                ai_scans.append({"station_limit": limit, "ai_makespan": round(ai_ms, 3)})
                print(f"scan AI stations={limit:02d} makespan={ai_ms:9.1f}s")
                if ai_ms <= Config.DEADLINE_SECONDS:
                    selected_limit = limit
                    selected_ai = ai_ms
                    break
            if selected_limit is None:
                best = min(ai_scans, key=lambda item: item["ai_makespan"])
                selected_limit = best["station_limit"]
                selected_ai = best["ai_makespan"]
            rr = run_simulation(env, "round_robin", selected_limit, seed=args.seed + selected_limit)
            random_ms = run_simulation(env, "random", selected_limit, seed=args.seed + selected_limit + 10000)
            records.append(

```

```

        {
            "date": day,
            "orders": len(daily_orders),
            "station_limit": selected_limit,
            "deadline_seconds": Config.DEADLINE_SECONDS,
            "deadline_met": selected_ai <= Config.DEADLINE_SECONDS,
            "round_robin_makespan": round(rr, 3),
            "random_makespan": round(random_ms, 3),
            "ai_makespan": round(float(selected_ai), 3),
            "ai_station_scan": ai_scans,
        }
    )
    print(
        f" selected={selected_limit:02d} AI={selected_ai:9.1f}s "
        f"RR={rr:9.1f}s Random={random_ms:9.1f}s"
    )
    continue
for limit in station_limits:
    rr = run_simulation(env, "round_robin", limit, seed=args.seed + limit)
    random_ms = run_simulation(env, "random", limit, seed=args.seed + limit + 10000)
    ai_ms = run_simulation(env, "ai", limit, model=model, seed=args.seed + limit) if model e
lse None
    records.append(
        {
            "date": day,
            "orders": len(daily_orders),
            "station_limit": limit,
            "round_robin_makespan": round(rr, 3),
            "random_makespan": round(random_ms, 3),
            "ai_makespan": round(ai_ms, 3) if ai_ms is not None else None,
        }
    )
    if model:
        print(
            f" stations={limit:02d} AI={ai_ms:9.1f}s RR={rr:9.1f}s Random={random_ms:9.1
f}s"
        )
    else:
        print(f" stations={limit:02d} RR={rr:9.1f}s Random={random_ms:9.1f}s")
    summary = summarize_records(records, selected_dates, orders_by_date, model_path)
    detail_path, summary_path = save_results(records, summary)
    image_path = plot_summary(summary)
    print("=" * 80)
    print(f"Detail JSON : {detail_path}")
    print(f"Summary JSON: {summary_path}")
    print(f"Chart      : {image_path}")
    print("=" * 80)
if __name__ == "__main__":
    main()
import argparse
import json
import os
import re
import shutil
import sys
from datetime import datetime
from pathlib import Path
import numpy as np
from sb3_contrib import MaskablePPO
CURRENT_DIR = Path(__file__).resolve().parent
PROJECT_ROOT = CURRENT_DIR.parents[1]
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))
from scenarios.order_picking.app_config import get_config_value, project_path
from scenarios.order_picking.config import Config
from scenarios.order_picking.data_paths import configured_model_path

```

```

from scenarios.order_picking.rl_environment import PickingEnv
DEFAULT_OUTPUT_DIR = project_path("output/model_selection")
DEFAULT_EVAL_ORDERS_PATH = DEFAULT_OUTPUT_DIR / "fixed_eval_orders.json"
STEP_PATTERN = re.compile(r"_(\d+)_steps\.zip$")
def parse_args():
    parser = argparse.ArgumentParser(
        description=(
            "Select the best PPO checkpoint by evaluating fixed test orders with AI only. "
            "The score is the sum of makespan values from max stations down to min stations."
        )
    )
    parser.add_argument("--eval-order-count", type=int, default=2000, help="Fixed sampled test order count.")
    parser.add_argument("--seed", type=int, default=20260630, help="Seed used when fixed eval orders are first created.")
    parser.add_argument("--checkpoint-stride", type=int, default=5, help="Evaluate every Nth checkpoint after sorting by step.")
    parser.add_argument("--min-stations", type=int, default=2, help="Minimum station count.")
    parser.add_argument("--max-stations", type=int, default=Config.NUM_STATIONS, help="Maximum station count.")
    parser.add_argument(
        "--checkpoint-dir",
        default=str(project_path(get_config_value("model", "checkpoint_dir", "scenarios/order_picking/checkpoints_v6"))),
        help="Directory containing checkpoint .zip files.",
    )
    parser.add_argument(
        "--target-model",
        default=str(configured_model_path()),
        help="Final model path to overwrite with the best checkpoint.",
    )
    parser.add_argument(
        "--eval-orders-file",
        default=str(DEFAULT_EVAL_ORDERS_PATH),
        help="JSON file storing fixed eval order IDs.",
    )
    parser.add_argument(
        "--refresh-eval-orders",
        action="store_true",
        help="Resample and overwrite fixed eval order IDs.",
    )
    return parser.parse_args()
def checkpoint_step(path: Path) -> int:
    match = STEP_PATTERN.search(path.name)
    return int(match.group(1)) if match else -1
def select_checkpoints(checkpoint_dir: Path, stride: int):
    checkpoints = sorted(checkpoint_dir.glob("*.zip"), key=lambda path: (checkpoint_step(path), path.name))
    if not checkpoints:
        raise FileNotFoundError(f"No checkpoint .zip files found under {checkpoint_dir}")
    stride = max(1, int(stride))
    selected = checkpoints[::stride]
    if checkpoints[-1] not in selected:
        selected.append(checkpoints[-1])
    return selected, checkpoints
def load_test_orders():
    env = PickingEnv(dataset_type="test")
    return list(getattr(env.unwrapped, "real_world_orders", []))
def order_key(order, index):
    return str(order.get("order_id") or f"__idx_{index}")
def load_or_create_fixed_orders(test_orders, args):
    output_path = Path(args.eval_orders_file)
    output_path.parent.mkdir(parents=True, exist_ok=True)
    order_by_id = {order_key(order, idx): order for idx, order in enumerate(test_orders)}
    if output_path.exists() and not args.refresh_eval_orders:
        with output_path.open("r", encoding="utf-8") as f:

```

```

        payload = json.load(f)
        order_ids = payload.get("order_ids", [])
        missing_ids = [order_id for order_id in order_ids if order_id not in order_by_id]
        if missing_ids:
            raise RuntimeError(
                f"Fixed eval order file contains {len(missing_ids)} missing order IDs. "
                f"Use --refresh-eval-orders if the source data changed."
            )
        return [order_by_id[order_id] for order_id in order_ids], payload
    if len(test_orders) < args.eval_order_count:
        raise RuntimeError(f"Test split has only {len(test_orders)} orders, less than {args.eval_order_count}.")
    rng = np.random.default_rng(args.seed)
    sampled_indices = sorted(rng.choice(len(test_orders), size=args.eval_order_count, replace=False).tolist())
    order_ids = [order_key(test_orders[idx], idx) for idx in sampled_indices]
    payload = {
        "created_at": datetime.now().isoformat(timespec="seconds"),
        "source": "DMS picking test split, last 33 percent orders",
        "seed": args.seed,
        "eval_order_count": args.eval_order_count,
        "order_ids": order_ids,
    }
    with output_path.open("w", encoding="utf-8") as f:
        json.dump(payload, f, ensure_ascii=False, indent=2)
    return [test_orders[idx] for idx in sampled_indices], payload
def set_eval_orders(env, orders):
    env.unwrapped.set_orders(list(orders), episode_length=len(orders))
def run_ai_makespan(model, env, orders, station_limit, seed):
    set_eval_orders(env, orders)
    obs, _ = env.reset(seed=seed)
    done = False
    enabled_mask = np.array(
        [True] * station_limit + [False] * (Config.NUM_STATIONS - station_limit),
        dtype=bool,
    )
    while not done:
        try:
            env_mask = env.unwrapped.action_masks()
        except AttributeError:
            env_mask = np.ones(Config.NUM_STATIONS, dtype=bool)
            action_mask = np.logical_and(enabled_mask, env_mask)
            if not np.any(action_mask):
                action_mask = enabled_mask
            action, _ = model.predict(obs, action_masks=action_mask, deterministic=True)
            obs, _, done, _ = env.step(int(action))
    return float(env.unwrapped.global_time)
def evaluate_checkpoint(checkpoint_path, eval_env, orders, station_limits, seed):
    model = MaskablePPO.load(str(checkpoint_path), env=eval_env)
    station_results = []
    for station_limit in station_limits:
        makespan = run_ai_makespan(model, eval_env, orders, station_limit, seed + station_limit)
        station_results.append(
            {
                "station_limit": int(station_limit),
                "ai_makespan": round(float(makespan), 3),
            }
        )
    score = sum(item["ai_makespan"] for item in station_results)
    return {
        "checkpoint": str(checkpoint_path),
        "checkpoint_name": checkpoint_path.name,
        "checkpoint_step": checkpoint_step(checkpoint_path),
        "station_results": station_results,
        "score_total_makespan": round(float(score), 3),
        "score_avg_makespan": round(float(score / len(station_results)), 3),
    }

```

```

    }
def save_json(path, payload):
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", encoding="utf-8") as f:
        json.dump(payload, f, ensure_ascii=False, indent=2)
def select_best_model(args=None):
    if args is None:
        args = parse_args()
    checkpoint_dir = Path(args.checkpoint_dir)
    target_model = Path(args.target_model)
    output_dir = DEFAULT_OUTPUT_DIR
    station_limits = list(range(min(args.max_stations, Config.NUM_STATIONS), max(1, args.min_stations) - 1, -1))
    print("=" * 80)
    print("PPO checkpoint selection")
    print("=" * 80)
    print(f"Checkpoint dir : {checkpoint_dir}")
    print(f"Target model : {target_model}")
    print(f"Eval orders : {args.eval_order_count}")
    print(f"Station limits : {station_limits[0]} -> {station_limits[-1]}")
    print(f"Stride : {args.checkpoint_stride}")
    selected_checkpoints, all_checkpoints = select_checkpoints(checkpoint_dir, args.checkpoint_stride)
    print(f"Checkpoints : {len(selected_checkpoints)} selected / {len(all_checkpoints)} total")
    test_orders = load_test_orders()
    eval_orders, eval_payload = load_or_create_fixed_orders(test_orders, args)
    print(f"Fixed orders : {len(eval_orders)} ({args.eval_orders_file})")
    eval_env = PickingEnv(dataset_type="test", initial_orders=eval_orders)
    results = []
    for idx, checkpoint in enumerate(selected_checkpoints, start=1):
        print(f"[{idx}/{len(selected_checkpoints)}] Evaluating {checkpoint.name}")
        result = evaluate_checkpoint(checkpoint, eval_env, eval_orders, station_limits, args.seed)
        results.append(result)
        print(
            f"score={result['score_total_makespan']:.3f}, "
            f"avg={result['score_avg_makespan']:.3f}"
        )
    best = min(results, key=lambda item: item["score_total_makespan"])
    target_model.parent.mkdir(parents=True, exist_ok=True)
    shutil.copy2(best["checkpoint"], target_model)
    detail_report = {
        "generated_at": datetime.now().isoformat(timespec="seconds"),
        "metric": "minimum sum of AI makespan from max stations down to min stations",
        "station_limits": station_limits,
        "checkpoint_stride": args.checkpoint_stride,
        "evaluated_checkpoint_count": len(results),
        "total_checkpoint_count": len(all_checkpoints),
        "eval_orders_file": str(Path(args.eval_orders_file)),
        "eval_order_count": len(eval_orders),
        "eval_orders_seed": eval_payload.get("seed"),
        "target_model": str(target_model),
        "results": results,
    }
    best_report = {
        "generated_at": detail_report["generated_at"],
        "best_checkpoint": best["checkpoint"],
        "best_checkpoint_name": best["checkpoint_name"],
        "best_checkpoint_step": best["checkpoint_step"],
        "target_model": str(target_model),
        "score_total_makespan": best["score_total_makespan"],
        "score_avg_makespan": best["score_avg_makespan"],
        "station_results": best["station_results"],
        "eval_order_count": len(eval_orders),
        "eval_orders_file": str(Path(args.eval_orders_file)),
    }
    detail_path = output_dir / "checkpoint_eval_results.json"

```



```

best_path = output_dir / "best_model_report.json"
save_json(detail_path, detail_report)
save_json(best_path, best_report)
print("-" * 80)
print(f"Best checkpoint: {best['checkpoint_name']}")
print(f"Best score : {best['score_total_makespan']:.3f}")
print(f"Model replaced : {target_model}")
print(f"Detail report : {detail_path}")
print(f"Best report : {best_path}")
print("-" * 80)
return best_report

def main():
    select_best_model(parse_args())
if __name__ == "__main__":
    main()
import simpy
import numpy as np
class LogicalStation:
    def __init__(self, station_id, capacity):
        self.station_id = station_id
        self.capacity = capacity
        self.box_finish_times = []
        self.free_at = 0.0
    @property
    def current_load(self):
        return len(self.box_finish_times)
    def update_load_at_time(self, current_time):
        self.box_finish_times = [t for t in self.box_finish_times if t > current_time]
    def reset(self):
        self.box_finish_times = []
        self.free_at = 0.0
class SimpyStation:
    供连续时间沙盘高保真推演的物理站台 (纯净高效排产版)
    def __init__(self, env: simpy.Environment, station_id: int, capacity: int, logger=None):
        self.env = env
        self.station_id = station_id
        self.logger = logger
        self.buffer = simpy.Store(env)
        self.machine = simpy.Resource(env, capacity=1)
        self.processed_boxes = 0
        self.active_orders = {}
        self.process_coroutine = self.env.process(self._run_process())
    @property
    def active_order_count(self):
        return len(self.active_orders)
    def process_box(self, box_id: str, order_id: str, p_time: float, travel_time: float, entity_type: in
t):
        if order_id not in self.active_orders:
            self.active_orders[order_id] = 0
        self.active_orders[order_id] += 1
        yield self.env.timeout(travel_time)
        if self.logger:
            self.logger.log_event(self.env.now, box_id, "enter_buffer", self.station_id)
        box_data = {
            "box_id": box_id,
            "order_id": order_id,
            "p_time": p_time,
        }
        yield self.buffer.put(box_data)
    def _run_process(self):
        while True:
            box = yield self.buffer.get()
            o_id = box['order_id']
            with self.machine.request() as worker_req:
                yield worker_req
                if self.logger:

```

```

        self.logger.log_event(self.env.now, box['box_id'], "start_process", self.station_id)
        yield self.env.timeout(box['p_time'])
        self.processed_boxes += 1
        if self.logger:
            self.logger.log_event(self.env.now, box['box_id'], "end_process", self.station_id)
        self.active_orders[o_id] -= 1
        if self.active_orders[o_id] <= 0:
            del self.active_orders[o_id]
class AGVRobot:
    def __init__(self, env: simpy.Environment, agv_id: str, speed: float, battery_capacity: float = 100.0):
        self.env = env
        self.agv_id = agv_id
        self.speed = speed
        self.battery = simpy.Container(env, capacity=battery_capacity, init=battery_capacity)
        self.resource = simpy.Resource(env, capacity=1)
        self.total_distance = 0.0
    def move_and_deliver(self, distance: float, load_time: float, unload_time: float):
        with self.resource.request() as req:
            yield req
            yield self.env.timeout(load_time)
            travel_time = distance / self.speed
            yield self.env.timeout(travel_time)
            power_consumed = distance * 0.1
            if self.battery.level > power_consumed:
                yield self.battery.get(power_consumed)
            self.total_distance += distance
            yield self.env.timeout(unload_time)
class ConveyorBelt:
    def __init__(self, env: simpy.Environment, length: float, speed: float, capacity: int):
        self.env = env
        self.length = length
        self.speed = speed
        self.travel_time = length / speed
        self.slots = simpy.Resource(env, capacity=capacity)
    def transport_item(self, item_id: str):
        with self.slots.request() as req:
            yield req
            yield self.env.timeout(self.travel_time)
class SetupStation:
    def __init__(self, env: simpy.Environment, station_id: str):
        self.env = env
        self.station_id = station_id
        self.worker = simpy.Resource(env, capacity=1)
        self.current_part_type = None
    def process_with_setup(self, item_id: str, p_time: float, part_type: int, setup_time: float = 15.0):
        with self.worker.request() as req:
            yield req
            if self.current_part_type is not None and self.current_part_type != part_type:
                yield self.env.timeout(setup_time)
            self.current_part_type = part_type
            yield self.env.timeout(p_time)
import simpy
import numpy as np
[第一部分] 现有模型：固定站台
class LogicalStation:
    def __init__(self, station_id, capacity):
        self.station_id = station_id
        self.capacity = capacity
        self.box_finish_times = []
        self.free_at = 0.0
    @property
    def current_load(self):
        return len(self.box_finish_times)
    def update_load_at_time(self, current_time):

```

```

        self.box_finish_times = [t for t in self.box_finish_times if t > current_time]
    def reset(self):
        self.box_finish_times = []
        self.free_at = 0.0
class SimpyStation:
    def __init__(self, env: simpy.Environment, station_id: int, capacity: int, logger=None):
        self.env = env
        self.station_id = station_id
        self.logger = logger
        self.machine = simpy.Resource(env, capacity=1)
        self.processed_boxes = 0
        self.active_orders = {}
    @property
    def active_order_count(self):
        return len(self.active_orders)
    def process_box(self, box_id: str, order_id: str, p_time: float, travel_time: float, entity_type: in
t):
        if order_id not in self.active_orders:
            self.active_orders[order_id] = 0
        self.active_orders[order_id] += 1
        if travel_time > 0:
            yield self.env.timeout(travel_time)
        if self.logger:
            self.logger.log_event(self.env.now, box_id, "enter buffer", self.station_id)
4. 申请刀具锁死加工 (这一步会自动阻塞, 直到机床空闲, 完美防止穿模)
        with self.machine.request() as worker_req:
            yield worker_req
            if self.logger:
                self.logger.log_event(self.env.now, box_id, "start process", self.station_id)
            yield self.env.timeout(p_time)
            self.processed_boxes += 1
            if self.logger:
                self.logger.log_event(self.env.now, box_id, "end process", self.station_id)
            self.active_orders[order_id] -= 1
            if self.active_orders[order_id] <= 0:
                del self.active_orders[order_id]
[第二部分] 新增模型: 通用动态物流资源 (完全保留防误删)
class AGVRobot:
    def __init__(self, env: simpy.Environment, agv_id: str, speed: float, battery_capacity: float = 1
00.0):
        self.env = env
        self.agv_id = agv_id
        self.speed = speed
        self.battery = simpy.Container(env, capacity=battery_capacity, init=battery_capacity)
        self.resource = simpy.Resource(env, capacity=1)
        self.total_distance = 0.0
    def move_and_deliver(self, distance: float, load_time: float, unload_time: float):
        with self.resource.request() as req:
            yield req
            yield self.env.timeout(load_time)
            travel_time = distance / self.speed
            yield self.env.timeout(travel_time)
            power_consumed = distance * 0.1
            if self.battery.level > power_consumed:
                yield self.battery.get(power_consumed)
            self.total_distance += distance
            yield self.env.timeout(unload_time)
class ConveyorBelt:
    def __init__(self, env: simpy.Environment, length: float, speed: float, capacity: int):
        self.env = env
        self.length = length
        self.speed = speed
        self.travel_time = length / speed
        self.slots = simpy.Resource(env, capacity=capacity)
    def transport_item(self, item_id: str):
        with self.slots.request() as req:

```

```

        yield req
        yield self.env.timeout(self.travel_time)
class SetupStation:
    def __init__(self, env: simpy.Environment, station_id: str):
        self.env = env
        self.station_id = station_id
        self.worker = simpy.Resource(env, capacity=1)
        self.current_part_type = None
    def process_with_setup(self, item_id: str, p_time: float, part_type: int, setup_time: float = 15.0):
        with self.worker.request() as req:
            yield req
            if self.current_part_type is not None and self.current_part_type != part_type:
                yield self.env.timeout(setup_time)
            self.current_part_type = part_type
            yield self.env.timeout(p_time)
import json
import os
class PhysicalEntity:
    def __init__(self, entity_id: str, entity_type: int, qty: int, single_p_time: float):
        self.entity_id = entity_id
        self.entity_type = entity_type
        self.qty = qty
        self.single_p_time = single_p_time
        🌟 核心修正：这个箱子在机床里的【真实总加工耗时】 = 数量 × 单件耗时
        self.p_time = round(single_p_time * qty, 1)
    def __repr__(self):
        return f"<Box {self.entity_id:<18} | Type:{self.entity_type:02d} | 数量:{self.qty:2d}个 | 单件:{self.single_p_time:5.1f}s | 整箱耗时:{self.p_time:6.1f}s>"
class LogicalOrder:
    def __init__(self, order_id: str):
        self.order_id = order_id
        self.entities = []
        self.total_process_time = 0.0
    def add_entity(self, entity: PhysicalEntity):
        self.entities.append(entity)
        self.total_process_time += entity.p_time
    @property
    def num_entities(self):
        return len(self.entities)
    def __repr__(self):
        return f"📦 Order [{self.order_id}] | 共 {self.num_entities} 箱物料 | 订单总加工流逝时间: {self.total_process_time:.1f}s"
class DataLoader:
    @staticmethod
    def get_project_root():
        current_dir = os.path.dirname(os.path.abspath(__file__))
        return os.path.abspath(os.path.join(current_dir, '../..'))
    @classmethod
    def load_parts_master(cls) -> dict:
        file_path = os.path.join(cls.get_project_root(), 'data', 'weichai_parts_master.json')
        if not os.path.exists(file_path):
            with open(file_path, 'r', encoding='utf-8') as f:
                return json.load(f)
    @classmethod
    def load_history_orders(cls) -> list:
        parts_master = cls.load_parts_master()
        file_path = os.path.join(cls.get_project_root(), 'data', 'weichai_history_orders.json')
        if not os.path.exists(file_path):
            with open(file_path, 'r', encoding='utf-8') as f:
                raw_orders = json.load(f)
        logical_orders = []
        for raw_order in raw_orders:
            order_obj = LogicalOrder(order_id=raw_order['order_id'])
            for item in raw_order['items']:
                part_type_str = str(item['part_type'])

```

```

    if part_type_str not in parts_master:
        print(f"⚠️ 警告: 订单 {order_obj.order_id} 包含了未知的零件类型 {part_type_str}, 已跳过。")
        continue
    real_single_p_time = parts_master[part_type_str]['process_time_sec']
    box_id = f"{order_obj.order_id}-P{part_type_str}"
    entity_obj = PhysicalEntity(
        entity_id=box_id,
        entity_type=int(part_type_str),
        qty=item['quantity'],
        single_p_time=real_single_p_time
    )
    order_obj.add_entity(entity_obj)
    logical_orders.append(order_obj)
    return logical_orders
if __name__ == "__main__":
    print("="*80)
    print("🚀 正在测试 DataLoader 数据解耦引擎 (数量与耗时修正版)...")
    print("="*80)
    try:
        orders = DataLoader.load_history_orders()
        for order in orders[:3]:
            print(order)
            for box in order.entities:
                print(f"  -> {box}")
            print(f"...n(共成功加载 {len(orders)} 个真实历史订单)")
            print("="*80)
            print("✅ Step 1.2 数据解析与灌装完美通过! 底层算法已具备车间级真实精度!")
    except Exception as e:
        print(f"❌ 测试失败: {e}")
import simpy
class ConveyorModel:
    def __init__(self, env: simpy.Environment, conveyor_id: str, length: float, speed: float):
        self.env = env
        self.conveyor_id = conveyor_id
        self.length = length
        self.speed = speed
        self.motor_capacity = simpy.Resource(env, capacity=100)
    def calculate_travel_time(self, start_pos: float, end_pos: float) -> float:
        if self.speed <= 0:
            raise ValueError(f"[{self.conveyor_id}] 传送带速度必须大于 0!")
        distance = abs(end_pos - start_pos)
        if distance > self.length:
            raise ValueError(f"[{self.conveyor_id}] 致命越界: 位移距离 ({distance}m) 超出传送带物理总长 ({self.length}m)! ")
        return distance / self.speed
    def transport_entity(self, start_pos: float, end_pos: float):
        travel_time = self.calculate_travel_time(start_pos, end_pos)
        yield self.env.timeout(travel_time)
import numpy as np
class DispatchRules:
    @staticmethod
    def rule_round_robin(step_counter: int, num_stations: int, valid_masks: np.ndarray = None) -> int:
        【传统启发式】: 严格轮询 (Round Robin)
        高级特性: 如果传入了 valid_masks (掩码防弹衣), 遇到满载爆仓的站台会自动跳过, 顺延给下一个闲人。
        base_idx = step_counter % num_stations
        if valid_masks is None:
            return base_idx
        for i in range(num_stations):
            check_idx = (base_idx + i) % num_stations
            if valid_masks[check_idx]:
                return check_idx
        return base_idx
    @staticmethod

```

```

def rule_shortest_processing_time(predicted_costs: np.ndarray, valid_masks: np.ndarray = None) -> int:
    【传统启发式】：最短处理时间优先 / 贪心算法 (SPT)
    if valid_masks is None:
        return int(np.argmin(predicted_costs))
    masked_costs = np.copy(predicted_costs)
    masked_costs[~valid_masks] = np.inf
    return int(np.argmin(masked_costs))
    @staticmethod
def rule_ai_policy(model, observation: np.ndarray, valid_masks: np.ndarray = None) -> int:
    【神级策略】：神经网络降维打击 (RL/PPO)
    action, _states = model.predict(observation, action_masks=valid_masks, deterministic=True)
e)
    return int(action)
import numpy as np
class DispatchRules:
    [第一部分] 现有核心策略 (完全保留，未做修改)
    @staticmethod
    def rule_ai_policy(model, obs, valid_masks):
        if not np.any(valid_masks):
            return None
        action, _ = model.predict(obs, action_masks=valid_masks, deterministic=True)
        return int(action)
    @staticmethod
    def rule_round_robin(step_counter: int, num_stations: int, valid_masks):
        if not np.any(valid_masks):
            return None
        start_idx = step_counter % num_stations
        for i in range(num_stations):
            idx = (start_idx + i) % num_stations
            if valid_masks[idx]:
                return idx
        return np.where(valid_masks)[0][0]
    [第二部分] 新增扩充策略 (丰富规则库，增加对比实验维度)
    @staticmethod
    def rule_random(valid_masks):
        if not np.any(valid_masks):
            return None
        valid_indices = np.where(valid_masks)[0]
        return np.random.choice(valid_indices)
    @staticmethod
    def rule_least_load(current_loads: list, valid_masks):
        if not np.any(valid_masks):
            return None
        valid_indices = np.where(valid_masks)[0]
        best_idx = valid_indices[0]
        min_load = current_loads[best_idx]
        for idx in valid_indices:
            if current_loads[idx] < min_load:
                min_load = current_loads[idx]
                best_idx = idx
        return best_idx
    @staticmethod
    def rule_spt(processing_times: list, valid_masks):
        if not np.any(valid_masks):
            return None
        valid_indices = np.where(valid_masks)[0]
        best_idx = valid_indices[0]
        min_time = processing_times[best_idx]
        for idx in valid_indices:
            if processing_times[idx] < min_time:
                min_time = processing_times[idx]
                best_idx = idx
        return best_idx
    将此代码追加到 core_engine/rules/dispatch_rules.py 末尾 (在 DispatchRules 类内部)
    @staticmethod

```

```

def rule_fifo(valid_masks):
    if not np.any(valid_masks):
        return None
    return np.where(valid_masks)[0][0]
将此代码追加到 core_engine/rules/dispatch_rules.py 末尾 (在 DispatchRules 类内部)
@staticmethod
def rule_edd(orders: list, valid_masks):
    if not np.any(valid_masks):
        return None
    raise NotImplementedError("EDD 策略暂未实装, 预留接口用于未来支持交期约束场景。")
import numpy as np
from typing import List, Any
class ControlRules:
    在强化学习中, 它生成动作掩码 (Action Masks); 在传统仿真中, 它是发车推杆的物理锁。
    @staticmethod
    def generate_capacity_masks(stations: List[Any], incoming_qty: int = 1) -> np.ndarray:
        【红线控制】: 生成容量准入掩码 (防爆仓安检门)
        - stations: 站台对象列表 (传入我们在 resource_models 里定义的积木)
        - 布尔数组 (True 表示允许进件, False 表示已爆仓禁止进件)
        num_stations = len(stations)
        masks = np.ones(num_stations, dtype=bool)
        for i, station in enumerate(stations):
            if hasattr(station, 'current_load') and hasattr(station, 'capacity'):
                if station.current_load + incoming_qty > station.capacity:
                    masks[i] = False
            elif isinstance(station, dict) and 'load' in station and 'capacity' in station:
                if station['load'] + incoming_qty > station['capacity']:
                    masks[i] = False
        if not np.any(masks):
            return np.ones(num_stations, dtype=bool)
        return masks
    @staticmethod
    def check_global_starvation(stations: List[Any]) -> bool:
        【状态探针】: 检测是否全线饥饿 (所有站台都处于空闲/无货状态)
        for station in stations:
            if hasattr(station, 'current_load') and station.current_load > 0:
                return False
        return True
import os
import sys
import numpy as np
from typing import List, Any
project_root = os.path.abspath(os.path.join(os.path.dirname(__file__), '../..'))
if project_root not in sys.path:
    sys.path.append(project_root)
from scenarios.order_picking.config import Config
class ControlRules:
    @staticmethod
    def generate_capacity_masks(stations: List[Any]) -> np.ndarray:
        num_stations = len(stations)
        masks = np.ones(num_stations, dtype=bool)
        for i, station in enumerate(stations):
            if hasattr(station, 'active_order_count'):
                if station.active_order_count >= Config.MAX_ORDERS_PER_STATION:
                    masks[i] = False
            elif isinstance(station, dict) and 'active_order_count' in station:
                if station['active_order_count'] >= Config.MAX_ORDERS_PER_STATION:
                    masks[i] = False
        🚨 RL 框架防崩溃兜底机制 (极其重要) 🚨
        让环境 (Environment) 通过虚拟时钟的暴增 (死等时间) 来给予 AI 巨额扣分惩罚。
        if not np.any(masks):
            return np.ones(num_stations, dtype=bool)
        return masks
    @staticmethod

```

```

def check_global_starvation(stations: List[Any]) -> bool:
    for station in stations:
        if hasattr(station, 'active_order_count') and station.active_order_count > 0:
            return False
    return True
import math
from typing import List
class RoutingRules:
    @staticmethod
    def calculate_uniform_time(distance: float, speed: float) -> float:
        if speed <= 0:
            raise ValueError("速度必须严格大于 0! ")
        return distance / speed
    @staticmethod
    def calculate_network_travel_time(segments: List[float], speed: float) -> float:
        total_distance = sum(segments)
        return RoutingRules.calculate_uniform_time(total_distance, speed)
    @staticmethod
    def calculate_kinematic_time(distance: float, max_speed: float, acceleration: float) -> float:
        if distance <= 0:
            return 0.0
        if max_speed <= 0 or acceleration <= 0:
            raise ValueError("最大速度和加速度必须严格大于 0! ")
        达到最大速度所需的单边加速/减速距离 (S = V^2 / 2a)
        accel_dist = (max_speed ** 2) / (2 * acceleration)
        if distance >= 2 * accel_dist:
            accel_time = max_speed / acceleration
            uniform_dist = distance - 2 * accel_dist
            uniform_time = uniform_dist / max_speed
            return (2 * accel_time) + uniform_time
        else:
            peak_speed = math.sqrt(distance * acceleration)
            return 2 * (peak_speed / acceleration)
import { createApp } from 'vue'
import App from './App.vue'
import ElementPlus from 'element-plus'
import 'element-plus/dist/index.css'
const app = createApp(App)
app.use(ElementPlus)
app.mount('#app')
<script setup>
import { ref, reactive, nextTick, onMounted } from 'vue'
import api from './api/index'
import Factory3D from './components/Factory3D.vue'
import AlgorithmResults from './components/AlgorithmResults.vue'
const batchNo = ref("")
const inventorySnapshotId = ref("")
const eveningSnapshotId = ref("")
const inventorySnapshots = ref([])
const orderBatches = ref([])
const isSimulating = ref(false)
const isRefreshingSkuTime = ref(false)
const progress = ref(0)
const statusMessage = ref('等待下发云端推演指令...')
const simResult = ref(null)
const factoryRef = ref(null)
const stationStatus = ref(Array(16).fill().map(() => ({ active: false, orderCount: 0, maxOrders: 2,
isPoweredOff: false })))
const kpiData = reactive({
    ordersDone: 0,
    ordersTotal: 0,
    boxesDone: 0,
    boxesActive: 0,
    boxesTotal: 0,
    progressPct: 0,
    currentTime: 0,

```



```

    maxTime: 0
  })
  let pollTimer = null
  const handleStart = async () => {
    isSimulating.value = true
    progress.value = 0
    simResult.value = null
    statusMessage.value = '正在呼叫后台 AI 调度大脑...'
    activeTab.value = 'sim'
    try {
      const res = await api.startSimulation(batchNo.value, inventorySnapshotId.value, eveningSnapshotId.value)
      if (res.code === 200) {
        startPolling(res.task_id)
      }
    } catch (error) {
      statusMessage.value = '呼叫失败！ 请检查 FastAPI 后端 8088 端口是否运行！'
      isSimulating.value = false
    }
  }
  const handleRefreshSkuTime = async () => {
    isRefreshingSkuTime.value = true
    statusMessage.value = '正在刷新 SKU 平均处理时间...'
    try {
      const res = await api.rebuildSkuTime()
      const data = res.data || {}
      const inserted = data.inserted ?? 0
      const removed = data.removed_outliers ?? 0
      statusMessage.value = `SKU 工时刷新完成： 写入 ${inserted} 条，过滤极长异常 ${removed} 条`
    } catch (error) {
    } finally {
      isRefreshingSkuTime.value = false
    }
  }
  const loadInventorySnapshots = async () => {
    try {
      const res = await api.getInventorySnapshots()
      if (res.code === 200 && Array.isArray(res.data)) {
        inventorySnapshots.value = res.data
      }
    } catch (error) {
      inventorySnapshots.value = [inventorySnapshotId.value, eveningSnapshotId.value]
        .filter(Boolean)
        .map(snapshotId => ({ snapshot_id: snapshotId, summary: {} }))
    }
  }
  const loadOrderBatches = async () => {
    try {
      const res = await api.getOrderBatches()
      if (res.code === 200 && Array.isArray(res.data)) {
        orderBatches.value = res.data
        if (!batchNo.value && orderBatches.value.length > 0) {
          batchNo.value = orderBatches.value[0].batch_no
        }
        if (batchNo.value && !orderBatches.value.some(item => item.batch_no === batchNo.value)) {
          orderBatches.value.unshift({ batch_no: batchNo.value, order_count: 0 })
        }
      }
    } catch (error) {
      if (batchNo.value) {
        orderBatches.value = [{ batch_no: batchNo.value, order_count: 0 }]
      }
    }
  }
}

```

```

const loadAppConfig = async () => {
  try {
    const res = await api.getAppConfig()
    if (res.code === 200 && res.data) {
      batchNo.value = res.data.default_batch_no || batchNo.value
      inventorySnapshotId.value = res.data.default_initial_snapshot_id || inventorySnapshotId.value
      eveningSnapshotId.value = res.data.default_evening_snapshot_id || eveningSnapshotId.value
    }
  } catch (error) {
    batchNo.value = batchNo.value || ""
    inventorySnapshotId.value = inventorySnapshotId.value || ""
    eveningSnapshotId.value = eveningSnapshotId.value || ""
  }
}

const startPolling = (taskId) => {
  pollTimer = setInterval(async () => {
    try {
      const res = await api.getSimulationStatus(taskId)
      if (res.code === 200) {
        progress.value = parseInt(res.data.progress.replace('%', ''))
        statusMessage.value = res.data.message || '推演中...'
        if (res.data.status === 'failed') {
          clearInterval(pollTimer)
          isSimulating.value = false
          return
        }
        if (res.data.status === 'completed' || progress.value === 100) {
          clearInterval(pollTimer)
          isSimulating.value = false
          simResult.value = res.data
          statusMessage.value = '宏观排产完成！正在解析 3D 微观时空剧本...'
          await nextTick()
          fetchPlaybookAndPlay(taskId)
        }
      }
    } catch (error) {
      clearInterval(pollTimer)
    }
  }, 1000)
}

const fetchPlaybookAndPlay = async (taskId) => {
  try {
    const res = await api.getSimulationPlaybook(taskId)
    if (res.code === 200) {
      statusMessage.value = '数据链加载完毕，物理引擎接管！'
      factoryRef.value.loadAndPlay(res.data)
    }
  } catch (error) {
    statusMessage.value = '获取 3D 剧本失败！'
  }
}

const onUpdateStations = (statusArray) => {
  stationStatus.value = statusArray
}

const onUpdateKpi = (data) => {
  Object.assign(kpiData, data)
}

onMounted(async () => {
  await loadAppConfig()
  await Promise.all([loadInventorySnapshots(), loadOrderBatches()])
})
</script>
<template>
  <div class="app-container">
    <div class="header">

```

```

<div class="logo-area"><img alt="潍柴 APS 智能排产数字孪生大屏" data-bbox="350 85 375 100"/> 潍柴 APS 智能排产数字孪生大屏 <span class="version-tag">
V5.1</span></div>
<div class="tab-switch">
  <div class="tab-btn" :activeTab === 'algo' ? 'active' : '' @click="activeTab = 'algo'">
    <img alt="智能算法对比" data-bbox="195 135 215 150"/> 智能算法对比
  </div>
  <div class="tab-btn" :activeTab === 'sim' ? 'active' : '' @click="activeTab = 'sim'">
    <img alt="3D 数字孪生" data-bbox="195 175 215 190"/> 3D 数字孪生
  </div>
</div>
</div>
<div class="layout-main">
  <div class="left-panel" v-show="activeTab === 'sim'">
    <div class="control-box">
      <div class="panel-title"><img alt="孪生排产控制中枢" data-bbox="375 260 395 275"/> 孪生排产控制中枢</div>
      <div style="display: flex; gap: 10px; flex-wrap: wrap;">
        <select class="custom-select batch-select" v-model="batchNo">
          <option v-for="item in orderBatches" :key="item.batch_no" :value="item.batch_no">
            {{ item.batch_no }} {{ item.order_count ? ` (${item.order_count}单)` : '' }}
          </option>
        </select>
        <button class="primary-btn" :disabled="isSimulating" @click="handleStart">
          {{ isSimulating ? '计算中...' : '⚡ 启动' }}
        </button>
        <button class="secondary-btn" :disabled="isSimulating || isRefreshingSkuTime" @click
="handleRefreshSkuTime">
          {{ isRefreshingSkuTime ? '刷新中...' : '刷新 SKU 工时' }}
        </button>
      </div>
      <div class="inventory-selectors">
        <label>
          <span>日初库存快照</span>
          <select class="custom-select" v-model="inventorySnapshotId">
            <option v-for="item in inventorySnapshots" :key="item.snapshot_id" :value="item.sn
apshot_id">
              {{ item.snapshot_id }}
            </option>
          </select>
        </label>
        <label>
          <span>日末校验快照</span>
          <select class="custom-select" v-model="eveningSnapshotId">
            <option v-for="item in inventorySnapshots" :key="item.snapshot_id" :value="item.sn
apshot_id">
              {{ item.snapshot_id }}
            </option>
          </select>
        </label>
      </div>
      <div v-if="isSimulating || isRefreshingSkuTime || progress > 0" class="progress-box">
        <div class="progress-track">
          <div class="progress-fill" :style="{ width: progress + '%' }"></div>
        </div>
        <p class="status-msg">{{ statusMessage }}</p>
      </div>
    </div>
    <div class="kpi-monitor" v-if="simResult">
      <div class="panel-title"><img alt="订单履约总进度" data-bbox="375 795 395 810"/> 订单履约总进度</div>
      <div class="progress-track" style="margin-bottom: 15px; height: 12px; border-radius: 6px;">
        <div class="progress-fill" :style="{ width: kpiData.progressPct + '%' }"></div>
      </div>
      <div class="kpi-row highlight">
        <span>主订单完工进度:</span>
        <span class="kpi-val">{{ kpiData.ordersDone }} / {{ kpiData.ordersTotal }}</span>
      </div>
    </div>
  </div>

```

```

<div class="kpi-row">
  <span>总计投放实体箱:</span>
  <span class="kpi-val">{{ kpiData.bboxesTotal }}</span>
</div>
<div class="kpi-row">
  <span>传送/排队中 (在途):</span>
  <span class="kpi-val text-warning">{{ kpiData.bboxesActive }}</span>
</div>
<div class="kpi-row">
  <span>已下线 (完成):</span>
  <span class="kpi-val text-success">{{ kpiData.bboxesDone }}</span>
</div>
<div class="kpi-divider"></div>
<div class="kpi-row sub">
  <span>包含回库在内的总耗时:</span>
  <span class="kpi-val">{{ kpiData.maxTime.toFixed(1) }} s</span>
</div>
<div class="kpi-divider"></div>
<div class="kpi-row sub">
  <span>库存预处理订单:</span>
  <span class="kpi-val">{{ simResult.inventory_result?.preprocess_stats?.processable_order_count || 0 }} / {{ simResult.inventory_result?.preprocess_stats?.input_order_count || 0 }}</span>
</div>
<div class="kpi-row sub">
  <span>缺料异常订单:</span>
  <span class="kpi-val text-warning">{{ simResult.inventory_result?.exception_order_count || 0 }}</span>
</div>
<div class="kpi-row sub">
  <span>稀缺 SKU 数量:</span>
  <span class="kpi-val">{{ simResult.inventory_result?.preprocess_stats?.scarce_sku_count || 0 }}</span>
</div>
<div class="kpi-row sub">
  <span>预处理重排订单:</span>
  <span class="kpi-val">{{ simResult.inventory_result?.preprocess_stats?.reordered_count || 0 }}</span>
</div>
</div>
<div class="station-monitor">
  <div class="panel-title">🚂 产线 16 站台实时状态矩阵</div>
  <div class="st-grid">
    <div v-for="(item, idx) in stationStatus" :key="idx"
      :class="['st-card', item.isPoweredOff ? 'off' : (item.active ? 'busy' : 'idle')]">
      <div class="st-header">
        <span class="st-id">S{{ (idx+1).toString().padStart(2, '0') }}</span>
        <span class="st-buffer" v-if="!item.isPoweredOff">订单: {{ item.orderCount }}/{{ item.maxOrders }}</span>
      </div>
      <div class="st-status-text">
        <span class="st-dot"></span>
        {{ item.isPoweredOff ? 'AI 节能休眠' : (item.active ? '组装中' : '待命中') }}
      </div>
    </div>
  </div>
</div>
<div class="right-panel">
  <AlgorithmResults v-if="activeTab === 'algo'" :simResult="simResult" />
  <Factory3D
    v-show="activeTab === 'sim'"
    ref="factoryRef"
    @update-stations="onUpdateStations"
    @update-kpi="onUpdateKpi"
  />

```

```

    </div>
  </div>
</div>
</template>
<style scoped>
.app-container { width: 100vw; height: 100vh; background-color:
.header {
  display: flex; justify-content: space-between; align-items: center;
  padding: 15px 30px; background: rgba(15, 25, 40, 0.9); border-bottom: 2px solid
}
.logo-area { font-size: 20px; font-weight: 900; text-shadow: 0 0 10px rgba(0, 229, 255, 0.5); letter-spacing: 1px; color:
.version-tag { font-size: 11px; background:
.tab-switch { display: flex; background: rgba(0, 0, 0, 0.5); border-radius: 8px; border: 1px solid
.tab-btn { padding: 8px 20px; font-size: 15px; font-weight: bold; cursor: pointer; color:
.tab-btn:hover { color:
.tab-btn.active { background:
.layout-main { display: flex; flex: 1; padding: 15px; gap: 15px; height: calc(100vh - 65px); box-sizing: border-box; }
.left-panel { flex: 0 0 400px; display: flex; flex-direction: column; gap: 15px; overflow-y: auto; padding-right: 5px; }
.left-panel::-webkit-scrollbar { width: 6px; }
.left-panel::-webkit-scrollbar-thumb { background:
.right-panel { flex: 1; position: relative; border-radius: 12px; overflow: hidden; box-shadow: 0 0 30px rgba(0, 0, 0, 0.5); }
.control-box, .station-monitor, .kpi-monitor { background: rgba(15, 25, 40, 0.7); padding: 20px; border-radius: 10px; border: 1px solid
.panel-title { font-size: 15px; color:
.custom-input { flex: 1; padding: 8px 12px; background: rgba(0, 0, 0, 0.4); border: 1px solid
.custom-input:focus { border-color:
.batch-select { flex: 1; min-width: 0; }
.inventory-selectors { display: grid; grid-template-columns: 1fr 1fr; gap: 10px; margin-top: 12px; }
.inventory-selectors label { display: flex; flex-direction: column; gap: 6px; color:
.custom-select { width: 100%; padding: 8px 10px; background: rgba(0, 0, 0, 0.4); border: 1px solid
.custom-select:focus { border-color:
.primary-btn { background: linear-gradient(135deg,
.primary-btn:hover:not(:disabled) { transform: translateY(-1px); box-shadow: 0 4px 10px rgba(0, 229, 255, 0.4); }
.primary-btn:disabled { opacity: 0.5; cursor: not-allowed; }
.secondary-btn { background: rgba(0, 229, 255, 0.08); border: 1px solid rgba(0, 229, 255, 0.55); padding: 8px 14px; border-radius: 6px; color:
.secondary-btn:hover:not(:disabled) { background: rgba(0, 229, 255, 0.16); transform: translateY(-1px); }
.secondary-btn:disabled { opacity: 0.5; cursor: not-allowed; }
.progress-box { margin-top: 15px; }
.progress-track { width: 100%; height: 16px; background:
.progress-fill { height: 100%; background: linear-gradient(90deg,
.status-msg { margin-top: 8px; color:
.kpi-row { display: flex; justify-content: space-between; align-items: center; margin-bottom: 8px; font-size: 13px; color:
.kpi-row.highlight { color:
.kpi-val { font-family: monospace; font-size: 16px; font-weight: bold; color:
.text-warning { color:
.text-success { color:
.kpi-divider { height: 1px; border-top: 1px dashed
.kpi-row.sub { font-size: 12px; }
.st-grid { display: grid; grid-template-columns: repeat(4, 1fr); gap: 8px; }
.st-card { background: rgba(0,0,0,0.4); border: 1px solid
.st-header { display: flex; justify-content: space-between; align-items: center; margin-bottom: 6px; }
.st-id { color:
.st-buffer { font-size: 10px; color:
.st-status-text { font-size: 12px; font-weight: bold; display: flex; align-items: center; gap: 5px; }
.st-dot { width: 8px; height: 8px; border-radius: 50%; }

```

```

.st-card.busy { border-color:
.st-card.busy .st-status-text { color:
.st-card.busy .st-dot { background:
.st-card.busy .st-id { color:
.st-card.busy .st-buffer { background: rgba(0, 229, 255, 0.2); border-color:
.st-card.idle { border-color:
.st-card.idle .st-status-text { color:
.st-card.idle .st-dot { background:
.st-card.off { border-color:
.st-card.off .st-status-text { color:
.st-card.off .st-dot { background:
.st-card.off .st-id { text-decoration: line-through; color:
</style>
import axios from 'axios'
const request = axios.create({
  baseURL: '/api/v1',
  timeout: 300000
})
request.interceptors.response.use(response => {
  return response.data
}, error => {
  console.error('API request failed:', error)
  return Promise.reject(error)
})
export default {
  uploadMasterData(data) {
    return request.post('/master_data/upload', data)
  },
  uploadOrders(data) {
    return request.post('/orders/upload', data)
  },
  startSimulation(batchNo, inventorySnapshotId, eveningSnapshotId) {
    return request.post('/simulation/start', {
      batch_no: batchNo,
      inventory_snapshot_id: inventorySnapshotId,
      evening_snapshot_id: eveningSnapshotId,
      shortage_policy: 'exception_queue'
    })
  },
  getSimulationStatus(taskId) {
    return request.get(`/simulation/status/${taskId}`)
  },
  getSimulationPlaybook(taskId) {
    return request.get(`/simulation/playbook/${taskId}`)
  },
  startScheduleDispatch(batchNo, inventorySnapshotId, strategy = 'ai', activeStationLimit = 16)
  {
    return request.post('/schedule/dispatch', {
      batch_no: batchNo,
      inventory_snapshot_id: inventorySnapshotId,
      strategy,
      active_station_limit: activeStationLimit
    })
  },
  getScheduleResult(taskId) {
    return request.get(`/schedule/result/${taskId}`)
  },
  getInventorySnapshots() {
    return request.get('/inventory/snapshots')
  },
  getOrderBatches() {
    return request.get('/orders/batches')
  },
  getAppConfig() {
    return request.get('/app/config')
  },

```

```

rebuildSkuTime() {
  return request.post('/sku-time/rebuild')
},
getInventorySummary(snapshotId) {
  return request.get(`/inventory/summary/${snapshotId}`)
},
getTrainingMetrics() {
  return request.get('/model/training_metrics')
}
}
<script setup>
import { ref, onMounted, onBeforeUnmount, reactive } from 'vue'
import * as THREE from 'three'
import { OrbitControls } from 'three/examples/jsm/controls/OrbitControls'
const containerRef = ref(null)
const labelsContainerRef = ref(null)
const stationsContainerRef = ref(null)
const emit = defineEmits(['update-stations', 'update-kpi'])
const BOX_S = 0.5;
const SURFACE_LOWER = 0.3;
const SURFACE_UPPER = 3.5;
const Y_LOWER = SURFACE_LOWER + BOX_S / 2;
const Y_UPPER = SURFACE_UPPER + BOX_S / 2;
const START_X = -90;
const currentConfig = {
  num_stations: 16,
  belt_speed: 0.7,
  station_positions: [41, 45, 47, 50, 52, 56, 59, 62, 70, 74, 76, 79, 83, 86, 88, 92],
  branch_in_length: 9.552,
  branch_out_length: 10.395
};
const branchLen = currentConfig.branch_in_length;
const branchOutLen = currentConfig.branch_out_length;
let scene, camera, renderer, controls, clock;
let animationId = null;
const isPlaying = ref(false)
const playbackSpeed = ref(5)
const currentTimeStr = ref('0.0')
const timeSliderVal = ref(0)
let simData = [];
const activeBoxes = new Map();
const stations = [];
let currentSimTime = 0;
let maxTime = 100;
let lastTime = 0;
let orderStats = { total: 0, completed: 0, map: {} };
const kpiData = reactive({
  ordersDone: 0, ordersTotal: 0,
  boxesDone: 0, boxesActive: 0, boxesTotal: 0,
  progressPct: 0, currentTime: 0, maxTime: 0
})
const tooltipParams = reactive({
  visible: false,
  x: 0, y: 0,
  orderId: "", partType: "", stName: ""
})
function togglePlay() {
  if (simData.length === 0) return;
  isPlaying.value = !isPlaying.value;
  if (isPlaying.value) clock.start();
  else clock.stop();
}
function setSpeed(speed) {
  playbackSpeed.value = Number(speed);
}
defineExpose({

```

```

loadAndPlay(playbook) {
  parsePlaybook(playbook);
  isPlaying.value = true;
  clock.start();
},
togglePlay,
setSpeed
})
function createWarningTexture() {
  const canvas = document.createElement('canvas'); canvas.width = 256; canvas.height = 256; const ctx = canvas.getContext('2d'); ctx.fillStyle = '#FFB800'; ctx.fillRect(0,0,256,256); ctx.fillStyle = '#111111';
  for(let i = -256; i < 512; i += 40) { ctx.beginPath(); ctx.moveTo(i, 0); ctx.lineTo(i+20, 0); ctx.lineTo(i+256+20, 256); ctx.lineTo(i+256, 256); ctx.fill(); }
  const tex = new THREE.CanvasTexture(canvas); tex.wrapS = THREE.RepeatWrapping; tex.wrapT = THREE.RepeatWrapping; tex.repeat.set(1, 2); return tex;
}
function stringToColor(str) {
  let hash = 0; for (let i = 0; i < str.length; i++) hash = str.charCodeAt(i) + ((hash << 5) - hash);
  return new THREE.Color(`hsl(${Math.abs(hash) % 360}, 85%, 60%)`);
}
function lerp(s, e, p) { return s + (e - s) * Math.max(0, Math.min(1, p)); }
function createBeltCore(len, color, y, z, hideFrontRail=false, centerX=0) {
  const group = new THREE.Group();
  const track = new THREE.Mesh(new THREE.BoxGeometry(len, 0.1, 1.2), new THREE.MeshStandardMaterial({ color: color }));
  track.position.set(centerX, y - 0.05, z); track.receiveShadow = true; group.add(track);
  const rMat = new THREE.MeshStandardMaterial({ color: 0xA0B2C6, metalness: 0.7 });
  if (!hideFrontRail) { const rF = new THREE.Mesh(new THREE.BoxGeometry(len, 0.2, 0.05), rMat); rF.position.set(centerX, y + 0.05, z + 0.6); group.add(rF); }
  const rB = new THREE.Mesh(new THREE.BoxGeometry(len, 0.2, 0.05), rMat); rB.position.set(centerX, y + 0.05, z - 0.6); group.add(rB);
  return group;
}
const initScene = () => {
  clock = new THREE.Clock();
  scene = new THREE.Scene(); scene.background = new THREE.Color(0x08121C);
  camera = new THREE.PerspectiveCamera(45, containerRef.value.clientWidth / containerRef.value.clientHeight, 0.1, 1000);
  camera.position.set(-20, 90, 140); camera.lookAt(20, 0, 0);
  renderer = new THREE.WebGLRenderer({ antialias: true, powerPreference: "high-performance", alpha: true });
  renderer.setSize(containerRef.value.clientWidth, containerRef.value.clientHeight);
  renderer.shadowMap.enabled = true; renderer.shadowMap.type = THREE.PCFSoftShadowMap;
  containerRef.value.appendChild(renderer.domElement);
  controls = new OrbitControls(camera, renderer.domElement);
  controls.enableDamping = true; controls.maxPolarAngle = Math.PI / 2.05;
  scene.add(new THREE.AmbientLight(0xffffff, 0.8)); scene.add(new THREE.HemisphereLight(0xffffff, 0x445577, 0.6));
  const dirLight = new THREE.DirectionalLight(0xffffff, 1.2); dirLight.position.set(30, 80, 40);
  dirLight.castShadow = true; dirLight.shadow.mapSize.width = 2048; dirLight.shadow.mapSize.height = 2048; scene.add(dirLight);
  const floor = new THREE.Mesh(new THREE.PlaneGeometry(450, 200), new THREE.MeshStandardMaterial({ color: 0x0A1525, roughness: 0.15, metalness: 0.8 }));
  floor.rotation.x = -Math.PI / 2; floor.position.y = -0.1; floor.receiveShadow = true; scene.add(floor);
  scene.add(new THREE.GridHelper(300, 100, 0x1E3A5F, 0x112233));
  const factoryGroup = new THREE.Group();
  scene.add(factoryGroup);
  const posArray = currentConfig.station_positions;
  const maxDist = Math.max(...posArray);
  const BELT_LEN = maxDist + 20;
  const centerX = START_X + BELT_LEN / 2;
  const whGroup = new THREE.Group();
  const whX = START_X - 8; const whFrameMat = new THREE.MeshStandardMaterial({color: 0xE8F0FE, roughness: 0.2, metalness: 0.2});

```



```

const frameBottom = new THREE.Mesh(new THREE.BoxGeometry(16, 2, 14), whFrameMat);
frameBottom.position.set(whX, 1, 0); whGroup.add(frameBottom);
const frameTop = new THREE.Mesh(new THREE.BoxGeometry(16, 2, 14), whFrameMat);
frameTop.position.set(whX, 13, 0); whGroup.add(frameTop);
const frameBack = new THREE.Mesh(new THREE.BoxGeometry(2, 10, 14), whFrameMat);
frameBack.position.set(whX - 7, 7, 0); whGroup.add(frameBack);
const frameSide = new THREE.Mesh(new THREE.BoxGeometry(16, 10, 2), whFrameMat);
frameSide.position.set(whX, 7, -6); whGroup.add(frameSide);
const glassMat = new THREE.MeshPhysicalMaterial({ color: 0x88CCFF, transparent: true, opacity: 0.25, metalness: 0.9, roughness: 0.05, transmission: 0.8, side: THREE.DoubleSide });
const glass = new THREE.Mesh(new THREE.BoxGeometry(14.2, 10, 12.2), glassMat);
glass.position.set(whX + 1, 7, 1); whGroup.add(glass);
const rackMat = new THREE.MeshStandardMaterial({ color: 0xFF5500, metalness: 0.8, roughness: 0.2 });
for(let r = -3; r <= 5; r += 4) { for(let c = whX - 4; c <= whX + 4; c += 4) { const rack = new THREE.Mesh(new THREE.BoxGeometry(2.5, 9, 1.2), rackMat); rack.position.set(c, 6.5, r); whGroup.add(rack); } }
const tunnelMat = new THREE.MeshStandardMaterial({ map: createWarningTexture() }); const holeMat = new THREE.MeshBasicMaterial({ color: 0x000000 });
const lowTun = new THREE.Mesh(new THREE.BoxGeometry(3, 2.2, 2.5), tunnelMat); lowTun.position.set(START_X - 1.5, SURFACE_LOWER + 1.1, 0); whGroup.add(lowTun);
const lowHole = new THREE.Mesh(new THREE.BoxGeometry(3.1, 2.0, 2.0), holeMat); lowHole.position.set(START_X - 1.5, SURFACE_LOWER + 1.1, 0); whGroup.add(lowHole);
const upTun = new THREE.Mesh(new THREE.BoxGeometry(3, 2.2, 2.5), tunnelMat); upTun.position.set(START_X - 1.5, SURFACE_UPPER + 1.1, 0); whGroup.add(upTun);
const upHole = new THREE.Mesh(new THREE.BoxGeometry(3.1, 2.0, 2.0), holeMat); upHole.position.set(START_X - 1.5, SURFACE_UPPER + 1.1, 0); whGroup.add(upHole);
factoryGroup.add(whGroup);
factoryGroup.add(createBeltCore(BELT_LEN, 0x111827, SURFACE_LOWER, 0, true, centerX));
factoryGroup.add(createBeltCore(BELT_LEN, 0x111827, SURFACE_UPPER, 0, true, centerX));
const pMat = new THREE.MeshStandardMaterial({ color: 0x475569, metalness: 0.6 });
for(let i = START_X + 5; i < START_X + BELT_LEN - 10; i += 12) {
const p1 = new THREE.Mesh(new THREE.CylinderGeometry(0.1, 0.1, SURFACE_UPPER), pMat); p1.position.set(i, SURFACE_UPPER/2, 0.65); p1.castShadow=true; factoryGroup.add(p1);
const p2 = new THREE.Mesh(new THREE.CylinderGeometry(0.1, 0.1, SURFACE_UPPER), pMat); p2.position.set(i, SURFACE_UPPER/2, -0.65); p2.castShadow=true; factoryGroup.add(p2);
}
for (let i = 0; i < currentConfig.num_stations; i++) {
const x = START_X + posArray[i];
const branchBelt = createBeltCore(branchLen, 0x1E293B, SURFACE_LOWER, 0, false, 0);
branchBelt.rotation.y = Math.PI / 2; branchBelt.position.set(x, 0, 0.6 + branchLen / 2); factoryGroup.add(branchBelt);
const dY = SURFACE_UPPER - SURFACE_LOWER; const dZ = branchOutLen; const rLen = Math.sqrt(dY*dY + dZ*dZ); const rAngle = Math.atan2(dY, dZ);
const rampGroup = new THREE.Group(); rampGroup.position.set(x, SURFACE_LOWER + dY/2, 0.6 + dZ/2); rampGroup.rotation.x = rAngle;
const rampMesh = new THREE.Mesh(new THREE.BoxGeometry(1.2, 0.1, rLen), new THREE.MeshStandardMaterial({ color: 0x1E293B })); rampMesh.position.set(0, -0.05, 0); rampMesh.castShadow = true; rampMesh.receiveShadow = true; rampGroup.add(rampMesh);
const rrMat = new THREE.MeshStandardMaterial({ color: 0xFF6B00, metalness: 0.4 });
const rrF = new THREE.Mesh(new THREE.BoxGeometry(0.05, 0.2, rLen), rrMat); rrF.position.set(0.6, 0.05, 0); rampGroup.add(rrF);
const rrB = new THREE.Mesh(new THREE.BoxGeometry(0.05, 0.2, rLen), rrMat); rrB.position.set(-0.6, 0.05, 0); rampGroup.add(rrB); factoryGroup.add(rampGroup);
const deskMat = new THREE.MeshStandardMaterial({ color: 0xF8FAFC, metalness: 0.3 });
const desk = new THREE.Mesh(new THREE.BoxGeometry(2.0, 0.2, 1.2), deskMat); desk.position.set(x, SURFACE_LOWER - 0.1, branchLen + 1.2); desk.castShadow = true; desk.receiveShadow = true; factoryGroup.add(desk);
const screen = new THREE.Mesh(new THREE.PlaneGeometry(1.5, 0.8), new THREE.MeshStandardMaterial({ color: 0x00E5FF, emissive: 0x00E5FF, emissiveIntensity: 0.1, transparent: true, opacity: 0.8, side: THREE.DoubleSide })); screen.position.set(x, 0.8, branchLen + 1.9); screen.rotation.x = -Math.PI / 8; factoryGroup.add(screen);

```

```

    const light = new THREE.Mesh(new THREE.SphereGeometry(0.15), new THREE.MeshStandardMaterial({color: 0x00ffcc, emissive: 0x00aa88})); light.position.set(x - 1.0, 1.4, branchLen + 1.9); factoryGroup.add(light);
    const laser = new THREE.Mesh(new THREE.PlaneGeometry(1.2, 0.05), new THREE.MeshBasicMaterial({color: 0x00FF00, transparent: true, opacity: 0.8, side: THREE.DoubleSide})); laser.position.set(x, Y_LOWER + 0.25, branchLen + 1.2); laser.visible = false; factoryGroup.add(laser);
    const worker = new THREE.Group(); const mSkin = new THREE.MeshStandardMaterial({color: 0xFFB800}); const mCloth = new THREE.MeshStandardMaterial({color: 0x1E3A8A}); const mVest = new THREE.MeshStandardMaterial({color: 0x10B981});
    const leg = new THREE.Mesh(new THREE.CylinderGeometry(0.1, 0.1, 0.4), mCloth); leg.position.set(x-0.15, 0.2, branchLen + 2.3); worker.add(leg);
    const leg2 = new THREE.Mesh(new THREE.CylinderGeometry(0.1, 0.1, 0.4), mCloth); leg2.position.set(x+0.15, 0.2, branchLen + 2.3); worker.add(leg2);
    const torso = new THREE.Mesh(new THREE.BoxGeometry(0.5, 0.6, 0.3), mVest); torso.position.set(x, 0.7, branchLen + 2.3); worker.add(torso);
    const head = new THREE.Mesh(new THREE.SphereGeometry(0.15), mSkin); head.position.set(x, 1.15, branchLen + 2.3); worker.add(head);
    const hat = new THREE.Mesh(new THREE.CylinderGeometry(0.18, 0.18, 0.1), new THREE.MeshStandardMaterial({color: 0xFF0000})); hat.position.set(x, 1.25, branchLen + 2.3); worker.add(hat);
    const armG = new THREE.CylinderGeometry(0.06, 0.06, 0.5); const armL = new THREE.Mesh(armG, mVest); armL.position.set(x-0.3, 0.8, branchLen + 2.0); armL.rotation.x = -Math.PI/3; armL.rotation.z = -Math.PI/8; worker.add(armL);
    const armR = new THREE.Mesh(armG, mVest); armR.position.set(x+0.3, 0.8, branchLen + 2.0); armR.rotation.x = -Math.PI/3; armR.rotation.z = Math.PI/8; worker.add(armR); worker.castShadow = true; factoryGroup.add(worker);
    const stLabel = document.createElement('div'); stLabel.className = 'station-label'; stLabel.innerHTMLText = `S${(i+1).toString().padStart(2, '0')}`;
    stationsContainerRef.value.appendChild(stLabel);
    stations.push({
      id: i, x: x, screen: screen, labelDom: stLabel, laser: laser, light: light,
      desk: desk, worker: worker, isPoweredOff: false
    });
  }
}
const parsePlaybook = (playbook) => {
  try {
    activeBoxes.forEach(b => {
      scene.remove(b.mesh);
      if (b.label && b.label.parentNode) b.label.parentNode.removeChild(b.label);
    });
    activeBoxes.clear();
    if (!playbook || !playbook.timeline || playbook.timeline.length === 0) return;
    const timeline = playbook.timeline;
    const baseTimeMs = new Date(timeline[0].spawn_time || timeline[0].start_time).getTime();
    const uniqueOrders = new Set();
    orderStats = { total: 0, completed: 0, map: {} };
    const BELT_SPEED = currentConfig.belt_speed;
    simData = timeline.map(item => {
      uniqueOrders.add(item.order_id);
      const orderId = item.order_id;
      if(!orderStats.map[orderId]) { orderStats.map[orderId] = { totalBoxes: 0, boxesFinished: 0 };
    }
    orderStats.map[orderId].totalBoxes++;
    const tSpawn = (new Date(item.spawn_time || item.start_time).getTime() - baseTimeMs) / 1000;
    const tStart = (new Date(item.start_time).getTime() - baseTimeMs) / 1000;
    const tEnd = (new Date(item.end_time).getTime() - baseTimeMs) / 1000;
    const stIdx = item.target_station - 1;
    const eX = stations[stIdx].x;
    const distIn = Math.abs(eX - START_X);
    const distOut = Math.abs(eX - START_X);
    let tl = { start: tStart, return_main: tEnd };
    tl.spawn = tSpawn;
    tl.branch = tSpawn + (distIn / BELT_SPEED);
  }
}

```

```

const timeUpBranch = branchOutLen / BELT_SPEED;
tl.return_branch_end = tl.return_main + timeUpBranch;
const timeReturnMain = distOut / BELT_SPEED;
tl.exit = tl.return_branch_end + timeReturnMain;
return {
  box_id: item.box_id,
  stIdx: stIdx,
  timeline: tl,
  sku: item.sku || ""
};
});
const activeCount = playbook.active_stations || 16;
stations.forEach((st, idx) => {
  st.isPoweredOff = (idx >= activeCount);
  if (st.isPoweredOff) {
    st.worker.visible = false;
    st.desk.material.color.setHex(0x334155); st.desk.material.emissive.setHex(0x000000);
    st.screen.material.color.setHex(0x1e293b); st.light.material.color.setHex(0x1e293b);
    st.labelDom.style.color = '#475569'; st.labelDom.style.borderColor = '#475569'; st.labelDom
m.style.background = 'rgba(10, 15, 25, 0.9)';
    st.labelDom.innerText = `S${(st.id+1).toString().padStart(2, '0')}` (休眠);
  } else {
    st.worker.visible = true;
    st.desk.material.color.setHex(0xF8F8FC); st.desk.material.emissive.setHex(0x000000);
    st.screen.material.color.setHex(0x00E5FF);
    st.labelDom.style.color = '#00E5FF'; st.labelDom.style.borderColor = '#00E5FF'; st.labelD
om.style.background = 'rgba(10, 25, 45, 0.9)';
    st.labelDom.innerText = `S${(st.id+1).toString().padStart(2, '0')}`;
  }
});
orderStats.total = uniqueOrders.size;
let localMaxTime = 10;
for (let i=0; i<simData.length; i++) {
  if (simData[i].timeline.exit > localMaxTime) {
    localMaxTime = simData[i].timeline.exit;
  }
}
maxTime = localMaxTime + 0.5;
kpiData.ordersTotal = orderStats.total; kpiData.bboxesTotal = timeline.length; kpiData.maxTim
e = maxTime;
currentSimTime = 0; lastTime = 0;
} catch (e) {
  console.error(" 🚨 剧本解析发生错误: ", e);
}
}
const raycaster = new THREE.Raycaster();
const mouse = new THREE.Vector2();
const handleMouseMove = (event) => {
  if (!containerRef.value || !camera) return;
  const rect = containerRef.value.getBoundingClientRect();
  const clientX = event.clientX - rect.left;
  const clientY = event.clientY - rect.top;
  mouse.x = (clientX / rect.width) * 2 - 1;
  mouse.y = -(clientY / rect.height) * 2 + 1;
  raycaster.setFromCamera(mouse, camera);
  const boxMeshes = Array.from(activeBoxes.values()).map(b => b.mesh);
  const intersects = raycaster.intersectObjects(boxMeshes);
  if (intersects.length > 0) {
    const boxObj = intersects[0].object;
    const data = boxObj.userData;
    let orderId = "未知订单";
    let partType = "未知";
    if (data.id.includes('-P')) {
      const parts = data.id.split('-P');
      orderId = parts[0]; partType = parts[1];
    } else {

```

```

    orderId = data.id;
  }
  tooltipParams.orderId = orderId;
  tooltipParams.partType = data.sku || partType;
  tooltipParams.stName = `S${(data.station + 1).toString().padStart(2, '0')}`;
  tooltipParams.x = clientX + 15;
  tooltipParams.y = clientY + 15;
  tooltipParams.visible = true;
  document.body.style.cursor = 'pointer';
  activeBoxes.forEach(b => b.mesh.material.emissive.setHex(0x000000));
  boxObj.material.emissive.setHex(0x333333);
} else {
  tooltipParams.visible = false;
  document.body.style.cursor = 'default';
  activeBoxes.forEach(b => b.mesh.material.emissive.setHex(0x000000));
}
};
const updateSimulation = (delta) => {
  if (containerRef.value) {
    stations.forEach(s => {
      const pos = new THREE.Vector3(s.x, 3.5, branchLen + 1.2);
      pos.project(camera);
      if (pos.z < 1) {
        s.labelDom.style.display = 'block';
        s.labelDom.style.left = `${(pos.x * .5 + .5) * containerRef.value.clientWidth}px`;
        s.labelDom.style.top = `${(pos.y * -.5 + .5) * containerRef.value.clientHeight}px`;
      } else {
        s.labelDom.style.display = 'none';
      }
    });
  }
  if (!isPlaying.value || simData.length === 0) return;
  let dt = delta * playbackSpeed.value;
  currentSimTime += dt;
  if (currentSimTime > maxTime) currentSimTime = maxTime;
  currentTimeStr.value = currentSimTime.toFixed(1);
  timeSliderVal.value = currentSimTime;
  let finishedCount = 0; let activeCount = 0;
  let currentOrderProgress = JSON.parse(JSON.stringify(orderStats.map()));
  let stStatusCount = Array(currentConfig.num_stations).fill(0).map(() => ({ active: false, buffer
Boxes: 0, concurrentOrders: new Set() }));
  simData.forEach(boxData => {
    const t = currentSimTime, tl = boxData.timeline, stIdx = boxData.stIdx, eX = stations[stIdx].x;
    const orderId = boxData.box_id.split('-P')[0];
    if (t >= tl.exit) { finishedCount++; currentOrderProgress[orderId].boxesFinished++; }
    else if (t >= tl.spawn) { activeCount++; }
    if (t >= tl.branch && t < tl.return_main) { stStatusCount[stIdx].concurrentOrders.add(orderId);
  }
  if (t >= tl.branch && t < tl.start) { stStatusCount[stIdx].bufferBoxes++; }
  if (t >= tl.start && t < tl.return_main) {
    stStatusCount[stIdx].active = true;
    if (!stations[stIdx].isPoweredOff) {
      stations[stIdx].screen.material.emissiveIntensity = 1.0;
      stations[stIdx].laser.visible = true;
      stations[stIdx].laser.position.z = branchLen + Math.sin(t * 10) * 0.35;
      stations[stIdx].light.material.color.setHex(0x00E5FF);
      stations[stIdx].light.material.emissive.setHex(0x00E5FF);
    }
  }
  if (t >= tl.spawn && t <= tl.exit) {
    let boxObj = activeBoxes.get(boxData.box_id);
    if (!boxObj) {
      let shortName = "BOX";
      if (boxData.box_id.includes('-P')) {
        const rawSKU = boxData.box_id.split('-P').pop();
        shortName = rawSKU.length > 4 ? rawSKU.slice(-4) : rawSKU;

```

```

    }
    const boxColor = stringToColor(boxData.box_id);
    const mesh = new THREE.Mesh(new THREE.BoxGeometry(BOX_S, BOX_S, BOX_S),
new THREE.MeshStandardMaterial({ color: boxColor, emissive: 0x000000, roughness:0.2, met
alness:0.4 }));
    mesh.castShadow = true; mesh.receiveShadow = true;
    mesh.userData = {
      id: boxData.box_id,
      station: stIdx,
      sku: boxData.sku
    };
    scene.add(mesh);
    const label = document.createElement('div'); label.className = 'box-label'; label.innerTe
xt = "P_" + shortName;
    labelsContainerRef.value.appendChild(label);
    boxObj = { mesh: mesh, label: label, currentZ: 0 }; activeBoxes.set(boxData.box_id, box
Obj);
  }
  const box = boxObj.mesh; let targetX, targetY, targetZ;
  if (t < tl.branch) {
    targetX = lerp(START_X, eX, (t-tl.spawn)/(tl.branch-tl.spawn)); targetY = Y_LOWER; t
argetZ = 0; boxObj.currentZ = targetZ;
  } else if (t < tl.return_main) {
    targetX = eX; targetY = Y_LOWER;
    let limitZ;
    if (t < tl.start) {
      let boxesAhead = 0;
      for(let i=0; i<simData.length; i++) {
        if (simData[i].stIdx === stIdx
          && simData[i].box_id !== boxData.box_id
          && simData[i].timeline.branch < tl.branch
          && t < simData[i].timeline.return_main) {
          boxesAhead++;
        }
      }
      limitZ = boxesAhead === 0 ? branchLen : (branchLen - 1.2) - (boxesAhead - 1) * 0.8;
    } else {
    }
    if (boxObj.currentZ < limitZ) {
      boxObj.currentZ += dt * currentConfig.belt_speed;
      if (boxObj.currentZ > limitZ) boxObj.currentZ = limitZ;
    } else {
      boxObj.currentZ = limitZ;
    }
    targetZ = boxObj.currentZ;
  } else if (t < tl.return_branch_end) {
    let progress = (t - tl.return_main) / (tl.return_branch_end - tl.return_main);
    targetX = eX; targetY = lerp(Y_LOWER, Y_UPPER, progress); targetZ = lerp(branchLe
n, 0, progress); boxObj.currentZ = targetZ;
  } else {
    targetX = lerp(eX, START_X, (t - tl.return_branch_end) / (tl.exit - tl.return_branch_end));
    targetY = Y_UPPER; targetZ = 0; boxObj.currentZ = targetZ;
  }
  box.position.set(targetX, targetY, targetZ); boxObj.label.style.display = 'block';
  if (containerRef.value) {
    const screenPos = box.position.clone(); screenPos.y += 0.5; screenPos.project(camera);
    boxObj.label.style.left = `${(screenPos.x * .5 + .5) * containerRef.value.clientWidth}px`;
    boxObj.label.style.top = `${(screenPos.y * -.5 + .5) * containerRef.value.clientHeight}px`;
  }
} else {
  let b = activeBoxes.get(boxData.box_id);
  if (b) { scene.remove(b.mesh); if(b.label && b.label.parentNode) b.label.parentNode.remov
eChild(b.label); activeBoxes.delete(boxData.box_id); }
}
});
for(let i=0; i<currentConfig.num_stations; i++) {

```

```

    if (stations[i].isPoweredOff) continue;
    if (!stStatusCount[i].active) {
      stations[i].screen.material.emissiveIntensity = 0.1;
      stations[i].laser.visible = false;
      stations[i].light.material.color.setHex(0x00ffcc);
      stations[i].light.material.emissive.setHex(0x00aa88);
    }
  }
  let completedOrdersCount = 0;
  Object.values(currentOrderProgress).forEach(order => { if (order.bboxesFinished === order.totalBboxes) completedOrdersCount++; });
  kpiData.ordersDone = completedOrdersCount;
  kpiData.bboxesDone = finishedCount;
  kpiData.bboxesActive = activeBboxes.size;
  kpiData.progressPct = simData.length > 0 ? ((finishedCount / simData.length) * 100).toFixed(1) : 0;
  kpiData.currentTime = currentSimTime;
  emit('update-kpi', kpiData);
  const statusArray = stStatusCount.map((st, idx) => ({
    active: st.active,
    orderCount: st.concurrentOrders.size,
    maxOrders: 2,
    isPoweredOff: stations[idx].isPoweredOff
  }));
  emit('update-stations', statusArray);
  if (finishedCount === simData.length && simData.length > 0) isPlaying.value = false;
  lastTime = currentSimTime;
}
const animate = () => {
  animationId = requestAnimationFrame(animate)
  const delta = clock.getDelta()
  updateSimulation(delta)
  controls.update()
  renderer.render(scene, camera)
}
const handleResize = () => {
  camera.aspect = containerRef.value.clientWidth / containerRef.value.clientHeight
  camera.updateProjectionMatrix()
  renderer.setSize(containerRef.value.clientWidth, containerRef.value.clientHeight)
}
onMounted(() => {
  initScene(); animate(); window.addEventListener('resize', handleResize);
  if (containerRef.value) { containerRef.value.addEventListener('mousemove', handleMouseMove); }
})
onBeforeUnmount(() => {
  window.removeEventListener('resize', handleResize);
  if (containerRef.value) { containerRef.value.removeEventListener('mousemove', handleMouseMove); }
  cancelAnimationFrame(animationId); renderer.dispose()
})
</script>
<template>
  <div class="factory-container" ref="containerRef">
    <div ref="labelsContainerRef" class="labels-layer"></div>
    <div ref="stationsContainerRef" class="labels-layer"></div>
    <div class="custom-tooltip" v-show="tooltipParams.visible" :style="{ left: tooltipParams.x + 'px', top: tooltipParams.y + 'px' }">
      <div style="border-bottom: 1px solid #00E5FF; margin-bottom: 10px; padding-bottom: 6px; font-size: 14px; text-shadow: 0 0 5px #00E5FF;">
        <strong>📦 物理物料箱档案</strong>
      </div>
      <div class="tooltip-row"><span class="tooltip-label">归属母订单</span><span class="tooltip-value highlight-text">{{ tooltipParams.orderId }}</span></div>
      <div class="tooltip-row"><span class="tooltip-label">子零件型号</span><span class="tooltip-value">P-{{ tooltipParams.partType }}</span></div>
    </div>
  </div>

```

```

    <div class="tooltip-row" style="margin-top:8px; border-top:1px dashed #3A5A7A; padding-top:8px;"><span class="tooltip-label">目标加工站</span><span class="tooltip-value" style="color:#00FF00;">🔴 {{ tooltipParams.stName }}</span></div>
  </div>
  <div class="time-overlay" v-if="simData.length > 0">
    <div class="time-value">T+ {{ currentTimeStr }} s</div>
    <div class="time-target">/ {{ maxTime.toFixed(1) }} s</div>
  </div>
  <div class="playback-controls">
    <button class="ctrl-btn play-btn" @click="togglePlay" :disabled="simData.length === 0">
      {{ isPlaying ? '⏸ 暂停' : '▶ 播放' }}</button>
    <input type="range" class="time-slider" min="0" :max="maxTime" :value="timeSliderVal" disabled />
    <div class="speed-group">
      <select class="speed-select" v-model="playbackSpeed" @change="setSpeed($event.target.value)">
        <option :value="1">1x 实时</option>
        <option :value="5">5x 倍速</option>
        <option :value="10">10x 极速</option>
        <option :value="20">20x 超神</option>
      </select>
    </div>
  </div>
  <div class="empty-overlay" v-if="simData.length === 0">
    <div class="icon">🏭</div>
    <div>数字孪生物理引擎就绪，等待注入排产剧本...</div>
  </div>
</div>
</template>
<style>
.box-label { position: absolute; color:
.station-label { position: absolute; color:
</style>
<style scoped>
.factory-container { width: 100%; height: 100%; position: relative; background: radial-gradient(circle at center,
.labels-layer { position: absolute; top: 0; left: 0; width: 100%; height: 100%; pointer-events: none; z-index: 10; }
.custom-tooltip { position: absolute; background: rgba(15, 25, 45, 0.95); color:
.tooltip-row { margin-bottom: 6px; display: flex; justify-content: space-between; }
.tooltip-label { color:
.tooltip-value { font-weight: 900; color:
.highlight-text { color:
.empty-overlay { position: absolute; top: 0; left: 0; width: 100%; height: 100%; display: flex; flex-direction: column; justify-content: center; align-items: center; color:
.empty-overlay .icon { font-size: 40px; margin-bottom: 10px; filter: grayscale(1); opacity: 0.5; }
.time-overlay { position: absolute; top: 20px; left: 20px; background: rgba(10, 25, 45, 0.85); border: 1px solid
.time-value { color:
.time-target { color:
.playback-controls { position: absolute; bottom: 25px; left: 50%; transform: translateX(-50%); background: rgba(15, 25, 40, 0.95); padding: 12px 25px; border-radius: 40px; z-index: 100; display: flex; align-items: center; gap: 15px; border: 1px solid
.ctrl-btn { background: linear-gradient(135deg,
.ctrl-btn:hover:not(:disabled) { transform: scale(1.05); }
.ctrl-btn:disabled { opacity: 0.5; cursor: not-allowed; }
.time-slider { flex-grow: 1; accent-color:
.speed-select { background:#0F1E32; color:#fff; border:1px solid
</style>
<script setup>
import { ref, watch, onMounted, onUnmounted, nextTick } from 'vue'
import * as echarts from 'echarts'
import api from '../api/index'
const props = defineProps({
  simResult: { type: Object, default: () => null }
})

```

```

const compareChartRef = ref(null)
const curveChartRef = ref(null)
let compareChartInstance = null
let curveChartInstance = null
const renderHolographicFallback = (msg, color) => {
  const mockData = Array.from({length: 50}, () => Math.random() * 10 + 20)
  curveChartInstance.setOption({
    backgroundColor: 'transparent',
    title: { text: msg, textStyle: { color: color, fontSize: 14, fontWeight: 'normal', lineHeight: 24 },
    left: 'center', top: 'center', z: 10 },
    xAxis: { show: false, data: Array.from({length: 50}, (_, i) => i) },
    yAxis: { show: false, min: 0, max: 40 },
    series: [{ type: 'line', data: mockData, smooth: true, lineStyle: { color: color, width: 1, type: 'dashed', opacity: 0.3 }, areaStyle: { color: color, opacity: 0.05 }, symbol: 'none' }]
  }, true)
}
const renderCurveChart = async () => {
  if (!curveChartRef.value) return
  if (!curveChartInstance) curveChartInstance = echarts.init(curveChartRef.value)
  curveChartInstance.showLoading({ text: '正在扫描硬盘读取最新 TensorBoard 日志...', color: '#00E5FF', textColor: '#A0B2C6', maskColor: 'rgba(10, 21, 37, 0.8)' })
  try {
    const res = await api.getTrainingMetrics()
    if (res && res.code === 200 && res.data) {
      const steps = res.data.steps
      const rewards = res.data.rewards
      const stableRewards = rewards.slice(Math.floor(rewards.length * 0.25))
      const minStable = Math.min(...stableRewards)
      const maxStable = Math.max(...stableRewards)
      const yRange = maxStable - minStable || 100
      const calcMin = Math.floor(minStable - yRange * 0.15)
      const option2 = {
        backgroundColor: 'transparent',
        title: { text: 'AI 调度大脑 (PPO) 真实训练收敛曲线', textStyle: { color: '#00E5FF', fontSi
ze: 16 }, left: 'center', top: 10 },
        tooltip: { trigger: 'axis', formatter: '{b}<br/>Reward 分数: {c}' },
        dataZoom: [
          { type: 'inside', xAxisIndex: 0, filterMode: 'filter' },
          { type: 'slider', xAxisIndex: 0, bottom: 5, height: 15, textStyle: { color: '#A0B2C6' } },
          { type: 'slider', yAxisIndex: 0, right: 5, width: 15, textStyle: { color: '#A0B2C6' } }
        ],
        grid: { left: '12%', right: '8%', bottom: '20%', top: '25%' },
        xAxis: { type: 'category', boundaryGap: false, data: steps, axisLabel: { color: '#fff' } },
        yAxis: {
          type: 'value',
          name: 'Reward (稳态缩放)',
          axisLabel: { color: '#fff' },
          nameTextStyle: { color: '#A0B2C6' },
          splitLine: { lineStyle: { color: '#1E3A5F', type: 'dashed' } },
          min: calcMin,
          scale: true
        },
        series: [
          {
            name: 'Reward',
            type: 'line',
            smooth: 0.3,
            data: rewards,
            clip: true,
            itemStyle: { color: '#67C23A' },
            lineStyle: { width: 3, shadowColor: 'rgba(103, 194, 58, 0.5)', shadowBlur: 10 },
            areaStyle: { color: new echarts.graphic.LinearGradient(0, 0, 0, 1, [{ offset: 0, color: 'rgba(103, 194, 58, 0.4)' }, { offset: 1, color: 'rgba(103, 194, 58, 0)' }]) }
          }
        ]
      }
    }
  }
}

```



```

    curveChartInstance.hideLoading()
    curveChartInstance.setOption(option2, true)
  } else {
    curveChartInstance.hideLoading()
    renderHolographicFallback(⚠ 曲线渲染中止: 训练日志未挂载`, '#FFB800')
  }
} catch (error) {
  curveChartInstance.hideLoading()
  renderHolographicFallback(✖ 接口通信失败: 无法连接至后端获取日志`, '#FF4D4F')
}
}
const renderCharts = () => {
  if (!props.simResult) return
  if (compareChartRef.value) {
    if (!compareChartInstance) compareChartInstance = echarts.init(compareChartRef.value)
    const data = props.simResult
    const option1 = {
      backgroundColor: 'transparent',
      title: { text: '多算法极限压榨比对测试', textStyle: { color: '#00E5FF', fontSize: 16 }, left: 'center', top: 10 },
      tooltip: { trigger: 'axis', axisPointer: { type: 'shadow' } },
      legend: { textStyle: { color: '#fff' }, top: 40 },
      grid: { left: '12%', right: '12%', bottom: '15%', top: '25%' },
      xAxis: { data: ['AI 强化学习', '传统轮询', '随机乱派'], axisLabel: { color: '#fff', fontSize: 14 } },
      yAxis: [
        { type: 'value', name: '总完工耗时(s)', axisLabel: { color: '#fff' }, nameTextStyle: { color: '#A0B2C6' }, splitLine: { lineStyle: { color: '#1E3A5F', type: 'dashed' } } },
        { type: 'value', name: '必需开机数(台)', min: 0, max: 16, axisLabel: { color: '#fff' }, nameTextStyle: { color: '#A0B2C6' }, splitLine: { show: false } }
      ],
      series: [
        {
          name: '总耗时', type: 'bar', barWidth: '35%',
          data: [
            { value: data.ai_result?.total_makespan || 0, itemStyle: { color: new echarts.graphic.LinearGradient(0, 0, 0, 1, [{ offset: 0, color: '#00E5FF' }, { offset: 1, color: '#0077FF' }]) } },
            { value: data.trad_result?.total_makespan || 0, itemStyle: { color: new echarts.graphic.LinearGradient(0, 0, 0, 1, [{ offset: 0, color: '#4A90E2' }, { offset: 1, color: '#003366' }]) } },
            { value: data.rand_result?.total_makespan || 0, itemStyle: { color: new echarts.graphic.LinearGradient(0, 0, 0, 1, [{ offset: 0, color: '#9CA3AF' }, { offset: 1, color: '#4B5563' }]) } }
          ],
          itemStyle: { borderRadius: [6, 6, 0, 0] }
        },
        {
          name: '所需机床', type: 'line', yAxisIndex: 1,
          data: [data.ai_result?.active_stations || 0, data.trad_result?.active_stations || 0, data.rand_result?.active_stations || 0],
          itemStyle: { color: '#FFB800' }, lineStyle: { width: 4, shadowColor: 'rgba(255, 184, 0, 0.5)', shadowBlur: 10 }, symbol: 'diamond', symbolSize: 12
        }
      ]
    }
    compareChartInstance.setOption(option1, true)
  }
  renderCurveChart()
}
watch(() => props.simResult, async (newVal) => {
  if (newVal) { await nextTick(); renderCharts() }
}, { immediate: true })
const handleResize = () => { compareChartInstance?.resize(); curveChartInstance?.resize() }
onMounted(() => { window.addEventListener('resize', handleResize); if (!props.simResult) renderCurveChart() })
onUnmounted(() => { window.removeEventListener('resize', handleResize); compareChartInstance?.dispose(); curveChartInstance?.dispose() })
</script>

```

```

<template>
  <div class="algo-container">
    <div v-if="!simResult" class="empty-state">
      <h2>等待后台 AI 推演战报...</h2>
      <p>请在【3D 数字孪生】面板点击“启动”按钮，获取极限压榨数据。</p>
      <div style="height: 300px; width: 100%; margin-top: 30px;" ref="curveChartRef"></div>
    </div>
    <div v-else class="charts-wrapper">
      <div class="summary-cards">
        <div class="card"><div class="card-title">AI 极限完工耗时</div><div class="card-value ai-color">{{ simResult.ai_result?.total_makespan || 0 }} <span class="unit">秒</span></div></div>
        <div class="card"><div class="card-title">为您节省实体机床</div><div class="card-value e highlight">{{ Math.max(0, (simResult.trad_result?.active_stations || 16) - (simResult.ai_result?.active_stations || 16)) }} <span class="unit">台</span></div></div>
        <div class="card"><div class="card-title">综合效率提升</div><div class="card-value highlight">{{ simResult.efficiency_up || '0%' }}</div></div>
      </div>
      <div class="inventory-cards">
        <div class="mini-card"><span>较轮询效率提升</span><strong>{{ simResult.efficiency_vs_round_robin || simResult.efficiency_up || '0%' }}</strong></div>
        <div class="mini-card"><span>较随机效率提升</span><strong>{{ simResult.efficiency_vs_random || '0%' }}</strong></div>
        <div class="mini-card"><span>可执行订单</span><strong>{{ simResult.inventory_result?.preprocess_stats?.processable_order_count || 0 }}</strong></div>
        <div class="mini-card"><span>异常订单</span><strong>{{ simResult.inventory_result?.exception_order_count || 0 }}</strong></div>
        <div class="mini-card"><span>稀缺 SKU</span><strong>{{ simResult.inventory_result?.preprocess_stats?.scarce_sku_count || 0 }}</strong></div>
        <div class="mini-card"><span>重排订单</span><strong>{{ simResult.inventory_result?.preprocess_stats?.reordered_count || 0 }}</strong></div>
      </div>
      <div class="charts-grid">
        <div ref="compareChartRef" class="chart-box"></div>
        <div ref="curveChartRef" class="chart-box"></div>
      </div>
    </div>
  </div>
</template>
<style scoped>
.algo-container { width: 100%; height: 100%; background:
.empty-state { text-align: center; color:
.empty-state h2 { color:
.charts-wrapper { width: 100%; height: 100%; padding: 20px; display: flex; flex-direction: column; box-sizing: border-box; }
.summary-cards { display: flex; justify-content: space-between; gap: 20px; margin-bottom: 20px; height: 110px; }
.inventory-cards { display: grid; grid-template-columns: repeat(4, 1fr); gap: 12px; margin-bottom: 16px; }
.mini-card { background: rgba(15, 25, 40, 0.72); border: 1px solid
.mini-card span { color:
.mini-card strong { color:
.card { flex: 1; background: rgba(15, 25, 40, 0.8); border: 1px solid
.card::before { content: ""; position: absolute; top: 0; left: 0; width: 4px; height: 100%; background:
.card:nth-child(2)::before { background:
.card:nth-child(3)::before { background:
.card-title { color:
.card-value { font-size: 36px; font-weight: 900; text-shadow: 0 0 20px rgba(255,255,255,0.1); }
.unit { font-size: 14px; font-weight: normal; color:
.ai-color { color:
.highlight { color:
.charts-grid { display: flex; gap: 20px; flex: 1; min-height: 0; }
.chart-box { flex: 1; background: rgba(15, 25, 40, 0.5); border: 1px solid
</style>
:root {

```

```
--text:
--text-h:
--bg:
--border:
--code-bg:
--accent:
--accent-bg: rgba(170, 59, 255, 0.1);
--accent-border: rgba(170, 59, 255, 0.5);
--social-bg: rgba(244, 243, 236, 0.5);
--shadow:
  rgba(0, 0, 0, 0.1) 0 10px 15px -3px, rgba(0, 0, 0, 0.05) 0 4px 6px -2px;
--sans: system-ui, 'Segoe UI', Roboto, sans-serif;
--heading: system-ui, 'Segoe UI', Roboto, sans-serif;
--mono: ui-monospace, Consolas, monospace;
font: 18px/145% var(--sans);
letter-spacing: 0.18px;
color-scheme: light dark;
color: var(--text);
background: var(--bg);
font-synthesis: none;
text-rendering: optimizeLegibility;
-webkit-font-smoothing: antialiased;
-moz-osx-font-smoothing: grayscale;
@media (max-width: 1024px) {
  font-size: 16px;
}
}
}
@media (prefers-color-scheme: dark) {
  :root {
    --text:
    --text-h:
    --bg:
    --border:
    --code-bg:
    --accent:
    --accent-bg: rgba(192, 132, 252, 0.15);
    --accent-border: rgba(192, 132, 252, 0.5);
    --social-bg: rgba(47, 48, 58, 0.5);
    --shadow:
      rgba(0, 0, 0, 0.4) 0 10px 15px -3px, rgba(0, 0, 0, 0.25) 0 4px 6px -2px;
  }
  social .button-icon {
    filter: invert(1) brightness(2);
  }
}
body {
  margin: 0;
}
h1,
h2 {
  font-family: var(--heading);
  font-weight: 500;
  color: var(--text-h);
}
h1 {
  font-size: 56px;
  letter-spacing: -1.68px;
  margin: 32px 0;
  @media (max-width: 1024px) {
    font-size: 36px;
    margin: 20px 0;
  }
}
h2 {
  font-size: 24px;
  line-height: 118%;
}
```

```
letter-spacing: -0.24px;
margin: 0 0 8px;
@media (max-width: 1024px) {
  font-size: 20px;
}
}
p {
  margin: 0;
}
code,
.counter {
  font-family: var(--mono);
  display: inline-flex;
  border-radius: 4px;
  color: var(--text-h);
}
code {
  font-size: 15px;
  line-height: 135%;
  padding: 4px 8px;
  background: var(--code-bg);
}
.counter {
  font-size: 16px;
  padding: 5px 10px;
  border-radius: 5px;
  color: var(--accent);
  background: var(--accent-bg);
  border: 2px solid transparent;
  transition: border-color 0.3s;
  margin-bottom: 24px;
  &:hover {
    border-color: var(--accent-border);
  }
  &:focus-visible {
    outline: 2px solid var(--accent);
    outline-offset: 2px;
  }
}
}
.hero {
  position: relative;
  .base,
  .framework,
  .vite {
    inset-inline: 0;
    margin: 0 auto;
  }
  .base {
    width: 170px;
    position: relative;
    z-index: 0;
  }
  .framework,
  .vite {
    position: absolute;
  }
  .framework {
    z-index: 1;
    top: 34px;
    height: 28px;
    transform: perspective(2000px) rotateZ(300deg) rotateX(44deg) rotateY(39deg)
      scale(1.4);
  }
  .vite {
    z-index: 0;
    top: 107px;
```

```
height: 26px;
width: auto;
transform: perspective(2000px) rotateZ(300deg) rotateX(40deg) rotateY(39deg)
  scale(0.8);
}
}
app {
width: 1126px;
max-width: 100%;
margin: 0 auto;
text-align: center;
border-inline: 1px solid var(--border);
min-height: 100svh;
display: flex;
flex-direction: column;
box-sizing: border-box;
}
center {
display: flex;
flex-direction: column;
gap: 25px;
place-content: center;
place-items: center;
flex-grow: 1;
@media (max-width: 1024px) {
padding: 32px 20px 24px;
gap: 18px;
}
}
next-steps {
display: flex;
border-top: 1px solid var(--border);
text-align: left;
& > div {
flex: 1 1 0;
padding: 32px;
@media (max-width: 1024px) {
padding: 24px 20px;
}
}
}
.icon {
margin-bottom: 16px;
width: 22px;
height: 22px;
}
@media (max-width: 1024px) {
flex-direction: column;
text-align: center;
}
}
docs {
border-right: 1px solid var(--border);
@media (max-width: 1024px) {
border-right: none;
border-bottom: 1px solid var(--border);
}
}
next-steps ul {
list-style: none;
padding: 0;
display: flex;
gap: 8px;
margin: 32px 0 0;
.logo {
height: 18px;
}
}
```

```

a {
  color: var(--text-h);
  font-size: 16px;
  border-radius: 6px;
  background: var(--social-bg);
  display: flex;
  padding: 6px 12px;
  align-items: center;
  gap: 8px;
  text-decoration: none;
  transition: box-shadow 0.3s;
  &:hover {
    box-shadow: var(--shadow);
  }
  .button-icon {
    height: 18px;
    width: 18px;
  }
}
@media (max-width: 1024px) {
  margin-top: 20px;
  flex-wrap: wrap;
  justify-content: center;
  li {
    flex: 1 1 calc(50% - 8px);
  }
  a {
    width: 100%;
    justify-content: center;
    box-sizing: border-box;
  }
}
}
spacer {
  height: 88px;
  border-top: 1px solid var(--border);
  @media (max-width: 1024px) {
    height: 48px;
  }
}
}
.ticks {
  position: relative;
  width: 100%;
  &::before,
  &::after {
    content: "";
    position: absolute;
    top: -4.5px;
    border: 5px solid transparent;
  }
  &::before {
    left: 0;
    border-left-color: var(--border);
  }
  &::after {
    right: 0;
    border-right-color: var(--border);
  }
}
}
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'
export default defineConfig({
  plugins: [vue()],
  server: {
    port: 5173,
    proxy: {

```

```
    '/api': {  
      target: 'http://127.0.0.1:8088',  
      changeOrigin: true,  
    }  
  }  
})
```